In [1]:
```
# Initialize Otter
import otter
grader = otter.Notebook("cs1090a_hw3.ipynb")
```



HARVARD
John A. Paulson
School of Engineering
and Applied Sciences

CS1090A Introduction to Data Science

Homework 3: How much should you trust the model?

Regularization, bootstrap inference, missing data, and principal components

Harvard University
Fall 2026
Instructors: Pavlos Protopapas, Kevin Rader, and Chris Gumb

Instructions

CS1090A is graded out of 100 points. AC209A students complete the two marked extensions and are graded out of 110.

Run the import cell before beginning, then run each supplied data or settings cell when you reach it. These cells appear beside the questions that use them. Your submitted notebook must run in order after restarting the kernel, using the supplied local files. Include visible outputs, labeled plots, and written answers, and follow the Canvas submission instructions.

Use the supplied partitions and keep test data reserved until the final evaluation questions.

Use of AI tools

You may use AI tools. You are accountable for everything you submit: you must understand it and be able to explain it.

- Written answers must be your own words.
- Quizzes and midterms ask you to explain and apply these methods without a tool.

All plots must have axis labels, a descriptive title, and a legend when colors, symbols, or reference lines distinguish information. Include units where applicable.

Written-response requests under **Constraints** are required parts of your answer, as are those listed under **Task** and **Artifacts**.

In [2]:
```
import os, time, warnings
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display
from sklearn.base import clone
from sklearn.linear_model import LinearRegression, Ridge, Lasso, lasso_path
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from scipy.stats import norm
from threadpoolctl import threadpool_limits
from hw3_support import (
    CORE,
    FrontageImputer,
    AmesPreprocessor,
    reference_pipeline,
    load_ames,
    fixed_folds,
    bootstrap_folds,
    support_checkpoint,
    checked_ols,
    bootstrap_coefficient_maps,
    load_cookie,
    load_cookie_test,
)

from hw3_cache import cached_bootstrap_results

thread_limit = threadpool_limits(limits=1)
notebook_start = time.time()
```

Part 1. Missingness and imputation

We return to the Ames housing data from Homework 2 to predict the natural logarithm of sale price. Before fitting regression models, investigate the missing measurements and decide how to replace them. The supplied loading and preparation code lets you begin with that investigation without repeating the earlier data-cleaning exercises.

We will use three fixed, disjoint groups of houses throughout the Ames questions. The supplied cell below loads the data and creates the variables listed here; run it before Q1.1.

Group	Variables	Houses	Purpose
Selection	`X_selection`, `y_selection`	1,318	Investigate missingness, tune models using cross-validation, and select predictors. Use these houses for Part 1.

Group	Variables	Houses	Purpose
Inference	`X_inference`, `y_inference`	879	Estimate the selected coefficients and assess uncertainty and predictive importance in Part 3. Keep these outcomes out of predictor selection.
Test	`partitions['test']`	733	Evaluate the frozen final procedures in Q3.8. Do not inspect or use these houses in the earlier analyses.

Each `X_...` table contains recorded predictors; its matching `y_...` array contains the natural logarithm of sale price, in the same row order. The selection and inference groups together form the non-test training data. Reserving fresh inference outcomes lets us assess coefficients without reusing the price evidence that selected their predictors; Part 3 explains that distinction in more detail.

`raw_ames` preserves the source records, including recorded zeros. `ames_x` contains the predictor columns prepared by the loader, with unknown frontage marked as missing. `selection_ids` contains the row labels of the houses in the selection partition; `inference_ids` does the same for the inference partition. These labels let us retrieve the corresponding original records from `raw_ames`. In Q1.1, use `raw_ames.loc[selection_ids]` for recorded-zero counts and `X_selection` for missing-value counts. Later Part 1 questions also use the selection houses. The fixed splits and helper definitions ensure everyone can work from the same starting data.

▶ **Column descriptions** (click to expand)

In [3]:
```
# Supplied Ames loading and fixed partitions; see the table above.
raw_ames, ames_x, partitions, prep_template = load_ames()
selection_ids = partitions["selection"]
inference_ids = partitions["inference"]
X_selection = ames_x.loc[selection_ids].copy()
y_selection = np.log(raw_ames.loc[selection_ids, "Sale_Price"].to_numpy())
X_inference = ames_x.loc[inference_ids].copy()
y_inference = np.log(raw_ames.loc[inference_ids, "Sale_Price"].to_numpy())
# Cross-validation indices are positions within the selection tables, not row label
selection_folds = fixed_folds(len(X_selection))
print({name: len(ids) for name, ids in partitions.items()})
```

```
{'selection': 1318, 'inference': 879, 'test': 733}
```

Q1.1. Audit recorded measurements

3 points

Before we begin modeling prices, we need to understand what information is available about each house. A recorded zero can describe an absent amenity, such as a pool, or stand

in for an unknown measurement. These meanings call for different treatment: replacing a genuine zero would change the recorded characteristics of the house.

Task: Use `raw_ames.loc[selection_ids]` to count recorded zeros, and `X_selection` to count missing predictor values. Report predictors with at least one blank or zero. Confirm the meanings of zeros in `Lot_Frontage`, `Garage_Area`, and `Pool_Area` using related columns and the Ames data documentation. Calculate how many selection houses remain if every row with a missing predictor is deleted.

Artifacts: A table showing missing-value and zero counts by predictor; the number and proportion of selection houses retained by complete-case deletion; and a short explanation of which values represent unknown measurements versus absent amenities.

Constraints: Preserve raw missingness; distinguish unknown frontage from genuine zeros. Do not repeat Homework 2 encoding or feature engineering.

Output names for the guided checks: `audit` (a missingness table) and `complete_rows` (the retained-row count).

The supplied loader preserves literal `None` amenity categories and marks zero frontage as unknown in `X_selection`. In the supplied selection rows, frontage is the only predictor with missing values; this does not make zeros in other predictors missing.

The supplied CSV has basement-type codes in `BsmtFin_SF_1`, rather than area measurements. The loader excludes this mislabeled duplicate; other basement measurements remain.

In [4]:
```
# Q1.1: recorded zeros come from the raw records; blanks come from the prepared pre
raw_selection = raw_ames.loc[selection_ids]
raw_numeric = raw_selection.drop(columns="Sale_Price").select_dtypes("number")
audit = pd.DataFrame(
    {
        "missing": X_selection.isna().sum(),
        "zeros": (raw_numeric == 0).sum().reindex(X_selection.columns),
    }
).fillna(0).astype(int)
audit["missing_share"] = (audit["missing"] / len(X_selection)).round(3)
audit["zero_share"] = (audit["zeros"] / len(X_selection)).round(3)
audit = audit[(audit["missing"] > 0) | (audit["zeros"] > 0)].sort_values(
    ["missing", "zeros"], ascending=False
)
audit.index.name = "predictor"
display(audit)

# Check what a zero means for three columns, using related recorded columns.
print("Garage_Area == 0 versus Garage_Cars == 0 and Garage_Type:")
display(
    pd.crosstab(
        raw_selection["Garage_Area"] == 0,
        raw_selection["Garage_Type"],
```

```
        rownames=["Garage_Area == 0"],
    )
)
print(
    "Houses with Garage_Area == 0 but Garage_Cars > 0:",
    int(((raw_selection.Garage_Area == 0) & (raw_selection.Garage_Cars > 0)).sum())
)
print("Pool_Area == 0 versus Pool_QC:")
display(
    pd.crosstab(
        raw_selection["Pool_Area"] == 0,
        raw_selection["Pool_QC"],
        rownames=["Pool_Area == 0"],
    )
)
frontage_zero = raw_selection["Lot_Frontage"] == 0
print(
    "Lot_Frontage == 0:", int(frontage_zero.sum()),
    "| of these, lots with Lot_Area > 0:", int((raw_selection.loc[frontage_zero, "L
    "| marked missing in X_selection:", int(X_selection.Lot_Frontage.isna().sum()),
)
print("Lot_Config of houses with zero frontage:")
display(raw_selection.loc[frontage_zero, "Lot_Config"].value_counts().to_frame("hou

# Complete-case deletion: drop every row with any missing predictor.
complete_rows = int(X_selection.dropna().shape[0])
print(
    f"Complete-case deletion keeps {complete_rows} of {len(X_selection)} selection
    f"({complete_rows / len(X_selection):.1%}); it drops {len(X_selection) - comple
)
```

predictor	missing	zeros	missing_share	zero_share
Lot_Frontage	206	206	0.156	0.156
Pool_Area	0	1310	0.000	0.994
Low_Qual_Fin_SF	0	1302	0.000	0.988
Three_season_porch	0	1301	0.000	0.987
Misc_Val	0	1277	0.000	0.969
Bsmt_Half_Bath	0	1241	0.000	0.942
Screen_Porch	0	1196	0.000	0.907
BsmtFin_SF_2	0	1159	0.000	0.879
Enclosed_Porch	0	1122	0.000	0.851
Half_Bath	0	828	0.000	0.628
Mas_Vnr_Area	0	775	0.000	0.588
Bsmt_Full_Bath	0	774	0.000	0.587
Second_Flr_SF	0	739	0.000	0.561
Wood_Deck_SF	0	686	0.000	0.520
Fireplaces	0	636	0.000	0.483
Open_Porch_SF	0	575	0.000	0.436
Bsmt_Unf_SF	0	111	0.000	0.084
Garage_Cars	0	81	0.000	0.061
Garage_Area	0	81	0.000	0.061
Total_Bsmt_SF	0	37	0.000	0.028
Full_Bath	0	5	0.000	0.004
Kitchen_AbvGr	0	3	0.000	0.002
Bedroom_AbvGr	0	1	0.000	0.001

`Garage_Area == 0 versus Garage_Cars == 0 and Garage_Type:`

Garage_Type Garage_Area == 0	Attchd	Basment	BuiltIn	CarPort	Detchd	More_Than_Two_Types	No_Garage
False	786	14	80	6	343	8	0
True	0	0	0	0	0	0	81

```
Houses with Garage_Area == 0 but Garage_Cars > 0: 0
Pool_Area == 0 versus Pool_QC:
```

Pool_QC	Excellent	Fair	Good	No_Pool	Typical
Pool_Area == 0					
False	2	2	2	0	2
True	0	0	0	1310	0

```
Lot_Frontage == 0: 206 | of these, lots with Lot_Area > 0: 206 | marked missing in X
_selection: 206
Lot_Config of houses with zero frontage:
```

	houses
Lot_Config	
Inside	126
Corner	41
CulDSac	35
FR2	4

```
Complete-case deletion keeps 1112 of 1318 selection houses (84.4%); it drops 206.
```

In [5]:
```
grader.check("q1_1")
```

Out[5]:

q1_1 passed! 🌈

Written response — Q1.1

Lot frontage is the only predictor with blanks: 206 selection houses (15.6%) record a frontage of 0 and the loader marks them missing. Complete-case deletion would keep 1,112 of 1,318 houses (84.4%).

Unknown, not zero. All 206 zero-frontage lots have positive lot area, and 80 of them are corner, cul-de-sac or FR2 (frontage on two sides) lots, which by definition touch a street, so a recorded 0 is a placeholder, not a measurement. We treat these zeros as unknown and impute them.

Absent, not unknown. All 81 houses with `Garage_Area == 0` also have `Garage_Cars == 0` and `Garage_Type == "No_Garage"`, and no house with cars has zero area. All 1,310 houses with `Pool_Area == 0` have `Pool_QC == "No_Pool"`, and the 8 pools have positive area. These zeros record that the amenity does not exist, so replacing them would invent amenities. Zeros in other amenity columns (porches, second floors, veneer, basement baths) most plausibly mean the same.

Q1.2. Investigate the consequences of deletion

3 points

The audit shows which houses we would lose if we simply dropped rows with missing measurements. Deletion reduces the sample size, but it can also change which kinds of houses are represented. If missingness is more common in particular neighborhoods or types of properties, the remaining data may give a distorted picture of the population we want to model.

Task: Relate missing-frontage indicators to neighborhood and other recorded lot characteristics, displaying group sizes. Interpret evidence relevant to complete-case deletion.

Artifacts: A neighborhood summary showing group sizes and missing-frontage rates; one comparison with a recorded lot characteristic; and a short explanation of whether deleting incomplete houses would change the represented population.

Constraints: Use selection data. Patterns can challenge MCAR but do not conclusively distinguish MAR from MNAR.

In [6]:
```
# Q1.2: does unknown frontage cluster in certain neighborhoods or lot types?
frontage_missing = X_selection["Lot_Frontage"].isna()
neighborhood_missing = (
    pd.DataFrame({"Neighborhood": X_selection["Neighborhood"], "missing": frontage_
    .groupby("Neighborhood")["missing"]
    .agg(houses="size", missing="sum", missing_rate="mean")
    .sort_values("missing_rate", ascending=False)
)
neighborhood_missing["missing_rate"] = neighborhood_missing["missing_rate"].round(3
print(f"Overall missing-frontage rate: {frontage_missing.mean():.3f}")
display(neighborhood_missing)
from scipy.stats import chi2_contingency
print("Chi-square test, unknown frontage by neighborhood: p =",
      f"{chi2_contingency(pd.crosstab(X_selection['Neighborhood'], frontage_missing

# Lot characteristics of houses with unknown versus recorded frontage.
lot_comparison = (
    X_selection.assign(frontage=np.where(frontage_missing, "unknown", "recorded"))
    .groupby("frontage")["Lot_Area"]
    .describe()[["count", "25%", "50%", "75%", "mean"]]
)
display(lot_comparison.round(0))
config_rates = (
    pd.DataFrame({"Lot_Config": X_selection["Lot_Config"], "missing": frontage_miss
    .groupby("Lot_Config")["missing"]
    .agg(houses="size", missing_rate="mean")
)
display(config_rates.round(3))

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
shown = neighborhood_missing[neighborhood_missing["houses"] >= 10]
axes[0].barh([f"{name} (n={n})" for name, n in zip(shown.index, shown["houses"])],
             color="tab:blue")
axes[0].axvline(frontage_missing.mean(), color="black", ls="--", label="overall rat
```

```
axes[0].invert_yaxis()
axes[0].set_xlabel("Share of houses with unknown frontage")
axes[0].set_ylabel("Neighborhood (n = selection houses; at least 10)")
axes[0].set_title("Unknown frontage rate by neighborhood")
axes[0].legend()
bins = np.logspace(np.log10(X_selection.Lot_Area.min()), np.log10(X_selection.Lot_A
axes[1].hist(X_selection.loc[~frontage_missing, "Lot_Area"], bins=bins, density=Tru
axes[1].hist(X_selection.loc[frontage_missing, "Lot_Area"], bins=bins, density=True
axes[1].set_xscale("log")
axes[1].set_xlabel("Lot area (square feet, log scale)")
axes[1].set_ylabel("Density")
axes[1].set_title("Lot area by frontage status")
axes[1].legend()
plt.tight_layout()
plt.show()
```

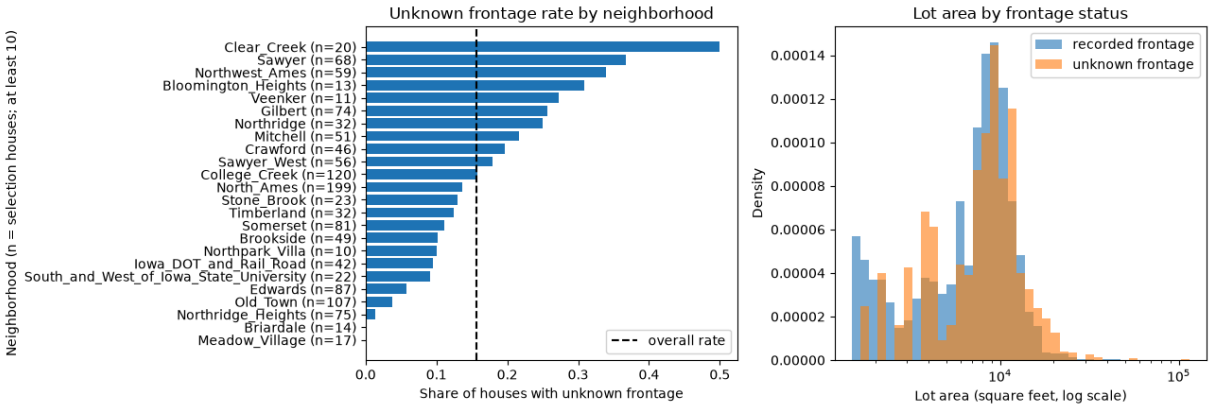
```
Overall missing-frontage rate: 0.156
```

Neighborhood	houses	missing	missing_rate
Green_Hills	2	2	1.000
Clear_Creek	20	10	0.500
Sawyer	68	25	0.368
Northwest_Ames	59	20	0.339
Bloomington_Heights	13	4	0.308
Veenker	11	3	0.273
Gilbert	74	19	0.257
Northridge	32	8	0.250
Greens	4	1	0.250
Mitchell	51	11	0.216
Crawford	46	9	0.196
Sawyer_West	56	10	0.179
College_Creek	120	19	0.158
North_Ames	199	27	0.136
Stone_Brook	23	3	0.130
Timberland	32	4	0.125
Somerset	81	9	0.111
Brookside	49	5	0.102
Northpark_Villa	10	1	0.100
Iowa_DOT_and_Rail_Road	42	4	0.095
South_and_West_of_Iowa_State_University	22	2	0.091
Edwards	87	5	0.057
Old_Town	107	4	0.037
Northridge_Heights	75	1	0.013
Briardale	14	0	0.000
Blueste	4	0	0.000
Meadow_Village	17	0	0.000

```
Chi-square test, unknown frontage by neighborhood: p = 2.4e-14
```

frontage	count	25%	50%	75%	mean
recorded	1112.0	7314.0	9255.0	11220.0	9693.0
unknown	206.0	8002.0	9984.0	12698.0	11593.0

Lot_Config	houses	missing_rate
Corner	225	0.182
CulDSac	75	0.467
FR2	31	0.129
FR3	2	0.000
Inside	985	0.128



Written response — Q1.2

Unknown frontage is far from uniform. The overall rate is 15.6%, but it is 50% in Clear Creek (10 of 20), 37% in Sawyer (25 of 68) and 34% in Northwest Ames (20 of 59), against 3.7% in Old Town (4 of 107), 1.3% in Northridge Heights (1 of 75) and none in Briardale and Meadow Village (0 of 31). It is 47% for cul-de-sac lots against 13% for inside lots, and lots with unknown frontage are larger (median 9,984 against 9,255 sq ft).

Deleting incomplete houses would therefore change the population we model: we would lose half of Clear Creek and a third of Sawyer and Northwest Ames, keep almost all of Old Town, Briardale and Meadow Village, and end up with slightly smaller lots and fewer cul-de-sacs than Ames overall. This is strong evidence against MCAR (chi-square test by neighborhood, $p < 10^{-13}$), since missingness depends on recorded neighborhood and lot type. It is consistent with MAR given those variables but cannot rule out MNAR (frontage may be missing more often when it is unusual).

Q1.3. Establish a simple imputation baseline

2 points

Imputation offers a way to retain houses whose measurements are incomplete. A simple rule, such as replacing unknown frontage with the observed median, gives us a useful reference before we try more elaborate methods. Its simplicity also makes its limitations visible: every house with unknown frontage receives the same replacement, regardless of its other characteristics.

Task: Use `FrontageImputer('median')` to fit an overall median replacement rule on the selection data. Apply it to unknown frontage values and compare the replacement distribution with the observed frontage distribution.

Artifacts: The filled selection predictors, a distribution comparison, and a short explanation of what the median rule preserves and what it does not capture.

Constraints: Fit using observed fitting-sample values; replace only unknown frontage; no sale-price inputs.

Output names for the guided checks: `median_filled` (the selection predictors after applying the baseline).

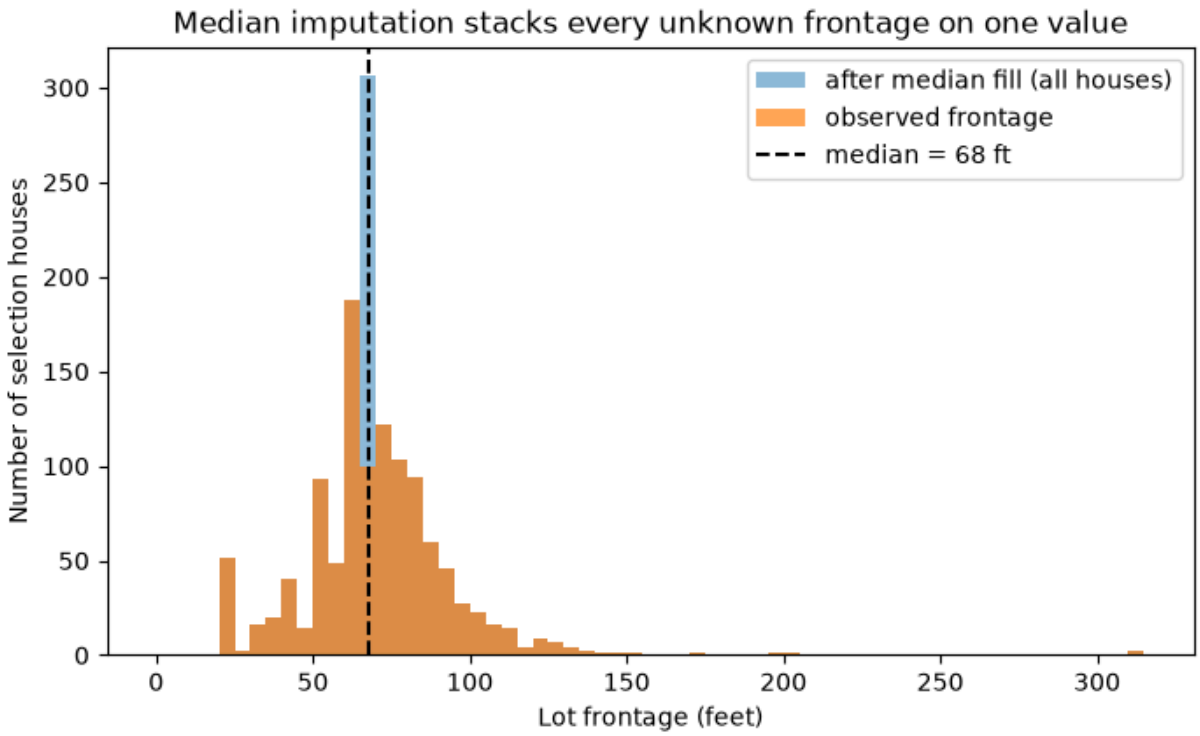
In [7]:
```
# Q1.3: one overall median, learned from the observed selection frontages only.
median_imputer = FrontageImputer("median").fit(X_selection)
median_filled = median_imputer.transform(X_selection)
print(f"Replacement value: {median_imputer.median_:.1f} ft for {int(frontage_missin
print("Observed values unchanged:",
      bool((median_filled.loc[~frontage_missing, "Lot_Frontage"] == X_selection.loc

frontage_summary = pd.DataFrame(
    {
        "observed only": X_selection["Lot_Frontage"].dropna().describe(),
        "after median fill": median_filled["Lot_Frontage"].describe(),
    }
)
display(frontage_summary.round(1))

fig, ax = plt.subplots(figsize=(8, 4.5))
bins = np.arange(0, 320, 5)
ax.hist(median_filled["Lot_Frontage"], bins=bins, alpha=0.5, label="after median fi
ax.hist(X_selection["Lot_Frontage"].dropna(), bins=bins, alpha=0.7, label="observed
ax.axvline(median_imputer.median_, color="black", ls="--", label=f"median = {median
ax.set_xlabel("Lot frontage (feet)")
ax.set_ylabel("Number of selection houses")
ax.set_title("Median imputation stacks every unknown frontage on one value")
ax.legend()
plt.show()
```

```
Replacement value: 68.0 ft for 206 houses
Observed values unchanged: True
```

	observed only	after median fill
count	1112.0	1318.0
mean	69.2	69.0
std	24.4	22.4
min	21.0	21.0
25%	59.0	60.0
50%	68.0	68.0
75%	80.0	78.0
max	313.0	313.0



In [8]:
```
grader.check("q1_3")
```

Out[8]:

q1_3 passed! 🙌

Written response — Q1.3

The median rule, fitted on the 1,112 observed frontages, sets all 206 unknown values to 68 ft. It keeps every observed value and the center (median 68 ft; mean 69.2 to 69.0 ft) but distorts the spread: 17% of selection houses now share exactly 68 ft, the standard deviation falls from 24.4 to 22.4 ft and the interquartile range narrows from 59–80 to 60–78 ft. It also ignores everything else we know: a large Clear Creek cul-de-sac lot and a small Northpark Villa townhouse lot both get 68 ft, although Q1.2 showed unknown frontage concentrates

on larger lots in particular neighborhoods. The filled column therefore understates variability and weakens frontage's link to lot area and location.

Q1.4. Use recorded characteristics for imputation

4 points

The median baseline ignores information we already have about each house. Street frontage may differ across neighborhoods and may be related to lot size, so these recorded characteristics could help us choose more plausible replacements. Comparing these approaches lets us investigate what additional information contributes before deciding whether the extra complexity is worthwhile.

Task: Fit and apply `FrontageImputer('neighborhood')` and `FrontageImputer('regression')`. The neighborhood rule uses an overall median when a fitting sample has no observed frontage for a neighborhood. The regression rule uses square-root lot area. Compare their replacement values with the median baseline, and assess whether their ranges and variation are plausible.

Note any neighborhoods with few or no observed frontages available to fit their replacement rule.

Artifacts: Replacement-value summaries for all three methods, a distribution plot, and a short comparison. Keep the method names for Q1.5, where each rule must be refitted within the relevant fold; do not reuse the rules fitted here on all selection rows.

Constraints: Fit only on observed frontage; exclude sale price; retain measurements actually observed.

In [9]:
```
# Q1.4: conditional rules. Each is fitted on the selection rows for description onl
# Q1.5 refits them inside folds.
fitted_imputers = {m: FrontageImputer(m).fit(X_selection) for m in ["median", "neig
replacements = pd.DataFrame(
    {m: imp.transform(X_selection).loc[frontage_missing, "Lot_Frontage"] for m, imp
)
replacement_summary = replacements.describe().T
replacement_summary["distinct values"] = replacements.nunique()
display(replacement_summary.round(1))
observed_summary = X_selection["Lot_Frontage"].dropna().describe()
print("Observed frontage for reference:", observed_summary.round(1).to_dict())

reg = fitted_imputers["regression"].regression_
print(f"Regression rule: frontage = {reg.intercept_:.2f} + {reg.coef_[0]:.3f} * sqr

# Neighborhoods with little observed frontage to learn from.
observed_counts = (
    X_selection.assign(observed=X_selection.Lot_Frontage.notna())
    .groupby("Neighborhood")["observed"].agg(houses="size", observed="sum")
)
```

```
observed_counts["unknown"] = observed_counts["houses"] - observed_counts["observed"
sparse = observed_counts[(observed_counts["observed"] < 5)]
print("Neighborhoods with fewer than 5 observed frontages:")
display(sparse)
in_schema = set(prep_template.categories[prep_template.categorical.index("Neighborh
print("Schema neighborhoods with no selection houses:", sorted(in_schema - set(obse

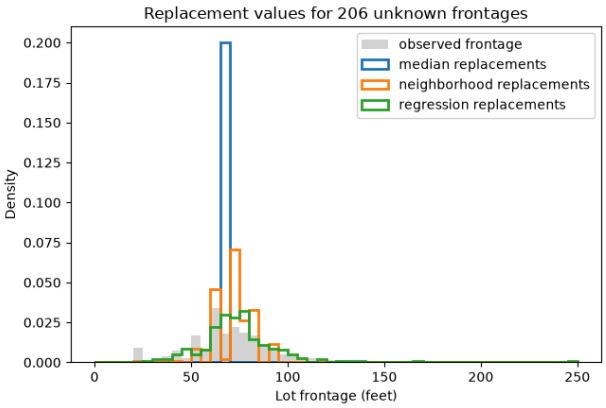
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
bins = np.arange(0, 255, 5)
axes[0].hist(X_selection["Lot_Frontage"].dropna(), bins=bins, density=True, color="
for m, color in zip(replacements, ["tab:blue", "tab:orange", "tab:green"]):
    axes[0].hist(replacements[m], bins=bins, density=True, histtype="step", lw=2, c
axes[0].set_xlabel("Lot frontage (feet)")
axes[0].set_ylabel("Density")
axes[0].set_title(f"Replacement values for {int(frontage_missing.sum())} unknown fr
axes[0].legend()
obs = X_selection.Lot_Frontage.notna()
axes[1].scatter(np.sqrt(X_selection.loc[obs, "Lot_Area"]), X_selection.loc[obs, "Lo
axes[1].scatter(np.sqrt(X_selection.loc[~obs, "Lot_Area"]), replacements["regressio
axes[1].set_xlabel("Square root of lot area (feet)")
axes[1].set_ylabel("Lot frontage (feet)")
axes[1].set_title("Regression rule versus observed frontage")
axes[1].legend()
plt.tight_layout()
plt.show()
```

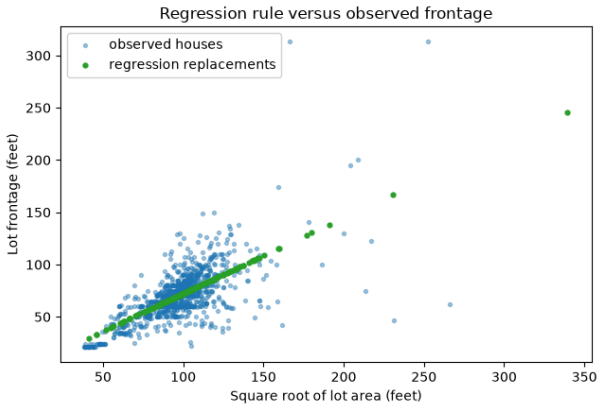
	count	mean	std	min	25%	50%	75%	max	distinct values
median	206.0	68.0	0.0	68.0	68.0	68.0	68.0	68.0	1
neighborhood	206.0	71.2	9.5	24.0	64.5	70.0	75.0	94.5	17
regression	206.0	74.8	21.8	29.9	64.7	72.3	81.5	245.4	190

```
Observed frontage for reference: {'count': 1112.0, 'mean': 69.2, 'std': 24.4, 'min':
21.0, '25%': 59.0, '50%': 68.0, '75%': 80.0, 'max': 313.0}
Regression rule: frontage = 0.05 + 0.723 * sqrt(Lot_Area)
Neighborhoods with fewer than 5 observed frontages:
```

	houses	observed	unknown
Neighborhood			
Blueste	4	4	0
Green_Hills	2	0	2
Greens	4	3	1

```
Schema neighborhoods with no selection houses: []
```





Written response — Q1.4

Both conditional rules give house-specific replacements, unlike the single median value. The neighborhood rule takes 17 distinct values (24 ft in Northpark Villa to 94.5 ft in Veenker; SD 9.5 ft), far below the observed SD of 24.4 ft because every house in a neighborhood gets one value. The regression rule, frontage ≈ 0.05 + 0.723·√(lot area), takes 190 values (29.9 to 245.4 ft; SD 21.8 ft), close to the observed spread. Their means (71.2 and 74.8 ft) exceed the median rule's 68 ft, consistent with Q1.2: unknown frontage sits on larger lots.

The regression rule extrapolates at the extreme: the 115,149 sq ft Clear Creek lot gets 245 ft, where observed frontages scatter widely. The neighborhood rule suffers in sparse neighborhoods: Green Hills has no observed frontage, so its 2 unknown houses fall back to the overall median (68 ft); Greens has 3 observed and Blueste 4 (no unknowns). Replacements resting on 0–3 houses are unreliable, and inside CV folds these groups shrink further.

Q1.5. Validate replacements and use the shared pipeline

6 points

The replacement distributions show whether an imputation method produces plausible values, but plausibility alone does not establish accuracy. Hiding measurements we actually know lets us compare replacements with their recorded values. When we then evaluate price predictions, we also need to consider when the imputer was fitted: held-out houses should not help determine the preprocessing used to predict them.

Task:

1. Restrict the reconstruction exercise to selection houses with observed frontage. Define `observed_positions` as their positional indices within `X_selection`. Use `fixed_folds(len(observed_positions), seed=109)` Each fold's index arrays point into `observed_positions`; for example, its held-out houses are `X_selection.iloc[observed_positions[va]]`. In each fold, hide the held-out

frontage, fit each `FrontageImputer` method on the remaining observed houses, and predict the hidden measurements. Use identical folds and hidden values for all three methods. Report mean absolute error and RMSE in feet, then justify your preferred rule.

2. On `selection_folds`, compare OLS from `reference_pipeline(prep_template)` under two procedures. First fill frontage using `FrontageImputer('neighborhood')` fitted once on all selection rows, then cross-validate on those filled predictors. Second cross-validate directly on `X_selection`, so the pipeline fits its imputer within each training fold. Keep the model and all other preprocessing the same. Record paired fold RMSEs in log-price units. For fold 0, count held-out observed frontage measurements that contributed to each procedure's imputer and compare replacements for the fold-0 held-out houses with unknown frontage.
3. Explain what held-out information affected each procedure and which evaluation matches prediction on new houses. For later Ames questions, use the supplied `shared_preprocessor` defined below.

Artifacts: `imputation_results`, with columns `fold`, `method`, `mae`, and `rmse`, using method labels `'median'`, `'neighborhood'`, and `'regression'`; `leakage_results`, with columns `fold`, `procedure`, and `rmse`, using procedure labels `impute_before_cv` and `impute_in_fold`; `replacement_comparison`, indexed by the IDs of the fold-0 held-out houses with unknown frontage with columns `before_cv` and `within_fold` for the two replacement values in feet; and your choice and leakage explanation.

Constraints: Use selection data only and exclude sale price from all imputers. The score difference may be small or favor either procedure; identify the information used regardless of the scores. Explain one limitation of hiding observed values as a proxy for genuinely missing frontage.

The shared preprocessing uses neighborhood medians, chosen in advance as a simple conditional rule with an overall-median fallback. It is not a claim that this method wins your reconstruction comparison. Your evaluation and justified choice are graded separately. The common pipeline prevents an earlier imputation error from affecting every later question, and it relearns replacements within each fitting fold rather than using a permanently imputed dataset.

In [10]:
```
# Supplied utility: prediction errors use the log-price scale in Ames.
def rmse(y, prediction):
    """Return root mean squared error in the same units as the response y."""
    return float(np.sqrt(np.mean((np.asarray(y) - np.asarray(prediction)) ** 2)))
```

In [11]:
```
# Q1.5 part 1: hide known frontages and see how well each rule recovers them.
METHODS = ["median", "neighborhood", "regression"]
observed_positions = np.flatnonzero(X_selection["Lot_Frontage"].notna().to_numpy())
reconstruction_folds = fixed_folds(len(observed_positions), seed=109)
imputation_rows = []
for fold, (tr, va) in enumerate(reconstruction_folds):
    fit_rows = X_selection.iloc[observed_positions[tr]]
```

```
    held_out = X_selection.iloc[observed_positions[va]]
    truth = held_out["Lot_Frontage"].to_numpy()
    hidden = held_out.assign(Lot_Frontage=np.nan)  # same hidden values for every m
    for method in METHODS:
        guess = FrontageImputer(method).fit(fit_rows).transform(hidden)["Lot_Fronta
        error = guess - truth
        imputation_rows.append(
            {"fold": fold, "method": method,
             "mae": float(np.mean(np.abs(error))), "rmse": float(np.sqrt(np.mean(er
        )
imputation_results = pd.DataFrame(imputation_rows)
display(imputation_results.groupby("method")[["mae", "rmse"]].agg(["mean", "std"]).
display(imputation_results.pivot(index="fold", columns="method", values="rmse").rou

# Q1.5 part 2: imputer fitted once on all selection rows, versus refitted in each f
prefilled = FrontageImputer("neighborhood").fit(X_selection).transform(X_selection)
leakage_rows = []
fold_pipelines = {}
for fold, (tr, va) in enumerate(selection_folds):
    for procedure, x in [("impute_before_cv", prefilled), ("impute_in_fold", X_sele
        pipe = reference_pipeline(prep_template).fit(x.iloc[tr], y_selection[tr])
        leakage_rows.append(
            {"fold": fold, "procedure": procedure, "rmse": rmse(y_selection[va], pi
        )
        fold_pipelines[(fold, procedure)] = pipe
leakage_results = pd.DataFrame(leakage_rows)
leakage_wide = leakage_results.pivot(index="fold", columns="procedure", values="rms
leakage_wide["difference (before - in fold)"] = leakage_wide["impute_before_cv"] -
display(leakage_wide.round(5))
print("Mean fold RMSE (log price):", leakage_results.groupby("procedure")["rmse"].m

# Fold 0: which held-out frontages reached each imputer, and what it filled in.
tr0, va0 = selection_folds[0]
held_out0 = X_selection.iloc[va0]
observed_held_out0 = int(held_out0["Lot_Frontage"].notna().sum())
print(f"Fold 0 held-out houses: {len(va0)}; with observed frontage: {observed_held_
print(f"  impute_before_cv: imputer used {observed_held_out0} held-out observed fro
      f"(all {int(X_selection.Lot_Frontage.notna().sum())} observed selection front
within_imputer = fold_pipelines[(0, "impute_in_fold")].named_steps["prep"].frontage
print(f"  impute_in_fold: imputer used {len(set(tr0) & set(va0))} held-out observed
      f"({int(X_selection.iloc[tr0].Lot_Frontage.notna().sum())} training-fold fron
unknown_ids = held_out0.index[held_out0["Lot_Frontage"].isna()]
replacement_comparison = pd.DataFrame(
    {
        "before_cv": prefilled.loc[unknown_ids, "Lot_Frontage"],
        "within_fold": within_imputer.transform(X_selection.loc[unknown_ids])["Lot_
    },
    index=unknown_ids,
)
replacement_comparison.index.name = "row_id"
replacement_difference = replacement_comparison["before_cv"] - replacement_comparis
print(f"Fold-0 held-out houses with unknown frontage: {len(unknown_ids)}; "
      f"replacements that differ: {int((replacement_difference.abs() > 1e-9).sum())
      f"largest absolute difference: {replacement_difference.abs().max():.1f} ft")
display(replacement_comparison.assign(
    Neighborhood=X_selection.loc[unknown_ids, "Neighborhood"], difference=replaceme
```

	mae		rmse	
	mean	std	mean	std
method				
median	16.66	1.06	24.28	2.70
neighborhood	12.34	1.10	21.26	2.73
regression	11.55	0.98	17.87	3.13

method	median	neighborhood	regression
fold			
0	21.87	19.13	15.16
1	28.01	25.43	17.36
2	26.09	22.52	19.56
3	23.51	20.29	22.35
4	21.91	18.93	14.94

procedure	impute_before_cv	impute_in_fold	difference (before - in fold)
fold			
0	0.20360	0.20360	-0.00000
1	0.20561	0.20556	0.00005
2	0.29461	0.29504	-0.00043
3	0.26959	0.26958	0.00001
4	0.23542	0.23536	0.00006

```
Mean fold RMSE (log price): {'impute_before_cv': 0.24177, 'impute_in_fold': 0.24183}
Fold 0 held-out houses: 264; with observed frontage: 225
  impute_before_cv: imputer used 225 held-out observed frontages (all 1112 observed
selection frontages)
  impute_in_fold: imputer used 0 held-out observed frontages (887 training-fold fron
tages)
Fold-0 held-out houses with unknown frontage: 39; replacements that differ: 25; larg
est absolute difference: 3.0 ft
```

row_id	before_cv	within_fold	Neighborhood	difference
2894	81.5	83.0	Timberland	-1.5
391	80.0	80.0	Northwest_Ames	0.0
312	73.0	74.0	Mitchell	-1.0
2723	70.0	71.0	Sawyer	-1.0
2725	80.0	80.0	Clear_Creek	0.0
2512	71.0	68.0	Somerset	3.0
1530	74.0	74.5	Crawford	-0.5
780	70.0	71.0	Sawyer	-1.0
2892	68.0	68.0	Green_Hills	0.0
1228	75.0	75.0	North_Ames	0.0
1516	60.0	60.0	South_and_West_of_Iowa_State_University	0.0
1565	73.0	74.0	Mitchell	-1.0
2282	73.0	74.0	Mitchell	-1.0
1199	80.0	80.0	Northwest_Ames	0.0
1862	71.0	68.0	Somerset	3.0
1469	70.0	70.0	College_Creek	0.0
1089	64.0	63.5	Gilbert	0.5
1846	71.0	68.0	Somerset	3.0
2537	75.0	75.0	North_Ames	0.0
2871	74.0	74.5	Crawford	-0.5
2424	51.0	51.0	Bloomington_Heights	0.0
478	64.0	63.5	Gilbert	0.5
2446	90.5	91.0	Northridge	-0.5
1006	64.0	63.5	Gilbert	0.5
1081	51.0	51.0	Bloomington_Heights	0.0
673	75.0	75.0	North_Ames	0.0
1080	51.0	51.0	Bloomington_Heights	0.0
568	71.0	68.0	Somerset	3.0
1013	64.0	63.5	Gilbert	0.5

row_id	before_cv	within_fold	Neighborhood	difference
2897	81.5	83.0	Timberland	-1.5
996	64.0	63.5	Gilbert	0.5
598	80.0	80.0	Northwest_Ames	0.0
563	94.5	94.0	Veenker	0.5
1763	90.5	91.0	Northridge	-0.5
1772	90.5	91.0	Northridge	-0.5
920	74.0	74.5	Crawford	-0.5
1446	70.0	70.0	College_Creek	0.0
1811	64.5	65.0	Sawyer_West	-0.5
938	74.0	74.5	Crawford	-0.5

In [12]:
```
# Supplied common checkpoint for subsequent Ames questions.
shared_preprocessor = clone(prep_template)
```

In [13]:
```
grader.check("q1_5")
```

Out[13]:

q1_5 passed! ✨

Written response — Q1.5

Reconstruction. On the same five folds, regression is most accurate: fold-mean MAE 11.5 ft and RMSE 17.9 ft, clearly ahead of the median rule (16.7 and 24.3 ft) but only 0.8 ft MAE ahead of the neighborhood rule (12.3 and 21.3 ft). It has the lowest RMSE in 4 of 5 folds (fold 3 favors the neighborhood rule, 20.3 against 22.4 ft). We prefer it because lot area tracks frontage better than neighborhood does and gives each house its own value; its weakness is extrapolation for very large lots (Q1.4). The shared pipeline's neighborhood medians were fixed in advance and are not the most accurate rule here.

Leakage. In `impute_before_cv` the imputer saw all 1,112 observed selection frontages, so in fold 0 the 225 held-out houses with observed frontage helped set the medians that fill the unknown houses; in `impute_in_fold` it saw only the 887 training-fold frontages. The leak is small here (it involves predictors, not prices, and only 206 of 1,318 houses are imputed): 25 of the 39 fold-0 unknown replacements differ, by at most 3 ft (Somerset: 71 against 68 ft), and mean fold RMSE is 0.24177 for `impute_before_cv` against 0.24183 for `impute_in_fold` (no fold differs by 0.0005 or more). Only `impute_in_fold` matches prediction for new houses, whose frontage cannot have shaped the imputer.

Limitation. Hiding observed values makes them missing at random, but real unknown frontage is not (Q1.2), so these errors may understate those on the houses we actually impute.

Part 2. Regularization

Q2.1. Assess the supplied unregularized baseline

3 points

With missing measurements addressed, we can return to predicting log sale price. The supplied least-squares models provide a starting point without repeating the modeling exercises from Homework 2. Comparing their training and cross-validation errors helps us assess whether fitting the training houses closely translates into accurate predictions for houses not used in fitting, and whether regularization is worth investigating.

Task: Run the supplied baseline calculation. Compare training and CV log-price errors for the modest model and the model using all original predictors, including the fold-to-fold variation in `ols_fold_results`. Describe the observed difference between fitting the training houses and predicting held-out houses, and explain whether this evidence motivates trying regularization.

Artifacts: Compact prediction-error comparison and short motivation.

Constraints: Use the supplied code and partitions; you do not need to rebuild the Homework 2 baseline. Use original predictors without polynomial expansion. Base your explanation on the observed errors, rather than assuming that more predictors always produce harmful overfitting.

Output names for the guided checks: The supplied `baseline_comparison` table; add your interpretation.

The supplied modest model uses the 13 numeric predictors in `CORE`, plus `Neighborhood` and `Overall_Qual`. In `baseline_comparison`, `terms` counts encoded predictor columns and `rank` includes the intercept. A rank below `terms + 1` indicates linear dependence. Read the overall errors and the fold-to-fold variation in `ols_fold_results`; no additional baseline implementation is required.

Supplied baseline calculation

In [14]:
```
# Supplied baseline calculation; interpret the resulting tables in Q2.1.
def baseline_cv(template, x, y, folds, columns=None):
    """Compare training and held-out errors for a fixed OLS predictor design.

    Clone and fit preprocessing separately in each fold, so imputation and
```

```
    scaling use only that fold's training houses. `columns` optionally selects
    encoded predictor positions; None keeps all predictors. Return an overall
    summary and a table of fold errors. RMSE is on the log-price scale.
    """
    held_out_predictions = np.empty(len(y))
    fold_rows = []
    for fold, (train_positions, validation_positions) in enumerate(folds):
        preprocessing = clone(template).fit(x.iloc[train_positions])
        train_predictors = preprocessing.transform(x.iloc[train_positions])
        validation_predictors = preprocessing.transform(x.iloc[validation_positions
        if columns is not None:
            train_predictors = train_predictors[:, columns]
            validation_predictors = validation_predictors[:, columns]
        # Add an intercept column to both designs, then fit only on training outcom
        train_design = np.column_stack(
            [np.ones(len(train_predictors)), train_predictors]
        )
        validation_design = np.column_stack(
            [np.ones(len(validation_predictors)), validation_predictors]
        )
        coefficients = np.linalg.lstsq(train_design, y[train_positions], rcond=1e-1
        held_out_predictions[validation_positions] = validation_design @ coefficien
        fold_rows.append(
            {
                "fold": fold,
                "train_rmse": rmse(y[train_positions], train_design @ coefficients)
                "cv_rmse": rmse(
                    y[validation_positions], validation_design @ coefficients
                ),
            }
        )

    # Refit on all selection houses to report the full-sample training error.
    preprocessing = clone(template).fit(x)
    predictors = preprocessing.transform(x)
    if columns is not None:
        predictors = predictors[:, columns]
    design = np.column_stack([np.ones(len(predictors)), predictors])
    coefficients = np.linalg.lstsq(design, y, rcond=1e-10)[0]
    summary = {
        "train_rmse": rmse(y, design @ coefficients),
        "cv_rmse": rmse(y, held_out_predictions),
        "terms": predictors.shape[1],
        "rank": np.linalg.matrix_rank(design),
    }
    return summary, pd.DataFrame(fold_rows)


# The smaller design reuses the familiar numeric predictors and two categorical inp
reference_prep = clone(shared_preprocessor).fit(X_selection)
feature_names = reference_prep.feature_names_
feature_sources = reference_prep.sources_
small_indices = np.array(
    [
        j
        for j, source in enumerate(feature_sources)
```

```
        if source in CORE + ["Neighborhood", "Overall_Qual"]
    ]
)
modest_summary, _ = baseline_cv(
    shared_preprocessor, X_selection, y_selection, selection_folds, small_indices
)
ols_summary, ols_fold_results = baseline_cv(
    shared_preprocessor, X_selection, y_selection, selection_folds
)
baseline_comparison = pd.DataFrame(
    [modest_summary, ols_summary], index=["modest OLS", "all-original OLS"]
)
display(baseline_comparison)
display(ols_fold_results)
```

	train_rmse	cv_rmse	terms	rank
modest OLS	0.150329	0.166752	48	49
all-original OLS	0.088413	0.279732	285	273

	fold	train_rmse	cv_rmse
0	0	0.085005	0.182490
1	1	0.088307	0.239442
2	2	0.075987	0.402850
3	3	0.081763	0.269579
4	4	0.082672	0.256095

In [15]:
```
grader.check("q2_1")
```

Out[15]:

q2_1 passed! 💯

Written response — Q2.1

model	encoded terms	rank	train RMSE	CV RMSE
modest OLS (`CORE` + `Neighborhood` + `Overall_Qual`)	48	49	0.150	0.167
all-original OLS	285	273	0.088	0.280

The modest model generalizes well: CV RMSE is only 0.016 above training RMSE. The all-original model fits the training houses far more closely (0.088 against 0.150) but predicts held-out houses much worse (0.280 against 0.167): its training RMSE is steady across folds (0.076–0.088) while held-out RMSE ranges from 0.182 to 0.403, with fold 2 far worse. Its rank is 273, not 286 (terms + intercept), so 13 coefficients are unidentified, and with about 1,050

training houses per fold many of the 285 coefficients rest on a few houses with rare levels, so they can fit noise and give extreme predictions for held-out houses with those levels.

This motivates regularization: the extra predictors are not useless, but unconstrained least squares estimates them with too much variance, and shrinkage can keep their information while reducing sensitivity to which houses are in the fit.

Q2.2. Compare ridge and lasso

8 points

The baseline comparison gives us evidence about how the unregularized models generalize. Ridge and lasso offer two ways to constrain coefficient estimates, potentially reducing sensitivity to the fitting sample. Their coefficient trajectories show how those constraints operate, while cross-validation helps us assess which penalty strengths predict well; an improvement over least squares is something to investigate rather than assume.

Task: Tune ridge on `RIDGE_ALPHAS` and lasso on `LASSO_ALPHAS`, using `selection_folds` and `shared_preprocessor`. For each penalty, pool squared prediction errors over all held-out houses and divide by the number of houses; choose the penalty with the smallest pooled mean squared error. Report its square root as CV RMSE. Use this minimum-error rule whenever the guided Ames questions ask you to retune lasso. Compute coefficient paths on all selection rows, show the same eight features in both panels, and explain how you chose them. Plot both CV-error curves with their selected penalties marked.

Artifacts: Performance table, readable trajectory panels, CV-error curves, and interpretation.

Constraints: Use selection data and the supplied common folds and penalty grids. Fit preprocessing within each training fold. Display the same features in both coefficient-path panels and state their coefficient units. Ridge and lasso have different penalty scales, so equal numerical penalties are not equivalent. Report the observed comparison; neither method is required to outperform the other.

Output names for the guided checks: `regularization_comparison`, with rows `'OLS'` (the all-original-predictor baseline), `'ridge'`, and `'lasso'`, and columns `cv_rmse` and `alpha` (use `NaN` for the OLS penalty), plus the required plots. Save the fitted tuned models for Q2.3 and a reusable tuning routine for Q3.1 and Q3.8.

In [16]:
```
# Supplied penalty grids for the ridge/lasso comparison.
LASSO_ALPHAS = np.logspace(-0.5, -5, 27)
RIDGE_ALPHAS = np.logspace(-2, 5, 25)
```

In [17]:
```
# Q2.2: one tuning routine for both penalties, reused in Q3.1 and Q3.8.
LASSO_SETTINGS = {"max_iter": 100_000, "tol": 1e-6}
```

```
def make_penalized(kind, alpha):
    """Return an unfitted ridge or lasso model with the course-wide solver settings
    if kind == "lasso":
        return Lasso(alpha=alpha, warm_start=True, **LASSO_SETTINGS)
    return Ridge(alpha=alpha)


def tune_penalty(kind, alphas, x, y, folds, template=None):
    """Pooled-CV tuning: refit preprocessing in each training fold, pool held-out s

    Returns the penalty with the smallest pooled MSE (sum of squared held-out error
    divided by the number of held-out rows), its CV RMSE, and the whole MSE curve.
    """
    template = shared_preprocessor if template is None else template
    alphas = np.asarray(alphas, dtype=float)
    order = np.argsort(alphas)[::-1]  # large to small, so lasso can warm-start
    squared = np.zeros(len(alphas))
    held_out = 0
    for tr, va in folds:
        prep = clone(template).fit(x.iloc[tr])
        a_tr, a_va = prep.transform(x.iloc[tr]), prep.transform(x.iloc[va])
        model = make_penalized(kind, alphas[order[0]])
        for i in order:
            model.set_params(alpha=alphas[i])
            model.fit(a_tr, y[tr])
            squared[i] += np.sum((y[va] - model.predict(a_va)) ** 2)
        held_out += len(va)
    mse = squared / held_out
    best = int(np.argmin(mse))
    return {"alphas": alphas, "mse": mse, "alpha": float(alphas[best]), "cv_rmse":


with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    ridge_tuning = tune_penalty("ridge", RIDGE_ALPHAS, X_selection, y_selection, se
    lasso_tuning = tune_penalty("lasso", LASSO_ALPHAS, X_selection, y_selection, se
print("Solver warnings during tuning:", len(caught))

regularization_comparison = pd.DataFrame(
    {
        "cv_rmse": [ols_summary["cv_rmse"], ridge_tuning["cv_rmse"], lasso_tuning["
        "alpha": [np.nan, ridge_tuning["alpha"], lasso_tuning["alpha"]],
    },
    index=["OLS", "ridge", "lasso"],
)
display(regularization_comparison)
for name, tuning in [("ridge", ridge_tuning), ("lasso", lasso_tuning)]:
    curve = np.sqrt(tuning["mse"])
    near = tuning["alphas"][curve <= curve.min() + 0.005]
    print(f"{name}: penalties with CV RMSE within 0.005 of the minimum span "
          f"{near.min():.3g} to {near.max():.3g} ({len(near)} grid values)")

# Tuned models refitted on all selection rows (kept for Q2.3).
selection_prep = clone(shared_preprocessor).fit(X_selection)
A_selection = selection_prep.transform(X_selection)
```

```
ridge_model = make_penalized("ridge", ridge_tuning["alpha"]).fit(A_selection, y_sel
lasso_model = make_penalized("lasso", lasso_tuning["alpha"]).fit(A_selection, y_sel
print("Nonzero lasso coefficients at the selected penalty:", int((np.abs(lasso_mode
      "of", len(feature_names))

# Coefficient paths on all selection rows, over the supplied grids.
ridge_path = np.array([Ridge(alpha=a).fit(A_selection, y_selection).coef_ for a in
path_model = make_penalized("lasso", LASSO_ALPHAS[0])
lasso_path_coefs = []
for a in LASSO_ALPHAS:  # the grid runs from large to small penalties
    path_model.set_params(alpha=a)
    lasso_path_coefs.append(path_model.fit(A_selection, y_selection).coef_.copy())
lasso_path_coefs = np.array(lasso_path_coefs)

# Show the eight features with the largest absolute coefficient in the tuned lasso.
path_features = np.argsort(-np.abs(lasso_model.coef_))[:8]
n_numeric = len(prep_template.numeric)
print("Path features (largest |tuned lasso coefficient|):")
display(pd.DataFrame({
    "term": [feature_names[j] for j in path_features],
    "type": ["numeric (per SD)" if j < n_numeric else "indicator (0/1)" for j in pa
    "tuned lasso": lasso_model.coef_[path_features],
    "tuned ridge": ridge_model.coef_[path_features],
}).round(4))

colors = plt.cm.tab10(np.arange(8))
fig, axes = plt.subplots(1, 2, figsize=(15, 5.5), sharey=True)
for ax, alphas, path, tuned, name in zip(
    axes,
    [RIDGE_ALPHAS, LASSO_ALPHAS],
    [ridge_path, lasso_path_coefs],
    [ridge_tuning["alpha"], lasso_tuning["alpha"]],
    ["Ridge", "Lasso"],
):
    for c, j in zip(colors, path_features):
        ax.plot(alphas, path[:, j], color=c, lw=2, label=feature_names[j])
    ax.axvline(tuned, color="black", ls="--", label=f"CV-selected alpha = {tuned:.3
    ax.axhline(0, color="gray", lw=0.8)
    ax.set_xscale("log")
    ax.set_xlabel(f"{name} penalty alpha (log scale)")
    ax.set_title(f"{name} coefficient paths, all selection houses")
    ax.legend(fontsize=7, loc="lower right")
axes[0].set_ylabel("Coefficient (change in log price per SD; per 0/1 for indicators
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
for ax, tuning, name in zip(axes, [ridge_tuning, lasso_tuning], ["Ridge", "Lasso"]):
    ax.plot(tuning["alphas"], np.sqrt(tuning["mse"]), marker="o", ms=4, label="pool
    ax.axvline(tuning["alpha"], color="black", ls="--", label=f"selected alpha = {t
    ax.axhline(ols_summary["cv_rmse"], color="tab:red", ls=":", label=f"OLS CV RMSE
    ax.set_xscale("log")
    ax.set_xlabel(f"{name} penalty alpha (log scale)")
    ax.set_ylabel("Pooled 5-fold CV RMSE (log price)")
    ax.set_title(f"{name}: cross-validation error by penalty")
    ax.legend(fontsize=8)
```

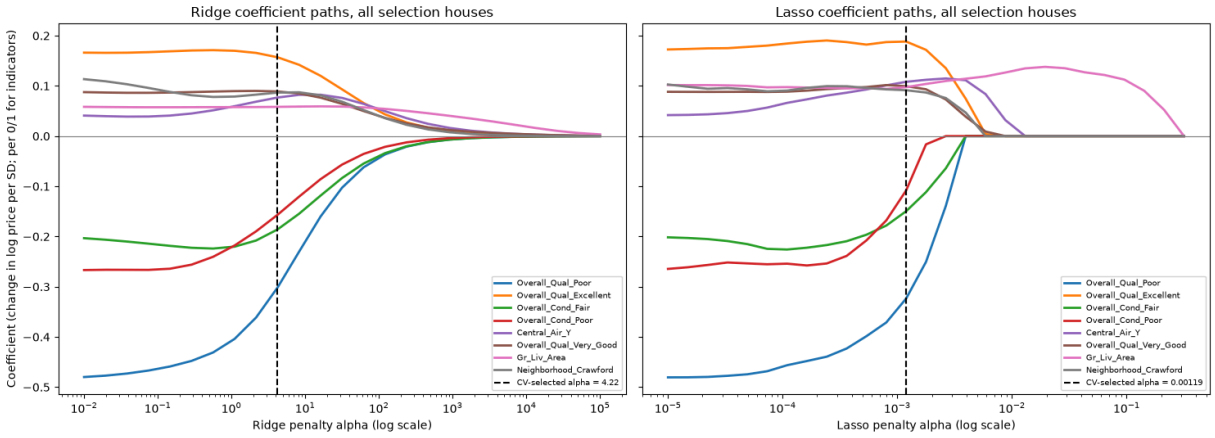
```
plt.tight_layout()
plt.show()
```

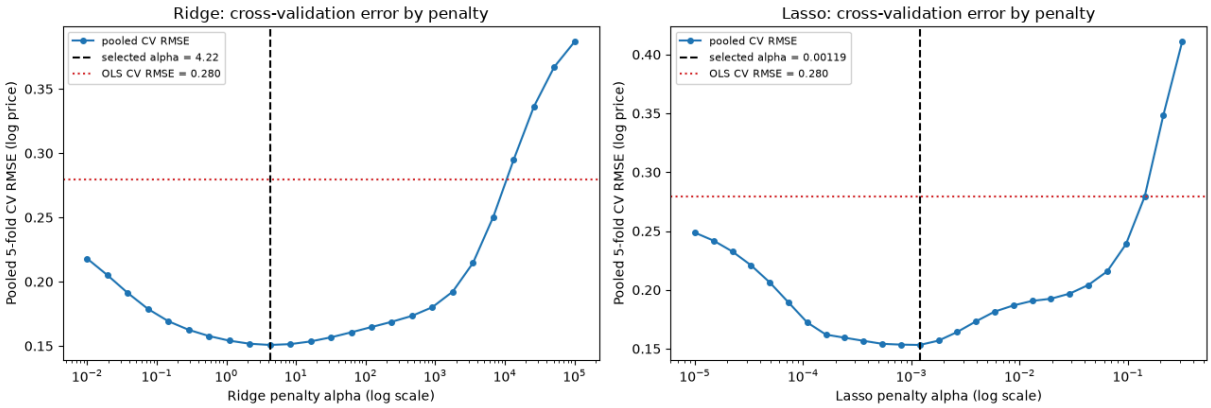
```
Solver warnings during tuning: 0
```

	cv_rmse	alpha
OLS	0.279732	NaN
ridge	0.150751	4.216965
lasso	0.153196	0.001194

```
ridge: penalties with CV RMSE within 0.005 of the minimum span 1.1 to 16.2 (5 grid v
alues)
lasso: penalties with CV RMSE within 0.005 of the minimum span 0.000361 to 0.00178
(5 grid values)
Nonzero lasso coefficients at the selected penalty: 90 of 285
Path features (largest |tuned lasso coefficient|):
```

	term	type	tuned lasso	tuned ridge
0	Overall_Qual_Poor	indicator (0/1)	-0.3236	-0.3015
1	Overall_Qual_Excellent	indicator (0/1)	0.1883	0.1570
2	Overall_Cond_Fair	indicator (0/1)	-0.1496	-0.1858
3	Overall_Cond_Poor	indicator (0/1)	-0.1082	-0.1567
4	Central_Air_Y	indicator (0/1)	0.1081	0.0768
5	Overall_Qual_Very_Good	indicator (0/1)	0.0994	0.0889
6	Gr_Liv_Area	numeric (per SD)	0.0975	0.0583
7	Neighborhood_Crawford	indicator (0/1)	0.0914	0.0867





In [18]:
```
grader.check("q2_2")
```

Out[18]:

q2_2 passed! 💯

Written response — Q2.2

model	selected α	pooled CV RMSE (log price)
all-original OLS	—	0.280
ridge	4.22	0.151
lasso	0.00119	0.153

Both penalties nearly halve the all-original OLS error and beat the modest OLS (0.167). Ridge is better by 0.0024, but the paired fold gaps (lasso − ridge) are +0.004, +0.001, +0.010, −0.004 and −0.002, and both curves are U-shaped and flat near the minimum: CV RMSE stays within 0.005 of its best for ridge α from 1.1 to 16 and lasso α from 3.6×10^{-4} to 1.8×10^{-3} (5 grid values each). We therefore do not call ridge clearly better. The α scales differ: scikit-learn's ridge penalizes the *sum* of squared errors plus $\alpha\|\beta\|^2$, lasso the *mean* squared error over 2 plus $\alpha\|\beta\|_1$.

Path features. Both panels show the eight features with the largest absolute tuned-lasso coefficient (`Gr_Liv_Area` and quality, condition, air-conditioning and neighborhood indicators), in log-price units per selection-sample SD (numeric) or per 0/1 (indicators). Ridge shrinks them toward zero without reaching it. In the lasso panel seven of the eight reach exactly zero, one after another, by $\alpha \approx 1.3\times10^{-2}$; `Gr_Liv_Area` instead grows from about 0.10 to 0.14 as the correlated quality and condition indicators leave, and stays nonzero until $\alpha \approx 0.2$. `Overall_Qual_Poor` is the largest (about −0.48 almost unpenalized, −0.30 ridge and −0.32 lasso at the selected penalties) and rests on very few houses.

Q2.3. Examine the tuned lasso selection

3 points

The trajectory plots show that lasso can set some coefficients exactly to zero. After choosing a penalty by cross-validation, its fitted nonzero features describe which inputs this particular model retained. This gives us an interpretable starting point for the next section, where we will ask whether the same features would be selected if the fitting data were different.

Task: Use the CV-selected penalty to fit lasso once on all selection data. Report which encoded features have nonzero coefficients and which are zero. Relate two examples to the coefficient trajectories in Q2.2.

Artifacts: A coefficient table for all encoded features, with retained features flagged, counts of retained and zero features, and concise interpretation.

Constraints: Selection unit is an individual encoded feature; nonzero does not establish significance or reproducibility.

Output names for the guided checks: `retained_terms`, with columns `term`, `coefficient`, and `nonzero`, and one row per column in `feature_names`. Use `abs(coefficient) > 1e-8` as the numerical nonzero rule.

In [19]:
```
# Q2.3: the tuned lasso refitted on all selection rows (fitted in Q2.2).
retained_terms = pd.DataFrame({"term": feature_names, "coefficient": lasso_model.co
retained_terms["nonzero"] = retained_terms["coefficient"].abs() > 1e-8
print(f"Penalty alpha = {lasso_tuning['alpha']:.4g}: {int(retained_terms.nonzero.su
      f"{int((~retained_terms.nonzero).sum())} set to zero, of {len(retained_terms)
with pd.option_context("display.max_rows", 400):
    display(retained_terms.assign(source=feature_sources)
            .sort_values("coefficient", key=np.abs, ascending=False).round(5))

by_source = retained_terms.assign(source=feature_sources).groupby("source", sort=Fa
print("Retained encoded features by original predictor (predictors with any feature
display(by_source[by_source.retained > 0].sort_values("retained", ascending=False).
print("Original predictors with every encoded feature set to zero:",
      sorted(by_source.index[by_source.retained == 0]))
```

```
Penalty alpha = 0.001194: 90 retained, 195 set to zero, of 285 encoded features.
```

	term	coefficient	nonzero	sourc
124	Overall_Qual_Poor	-0.32355	True	Overall_Qu
121	Overall_Qual_Excellent	0.18832	True	Overall_Qu
131	Overall_Cond_Fair	-0.14964	True	Overall_Con
133	Overall_Cond_Poor	-0.10825	True	Overall_Con
221	Central_Air_Y	0.10812	True	Central_A
126	Overall_Qual_Very_Good	0.09942	True	Overall_Qu
11	Gr_Liv_Area	0.09751	True	Gr_Liv_Are
74	Neighborhood_Crawford	0.09137	True	Neighborhoo
283	Sale_Condition_Normal	0.08260	True	Sale_Conditio
85	Neighborhood_Northridge_Heights	0.08151	True	Neighborhoo
277	Sale_Type_New	0.07414	True	Sale_Typ
75	Neighborhood_Edwards	-0.06397	True	Neighborhoo
109	Bldg_Type_Twnhs	-0.05505	True	Bldg_Typ
20	Garage_Cars	0.04869	True	Garage_Ca
122	Overall_Qual_Fair	-0.04852	True	Overall_Qu
188	Bsmt_Qual_Fair	-0.04759	True	Bsmt_Qu
52	MS_Zoning_Residential_Medium_Density	-0.04678	True	MS_Zonin
33	MS_SubClass_One_Story_1945_and_Older	-0.04498	True	MS_SubCla
2	Year_Built	0.04421	True	Year_Bui
120	Overall_Qual_Below_Average	-0.04366	True	Overall_Qu
234	Functional_Typ	0.04184	True	Function
197	Bsmt_Exposure_Gd	0.04129	True	Bsmt_Exposu
44	MS_SubClass_Two_Story_1945_and_Older	0.04119	True	MS_SubCla
96	Condition_1_Norm	0.04111	True	Condition_
123	Overall_Qual_Good	0.03851	True	Overall_Qu
108	Bldg_Type_OneFam	0.03789	True	Bldg_Typ
179	Exter_Cond_Fair	-0.03724	True	Exter_Con
206	BsmtFin_Type_1_Unf	-0.03688	True	BsmtFin_Type_
7	Total_Bsmt_SF	0.03638	True	Total_Bsmt_S
149	Exterior_1st_BrkFace	0.03594	True	Exterior_1

	term	coefficient	nonzero	sourc
59	Land_Contour_HLS	0.03552	True	Land_Contou
129	Overall_Cond_Below_Average	-0.03530	True	Overall_Con
3	Year_Remod_Add	0.03174	True	Year_Remod_Ad
132	Overall_Cond_Good	0.02910	True	Overall_Con
192	Bsmt_Cond_Fair	-0.02857	True	Bsmt_Con
178	Exter_Qual_Typical	-0.02629	True	Exter_Qu
90	Neighborhood_Somerset	0.02614	True	Neighborhoo
19	Fireplaces	0.02432	True	Fireplace
92	Neighborhood_Stone_Brook	0.02411	True	Neighborhoo
46	MS_SubClass_Two_Story_PUD_1946_and_Newer	-0.02327	True	MS_SubCla
27	Pool_Area	-0.02308	True	Pool_Are
202	BsmtFin_Type_1_GLQ	0.02274	True	BsmtFin_Type_
220	Heating_QC_Typical	-0.02243	True	Heating_Q
14	Full_Bath	0.02054	True	Full_Bat
12	Bsmt_Full_Bath	0.02043	True	Bsmt_Full_Bat
26	Screen_Porch	0.01996	True	Screen_Porc
228	Kitchen_Qual_Typical	-0.01956	True	Kitchen_Qu
24	Enclosed_Porch	0.01944	True	Enclosed_Porc
22	Wood_Deck_SF	0.01706	True	Wood_Deck_S
18	TotRms_AbvGrd	0.01555	True	TotRms_AbvGr
119	Overall_Qual_Average	-0.01552	True	Overall_Qu
8	First_Flr_SF	0.01474	True	First_Flr_S
171	Exterior_2nd_Wd Sdng	0.01419	True	Exterior_2n
89	Neighborhood_Sawyer_West	-0.01383	True	Neighborhoo
17	Kitchen_AbvGr	-0.01282	True	Kitchen_AbvG
258	Garage_Cond_Typical	0.01272	True	Garage_Con
248	Garage_Finish_Unf	-0.01181	True	Garage_Finis
80	Neighborhood_Meadow_Village	-0.01175	True	Neighborhoo
87	Neighborhood_Old_Town	-0.01165	True	Neighborhoo
32	Latitude	0.01105	True	Latitud

	term	coefficient	nonzero	sourc
30	Year_Sold	-0.01088	True	Year_Sol
31	Longitude	-0.00868	True	Longituc
152	Exterior_1st_MetalSd	0.00805	True	Exterior_1
239	Fireplace_Qu_Typical	-0.00714	True	Fireplace_Q
76	Neighborhood_Gilbert	-0.00668	True	Neighborhoo
23	Open_Porch_SF	0.00650	True	Open_Porch_S
205	BsmtFin_Type_1_Rec	-0.00603	True	BsmtFin_Type_
6	Bsmt_Unf_SF	-0.00599	True	Bsmt_Unf_S
61	Land_Contour_Lvl	0.00589	True	Land_Contou
128	Overall_Cond_Average	-0.00557	True	Overall_Con
134	Overall_Cond_Very_Good	0.00549	True	Overall_Con
15	Half_Bath	0.00531	True	Half_Bat
13	Bsmt_Half_Bath	0.00483	True	Bsmt_Half_Bat
1	Lot_Area	0.00461	True	Lot_Are
82	Neighborhood_North_Ames	-0.00411	True	Neighborhoo
199	Bsmt_Exposure_No	-0.00405	True	Bsmt_Exposur
4	Mas_Vnr_Area	0.00368	True	Mas_Vnr_Are
189	Bsmt_Qual_Good	-0.00349	True	Bsmt_Qu
260	Paved_Drive_Paved	0.00295	True	Paved_Driv
58	Lot_Shape_Slightly_Irregular	0.00243	True	Lot_Shap
10	Low_Qual_Fin_SF	0.00231	True	Low_Qual_Fin_S
28	Misc_Val	-0.00204	True	Misc_V
16	Bedroom_AbvGr	0.00180	True	Bedroom_AbvG
0	Lot_Frontage	-0.00159	True	Lot_Frontag
29	Mo_Sold	-0.00143	True	Mo_Sol
227	Kitchen_Qual_Good	-0.00121	True	Kitchen_Qu
25	Three_season_porch	0.00103	True	Three_season_porc
37	MS_SubClass_One_and_Half_Story_Finished_All_Ages	0.00075	True	MS_SubClas
193	Bsmt_Cond_Good	0.00015	True	Bsmt_Con
279	Sale_Type_WD	-0.00000	True	Sale_Typ

	term	coefficient	nonzero	sourc
64	Lot_Config_FR2	-0.00000	False	Lot_Confi
65	Lot_Config_FR3	-0.00000	False	Lot_Confi
66	Lot_Config_Inside	-0.00000	False	Lot_Confi
67	Land_Slope_Mod	0.00000	False	Land_Slop
68	Land_Slope_Sev	-0.00000	False	Land_Slop
69	Neighborhood_Blueste	0.00000	False	Neighborhoo
70	Neighborhood_Briardale	-0.00000	False	Neighborhoo
71	Neighborhood_Brookside	0.00000	False	Neighborhoo
72	Neighborhood_Clear_Creek	0.00000	False	Neighborhoo
73	Neighborhood_College_Creek	-0.00000	False	Neighborhoo
77	Neighborhood_Green_Hills	0.00000	False	Neighborhoo
78	Neighborhood_Greens	0.00000	False	Neighborhoo
79	Neighborhood_Iowa_DOT_and_Rail_Road	-0.00000	False	Neighborhoo
81	Neighborhood_Mitchell	0.00000	False	Neighborhoo
83	Neighborhood_Northpark_Villa	0.00000	False	Neighborhoo
84	Neighborhood_Northridge	0.00000	False	Neighborhoo
86	Neighborhood_Northwest_Ames	-0.00000	False	Neighborhoo
88	Neighborhood_Sawyer	-0.00000	False	Neighborhoo
91	Neighborhood_South_and_West_of_Iowa_State_Univ...	0.00000	False	Neighborhoo
93	Neighborhood_Timberland	0.00000	False	Neighborhoo
94	Neighborhood_Veenker	0.00000	False	Neighborhoo
95	Condition_1_Feedr	-0.00000	False	Condition_
97	Condition_1_PosA	0.00000	False	Condition_
98	Condition_1_PosN	-0.00000	False	Condition_
99	Condition_1_RRAe	-0.00000	False	Condition_
100	Condition_1_RRAn	0.00000	False	Condition_
101	Condition_1_RRNe	-0.00000	False	Condition_
102	Condition_1_RRNn	0.00000	False	Condition_
103	Condition_2_Feedr	-0.00000	False	Condition_
104	Condition_2_Norm	0.00000	False	Condition_

	term	coefficient	nonzero	sourc
105	Condition_2_PosA	0.00000	False	Condition_
106	Condition_2_PosN	-0.00000	False	Condition_
107	Condition_2_RRAe	-0.00000	False	Condition_
110	Bldg_Type_TwnhsE	-0.00000	False	Bldg_Typ
111	Bldg_Type_TwoFmCon	0.00000	False	Bldg_Typ
112	House_Style_One_and_Half_Fin	0.00000	False	House_Sty
113	House_Style_One_and_Half_Unf	0.00000	False	House_Sty
114	House_Style_SFoyer	0.00000	False	House_Sty
115	House_Style_SLvl	-0.00000	False	House_Sty
116	House_Style_Two_Story	-0.00000	False	House_Sty
117	House_Style_Two_and_Half_Fin	0.00000	False	House_Sty
118	House_Style_Two_and_Half_Unf	0.00000	False	House_Sty
125	Overall_Qual_Very_Excellent	0.00000	False	Overall_Qu
127	Overall_Qual_Very_Poor	-0.00000	False	Overall_Qu
130	Overall_Cond_Excellent	0.00000	False	Overall_Con
135	Overall_Cond_Very_Poor	-0.00000	False	Overall_Con
136	Roof_Style_Gable	0.00000	False	Roof_Sty
137	Roof_Style_Gambrel	-0.00000	False	Roof_Sty
138	Roof_Style_Hip	-0.00000	False	Roof_Sty
139	Roof_Style_Mansard	0.00000	False	Roof_Sty
140	Roof_Style_Shed	-0.00000	False	Roof_Sty
141	Roof_Matl_CompShg	-0.00000	False	Roof_Ma
142	Roof_Matl_Metal	0.00000	False	Roof_Ma
143	Roof_Matl_Roll	0.00000	False	Roof_Ma
144	Roof_Matl_Tar&Grv	0.00000	False	Roof_Ma
145	Roof_Matl_WdShake	-0.00000	False	Roof_Ma
146	Roof_Matl_WdShngl	0.00000	False	Roof_Ma
147	Exterior_1st_AsphShn	0.00000	False	Exterior_1
148	Exterior_1st_BrkComm	0.00000	False	Exterior_1
5	BsmtFin_SF_2	0.00000	False	BsmtFin_SF_

	term	coefficient	nonzero	sourc
150	Exterior_1st_CemntBd	0.00000	False	Exterior_1
151	Exterior_1st_HdBoard	-0.00000	False	Exterior_1
9	Second_Flr_SF	0.00000	False	Second_Flr_S
153	Exterior_1st_Plywood	-0.00000	False	Exterior_1
154	Exterior_1st_PreCast	0.00000	False	Exterior_1
155	Exterior_1st_Stucco	-0.00000	False	Exterior_1
156	Exterior_1st_VinylSd	0.00000	False	Exterior_1
157	Exterior_1st_Wd Sdng	0.00000	False	Exterior_1
158	Exterior_1st_WdShing	-0.00000	False	Exterior_1
159	Exterior_2nd_AsphShn	0.00000	False	Exterior_2n
160	Exterior_2nd_Brk Cmn	0.00000	False	Exterior_2n
161	Exterior_2nd_BrkFace	0.00000	False	Exterior_2n
162	Exterior_2nd_CBlock	0.00000	False	Exterior_2n
163	Exterior_2nd_CmentBd	0.00000	False	Exterior_2n
164	Exterior_2nd_HdBoard	-0.00000	False	Exterior_2n
165	Exterior_2nd_ImStucc	0.00000	False	Exterior_2n
166	Exterior_2nd_MetalSd	0.00000	False	Exterior_2n
167	Exterior_2nd_Plywood	-0.00000	False	Exterior_2n
168	Exterior_2nd_PreCast	0.00000	False	Exterior_2n
169	Exterior_2nd_Stucco	-0.00000	False	Exterior_2n
170	Exterior_2nd_VinylSd	0.00000	False	Exterior_2n
253	Garage_Qual_Typical	0.00000	False	Garage_Qu
172	Exterior_2nd_Wd Shng	-0.00000	False	Exterior_2n
173	Mas_Vnr_Type_BrkFace	-0.00000	False	Mas_Vnr_Typ
174	Mas_Vnr_Type_None	0.00000	False	Mas_Vnr_Typ
175	Mas_Vnr_Type_Stone	0.00000	False	Mas_Vnr_Typ
176	Exter_Qual_Fair	-0.00000	False	Exter_Qu
177	Exter_Qual_Good	0.00000	False	Exter_Qu
269	Misc_Feature_None	0.00000	False	Misc_Featu
254	Garage_Cond_Fair	-0.00000	False	Garage_Con

	term	coefficient	nonzero	sourc
180	Exter_Cond_Good	0.00000	False	Exter_Con
181	Exter_Cond_Poor	-0.00000	False	Exter_Con
182	Exter_Cond_Typical	0.00000	False	Exter_Con
183	Foundation_CBlock	0.00000	False	Foundatio
184	Foundation_PConc	0.00000	False	Foundatio
185	Foundation_Slab	-0.00000	False	Foundatio
186	Foundation_Stone	0.00000	False	Foundatio
187	Foundation_Wood	0.00000	False	Foundatio
255	Garage_Cond_Good	-0.00000	False	Garage_Con
256	Garage_Cond_No_Garage	-0.00000	False	Garage_Con
190	Bsmt_Qual_No_Basement	-0.00000	False	Bsmt_Qu
191	Bsmt_Qual_Typical	0.00000	False	Bsmt_Qu
257	Garage_Cond_Poor	-0.00000	False	Garage_Con
259	Paved_Drive_Partial_Pavement	0.00000	False	Paved_Driv
194	Bsmt_Cond_No_Basement	-0.00000	False	Bsmt_Con
195	Bsmt_Cond_Poor	-0.00000	False	Bsmt_Con
196	Bsmt_Cond_Typical	0.00000	False	Bsmt_Con
261	Pool_QC_Fair	0.00000	False	Pool_Q
198	Bsmt_Exposure_Mn	-0.00000	False	Bsmt_Exposur
262	Pool_QC_Good	-0.00000	False	Pool_Q
200	Bsmt_Exposure_No_Basement	-0.00000	False	Bsmt_Exposur
201	BsmtFin_Type_1_BLQ	0.00000	False	BsmtFin_Type_
263	Pool_QC_No_Pool	0.00000	False	Pool_Q
203	BsmtFin_Type_1_LwQ	-0.00000	False	BsmtFin_Type_
204	BsmtFin_Type_1_No_Basement	-0.00000	False	BsmtFin_Type_
264	Pool_QC_Typical	0.00000	False	Pool_Q
265	Fence_Good_Wood	-0.00000	False	Fenc
207	BsmtFin_Type_2_BLQ	-0.00000	False	BsmtFin_Type_
208	BsmtFin_Type_2_GLQ	0.00000	False	BsmtFin_Type_
209	BsmtFin_Type_2_LwQ	0.00000	False	BsmtFin_Type_

	term	coefficient	nonzero	sourc
210	BsmtFin_Type_2_No_Basement	-0.00000	False	BsmtFin_Type_
211	BsmtFin_Type_2_Rec	-0.00000	False	BsmtFin_Type_
212	BsmtFin_Type_2_Unf	0.00000	False	BsmtFin_Type_
213	Heating_GasW	0.00000	False	Heatin
266	Fence_Minimum_Privacy	-0.00000	False	Fenc
215	Heating_OthW	-0.00000	False	Heatin
216	Heating_Wall	-0.00000	False	Heatin
217	Heating_QC_Fair	-0.00000	False	Heating_Q
218	Heating_QC_Good	-0.00000	False	Heating_Q
219	Heating_QC_Poor	-0.00000	False	Heating_Q
267	Fence_Minimum_Wood_Wire	-0.00000	False	Fenc
34	MS_SubClass_One_Story_1946_and_Newer_All_Styles	0.00000	False	MS_SubClas
222	Electrical_FuseF	-0.00000	False	Electric
223	Electrical_FuseP	-0.00000	False	Electric
224	Electrical_Mix	-0.00000	False	Electric
225	Electrical_SBrkr	-0.00000	False	Electric
226	Kitchen_Qual_Fair	-0.00000	False	Kitchen_Qu
35	MS_SubClass_One_Story_PUD_1946_and_Newer	0.00000	False	MS_SubClas
36	MS_SubClass_One_Story_with_Finished_Attic_All_...	-0.00000	False	MS_SubClas
229	Functional_Maj2	-0.00000	False	Function
230	Functional_Min1	0.00000	False	Function
231	Functional_Min2	0.00000	False	Function
232	Functional_Mod	-0.00000	False	Function
233	Functional_Sev	-0.00000	False	Function
38	MS_SubClass_One_and_Half_Story_PUD_All_Ages	-0.00000	False	MS_SubClas
235	Fireplace_Qu_Fair	0.00000	False	Fireplace_Q
236	Fireplace_Qu_Good	0.00000	False	Fireplace_Q
237	Fireplace_Qu_No_Fireplace	-0.00000	False	Fireplace_Q
238	Fireplace_Qu_Poor	-0.00000	False	Fireplace_Q
39	MS_SubClass_One_and_Half_Story_Unfinished_All_...	0.00000	False	MS_SubClas

	term	coefficient	nonzero	sourc
240	Garage_Type_Basment	-0.00000	False	Garage_Typ
241	Garage_Type_BuiltIn	-0.00000	False	Garage_Typ
242	Garage_Type_CarPort	-0.00000	False	Garage_Typ
243	Garage_Type_Detchd	-0.00000	False	Garage_Typ
244	Garage_Type_More_Than_Two_Types	-0.00000	False	Garage_Typ
245	Garage_Type_No_Garage	-0.00000	False	Garage_Typ
246	Garage_Finish_No_Garage	-0.00000	False	Garage_Finis
247	Garage_Finish_RFn	-0.00000	False	Garage_Finis
40	MS_SubClass_PUD_Multilevel_Split_Level_Foyer	-0.00000	False	MS_SubClas
214	Heating_Grav	-0.00000	False	Heatin
250	Garage_Qual_Good	0.00000	False	Garage_Qu
251	Garage_Qual_No_Garage	-0.00000	False	Garage_Qu
252	Garage_Qual_Poor	-0.00000	False	Garage_Qu
268	Fence_No_Fence	0.00000	False	Fenc
270	Misc_Feature_Othr	0.00000	False	Misc_Featur
271	Misc_Feature_Shed	-0.00000	False	Misc_Featur
272	Misc_Feature_TenC	0.00000	False	Misc_Featur
273	Sale_Type_CWD	-0.00000	False	Sale_Typ
274	Sale_Type_ConLD	0.00000	False	Sale_Typ
275	Sale_Type_ConLI	-0.00000	False	Sale_Typ
276	Sale_Type_ConLw	0.00000	False	Sale_Typ
41	MS_SubClass_Split_Foyer	0.00000	False	MS_SubClas
278	Sale_Type_Oth	0.00000	False	Sale_Typ
42	MS_SubClass_Split_or_Multilevel	-0.00000	False	MS_SubClas
280	Sale_Condition_AdjLand	0.00000	False	Sale_Conditio
281	Sale_Condition_Alloca	0.00000	False	Sale_Conditio
282	Sale_Condition_Family	0.00000	False	Sale_Conditio
43	MS_SubClass_Two_Family_conversion_All_Styles_a...	0.00000	False	MS_SubClas
45	MS_SubClass_Two_Story_1946_and_Newer	-0.00000	False	MS_SubClas
284	Sale_Condition_Partial	0.00000	False	Sale_Conditio

	term	coefficient	nonzero	sourc
47	MS_SubClass_Two_and_Half_Story_All_Ages	0.00000	False	MS_SubCla
48	MS_Zoning_Floating_Village_Residential	0.00000	False	MS_Zonin
49	MS_Zoning_I_all	0.00000	False	MS_Zonin
50	MS_Zoning_Residential_High_Density	-0.00000	False	MS_Zonin
51	MS_Zoning_Residential_Low_Density	0.00000	False	MS_Zonin
53	Street_Pave	0.00000	False	Stree
54	Alley_No_Alley_Access	0.00000	False	Alle
21	Garage_Area	-0.00000	False	Garage_Are
55	Alley_Paved	0.00000	False	Alle
56	Lot_Shape_Moderately_Irregular	0.00000	False	Lot_Shap
57	Lot_Shape_Regular	-0.00000	False	Lot_Shap
60	Land_Contour_Low	0.00000	False	Land_Contou
62	Utilities_NoSewr	-0.00000	False	Utilitie
63	Lot_Config_CulDSac	0.00000	False	Lot_Confi
249	Garage_Qual_Fair	-0.00000	False	Garage_Qu

```
Retained encoded features by original predictor (predictors with any feature retaine
d):
```

source	Neighborhood	Overall_Qual	Overall_Cond	MS_SubClass	BsmtFin_Type_1	Exterior_
retained	10	7	6	4	3	
features	26	9	8	15	6	

2 rows × 57 columns

```
Original predictors with every encoded feature set to zero: ['Alley', 'BsmtFin_SF_
2', 'BsmtFin_Type_2', 'Condition_2', 'Electrical', 'Fence', 'Foundation', 'Garage_Ar
ea', 'Garage_Qual', 'Garage_Type', 'Heating', 'House_Style', 'Land_Slope', 'Lot_Conf
ig', 'Mas_Vnr_Type', 'Misc_Feature', 'Pool_QC', 'Roof_Matl', 'Roof_Style', 'Second_F
lr_SF', 'Street', 'Utilities']
```

In [20]:
```
grader.check("q2_3")
```

Out[20]:

q2_3 passed! 🌈

Written response — Q2.3

At the CV-selected α = 0.00119 the lasso refitted on all selection houses keeps **90** of 285 encoded features and sets **195** to zero. The 90 include 30 numeric measurements (for example `Gr_Liv_Area`, `Year_Built`, `Total_Bsmt_SF`, `Garage_Cars`) and indicators from many categorical predictors (10 of 26 neighborhood, 7 of 9 `Overall_Qual` and 6 of 8 `Overall_Cond`). Twenty-two original predictors lose every feature, among them `Garage_Area`, `Second_Flr_SF`, `House_Style`, `Foundation` and `Lot_Config`.

Two Q2.2 trajectories illustrate this. `Overall_Qual_Poor` (−0.32) is already shrunk from about −0.48 and would reach zero near $\alpha \approx 4\times10^{-3}$, about three grid steps up, so its size depends strongly on the penalty. `Overall_Cond_Poor` (−0.11) is already near zero at the next grid point up ($\alpha \approx 1.8\times10^{-3}$, −0.016) and exactly zero from 2.6×10^{-3}, so a slightly larger penalty that the flat CV curve barely distinguishes would drop it.

A nonzero coefficient only means this fit, at this penalty, on these houses used the feature; it is neither significance nor reproducibility (Q3.1 tests the latter).

Part 3. Stability, uncertainty, and interpretation

Reusing saved bootstrap results

The supplied `cached_bootstrap_results` helper can save numerical bootstrap results and reload them on later runs. It checks the supplied data, settings, fitting-function code, listed code dependencies, and package versions before reuse. Keep plotting code separate from the computation so revising a figure does not require rerunning the fits.

If you change your model or fitting implementation, delete the corresponding cache file or set `REFRESH_BOOTSTRAP_CACHE = True` and rerun. Update the `implementation_version` setting as well. The checks cannot discover every external dependency of your code. Pass all fitting settings and relevant dependencies; do not rely on the presence of a saved file alone. After refreshing, set the flag back to `False` to permit reuse.

A fresh run still performs all required bootstrap fits. Cached results must come from your own current implementation. The autograder must be able to run without a preexisting cache.

Run `help(cached_bootstrap_results)` for the full interface. Pass a no-argument computation function, `cache_path`, the input `X` and `y`, a `settings` dictionary, and any fitting-function `dependencies`; `force=True` requests a fresh computation. Include a string `implementation_version` in `settings`. Use numeric values, strings, lists, dictionaries, or arrays for settings, rather than fitted model objects. The computation must return a dictionary of numerical or string arrays.

Q3.1. Assess selection stability

7 points

The tuned lasso retained a particular set of features, but those choices were learned from one sample of houses. Another sample could lead to a different penalty and a different subset. Repeating both tuning and fitting on bootstrap samples lets us investigate the reproducibility of the selection procedure we actually used, rather than treating the original penalty as known.

Task: Bootstrap selection houses and retune by CV within each resample, then record feature selection frequencies. Apply the staff-specified stability/support rule to freeze an estimable subset. Explain why retuning assesses the entire selection procedure.

Artifacts: Frequency bars/table and frozen encoded-feature list.

Constraints: Use the supplied CV helper to keep every copy of a resampled house in the same fold. Refit preprocessing within each fitting fold and compare the same encoded features across resamples. Use the supplied 80% selection-frequency threshold and support

checkpoint. Do not use inference outcomes to choose features. The frequency threshold is a selection rule, not a significance level.

Repeated selection does not guarantee adequate support in independent inference rows. A very rare predictor value can disappear from bootstrap fitting samples, leaving its coefficient unidentified. The supplied predictor-only support/rank helper reports exclusions and freezes an estimable design. Exclusion does not establish no association or no predictive value.

Output names for the guided checks: `selection_bootstrap_results`, with one row per bootstrap and columns `replicate`, `alpha`, and `nonzero` (the number of nonzero coefficients in that refit); `selection_frequencies`, an array in `feature_names` order; `frozen_indices`, the retained column indices; and the exclusion report, with an explanation of any exclusions. Save the penalties and counts now: Q3.2 reuses them without another bootstrap run.

Use `B_SELECTION` draws from `np.random.default_rng(SELECTION_BOOTSTRAP_SEED)`. For draw `b`, pass `selection_ids[idx]` to `bootstrap_folds(..., seed=109+b)`, retune with the Q2.2 rule, and refit on that entire resample. Record nonzero coefficients using the Q2.3 numerical rule. The supplied category schema keeps column order fixed. Candidates have frequencies at least `STABILITY_CUTOFF`. Call `support_checkpoint(shared_preprocessor, X_inference, candidates, selection_frequencies, MIN_SUPPORT)` to obtain `frozen_indices` and its report. The helper excludes inadequate predictor support or linear dependence. This list is frozen: later inference results must not change it.

In [21]:
```
# Supplied settings for retuned lasso selection stability.
B_SELECTION = 100
SELECTION_BOOTSTRAP_SEED = 2109
STABILITY_CUTOFF = 0.80
MIN_SUPPORT = 10
# Refresh after changing the fitting implementation; see the cache instructions abo
REFRESH_BOOTSTRAP_CACHE = False
```

In [22]:
```
# Q3.1: bootstrap the whole lasso procedure (resample, retune by grouped CV, refit)
NONZERO_TOL = 1e-8


def selection_bootstrap():
    """Return per-replicate penalties, nonzero counts and the selected-feature matr
    rng = np.random.default_rng(SELECTION_BOOTSTRAP_SEED)
    n = len(X_selection)
    alphas = np.empty(B_SELECTION)
    nonzero = np.empty(B_SELECTION, dtype=int)
    selected = np.zeros((B_SELECTION, len(feature_names)), dtype=bool)
    for b in range(B_SELECTION):
        idx = rng.integers(0, n, size=n)
        x_b, y_b = X_selection.iloc[idx], y_selection[idx]
        # Copies of the same house always share a fold.
        folds_b = bootstrap_folds(selection_ids[idx], seed=109 + b)
```

```
        tuning_b = tune_penalty("lasso", LASSO_ALPHAS, x_b, y_b, folds_b)
        prep_b = clone(shared_preprocessor).fit(x_b)
        model_b = make_penalized("lasso", tuning_b["alpha"]).fit(prep_b.transform(x
        alphas[b] = tuning_b["alpha"]
        selected[b] = np.abs(model_b.coef_) > NONZERO_TOL
        nonzero[b] = selected[b].sum()
    return {"alpha": alphas, "nonzero": nonzero, "selected": selected}


selection_cache = cached_bootstrap_results(
    selection_bootstrap,
    cache_path="cache/q3_1_selection_bootstrap.npz",
    X=X_selection,
    y=y_selection,
    settings={
        "implementation_version": "q3_1-v1",
        "B_SELECTION": B_SELECTION,
        "SELECTION_BOOTSTRAP_SEED": SELECTION_BOOTSTRAP_SEED,
        "fold_seed_base": 109,
        "LASSO_ALPHAS": LASSO_ALPHAS,
        "LASSO_SETTINGS": LASSO_SETTINGS,
        "NONZERO_TOL": NONZERO_TOL,
        "preprocessor": repr(shared_preprocessor.get_params()),
    },
    dependencies=[tune_penalty, make_penalized, AmesPreprocessor, FrontageImputer,
    force=REFRESH_BOOTSTRAP_CACHE,
)

selection_bootstrap_results = pd.DataFrame(
    {
        "replicate": np.arange(B_SELECTION),
        "alpha": selection_cache["alpha"],
        "nonzero": selection_cache["nonzero"].astype(int),
    }
)
selection_frequencies = selection_cache["selected"].mean(axis=0)
candidates = np.flatnonzero(selection_frequencies >= STABILITY_CUTOFF)
print(f"{len(candidates)} encoded features selected in at least {STABILITY_CUTOFF:.

# Predictor-only support and rank check on the inference rows (no outcomes used).
frozen_indices, support_report = support_checkpoint(
    shared_preprocessor, X_inference, candidates, selection_frequencies, MIN_SUPPOR
)
print(f"Frozen design: {len(frozen_indices)} encoded features.")
excluded = support_report[support_report.decision != "retained"]
display(excluded)
print("Excluded candidates:", "none" if excluded.empty else excluded["term"].tolist
frozen_terms = [feature_names[j] for j in frozen_indices]
print("Frozen encoded features:", frozen_terms)

frequency_table = pd.DataFrame({"term": feature_names, "source": feature_sources, "
frequency_table["status"] = np.where(
    np.isin(np.arange(len(feature_names)), frozen_indices), "frozen",
    np.where(selection_frequencies >= STABILITY_CUTOFF, "candidate excluded by supp
)
frequency_table = frequency_table.sort_values("frequency", ascending=False)
```

```
print("Distribution of selection frequencies over all", len(feature_names), "encode
display(pd.cut(selection_frequencies, [-0.001, 0, 0.2, 0.5, 0.8 - 1e-9, 1.0],
               labels=["0", "(0, 0.2]", "(0.2, 0.5]", "(0.5, 0.8)", "[0.8, 1]"]).va

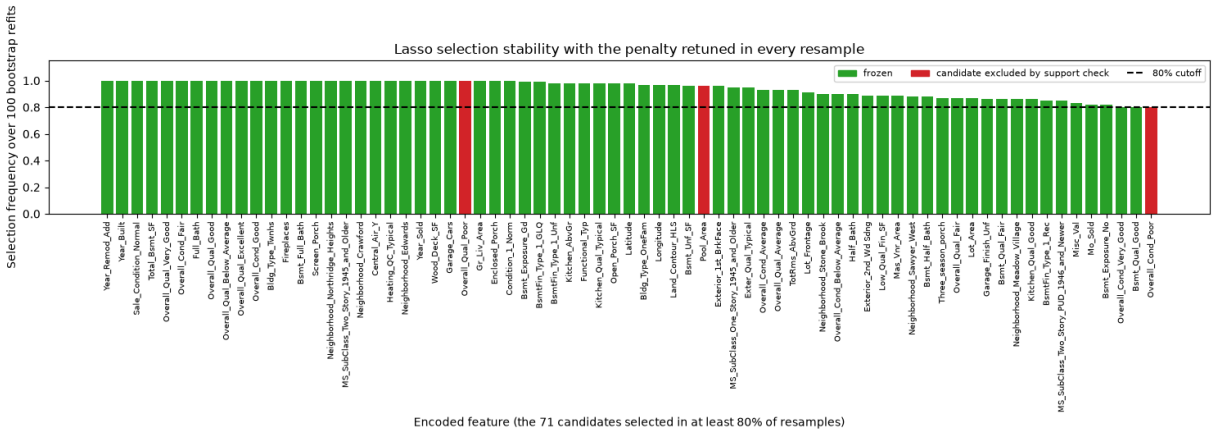
shown = frequency_table[frequency_table["frequency"] >= STABILITY_CUTOFF]
color_map = {"frozen": "tab:green", "candidate excluded by support check": "tab:red
fig, ax = plt.subplots(figsize=(15, 5))
ax.bar(np.arange(len(shown)), shown["frequency"], color=shown["status"].map(color_m
ax.axhline(STABILITY_CUTOFF, color="black", ls="--")
ax.set_ylim(0, 1.15)
ax.set_xticks(np.arange(len(shown)), shown["term"], rotation=90, fontsize=7)
ax.set_ylabel(f"Selection frequency over {B_SELECTION} bootstrap refits")
ax.set_xlabel(f"Encoded feature (the {len(shown)} candidates selected in at least {
ax.set_title("Lasso selection stability with the penalty retuned in every resample"
from matplotlib.patches import Patch
from matplotlib.lines import Line2D
ax.legend(handles=[Patch(color=c, label=l) for l, c in color_map.items() if l in se
          + [Line2D([], [], color="black", ls="--", label=f"{STABILITY_CUTOFF:.0%}
          fontsize=8, loc="upper right", ncol=3)
plt.tight_layout()
plt.show()
```

```
Saved fresh bootstrap results to cache.
71 encoded features selected in at least 80% of 100 resamples.
Frozen design: 68 encoded features.
```

	term	frequency	nonmodal_support	decision
19	Overall_Qual_Poor	1.00	2	inadequate predictor support
41	Pool_Area	0.96	1	inadequate predictor support
69	Overall_Cond_Poor	0.80	3	inadequate predictor support

```
Excluded candidates: ['Overall_Qual_Poor', 'Pool_Area', 'Overall_Cond_Poor']
Frozen encoded features: ['Bldg_Type_Twnhs', 'Bsmt_Full_Bath', 'Central_Air_Y', 'Con
dition_1_Norm', 'Enclosed_Porch', 'Fireplaces', 'Full_Bath', 'Garage_Cars', 'Gr_Liv_
Area', 'Heating_QC_Typical', 'MS_SubClass_Two_Story_1945_and_Older', 'Neighborhood_C
rawford', 'Neighborhood_Edwards', 'Neighborhood_Northridge_Heights', 'Overall_Cond_F
air', 'Overall_Cond_Good', 'Overall_Qual_Below_Average', 'Overall_Qual_Excellent',
'Overall_Qual_Good', 'Overall_Qual_Very_Good', 'Sale_Condition_Normal', 'Screen_Porc
h', 'Total_Bsmt_SF', 'Wood_Deck_SF', 'Year_Built', 'Year_Remod_Add', 'Year_Sold', 'B
smtFin_Type_1_GLQ', 'Bsmt_Exposure_Gd', 'BsmtFin_Type_1_Unf', 'Functional_Typ', 'Kit
chen_AbvGr', 'Kitchen_Qual_Typical', 'Latitude', 'Open_Porch_SF', 'Bldg_Type_OneFa
m', 'Land_Contour_HLS', 'Longitude', 'Bsmt_Unf_SF', 'Exterior_1st_BrkFace', 'Exter_Q
ual_Typical', 'MS_SubClass_One_Story_1945_and_Older', 'Overall_Cond_Average', 'Overa
ll_Qual_Average', 'TotRms_AbvGrd', 'Lot_Frontage', 'Half_Bath', 'Neighborhood_Stone_
Brook', 'Overall_Cond_Below_Average', 'Exterior_2nd_Wd Sdng', 'Low_Qual_Fin_SF', 'Ma
s_Vnr_Area', 'Bsmt_Half_Bath', 'Neighborhood_Sawyer_West', 'Lot_Area', 'Overall_Qual
_Fair', 'Three_season_porch', 'Bsmt_Qual_Fair', 'Garage_Finish_Unf', 'Kitchen_Qual_G
ood', 'Neighborhood_Meadow_Village', 'BsmtFin_Type_1_Rec', 'MS_SubClass_Two_Story_PU
D_1946_and_Newer', 'Misc_Val', 'Bsmt_Exposure_No', 'Mo_Sold', 'Bsmt_Qual_Good', 'Ove
rall_Cond_Very_Good']
Distribution of selection frequencies over all 285 encoded features:
```

	features
0	20
(0, 0.2]	78
(0.2, 0.5]	81
(0.5, 0.8)	35
[0.8, 1]	71



In [23]:
```
grader.check("q3_1")
```

Out[23]:

q3_1 passed! ✨

Written response — Q3.1

In each of 100 resamples of the 1,318 selection houses we retuned the lasso penalty with the Q2.2 pooled-MSE rule on `bootstrap_folds` folds (copies of a house stay together), then refitted on the whole resample. Frequencies are widely spread: 20 of 285 encoded features were never selected, 78 were selected in (0, 20%] of resamples, 81 in (20%, 50%], 35 in (50%, 80%) and **71 reached the 80% cutoff** (for example `Gr_Liv_Area`, `Total_Bsmt_SF`, `Year_Built`, the main `Overall_Qual` and `Overall_Cond` levels).

The predictor-only support check on the inference rows excluded 3 candidates for **inadequate support**: `Overall_Qual_Poor` (frequency 1.00; 2 inference houses), `Overall_Cond_Poor` (0.80; 3) and `Pool_Area` (0.96; 1 house with a pool). Their coefficients could not be estimated reliably, and bootstrap resamples could lose them entirely. This says only that the design cannot support inference on them, not that they are unrelated to price. The frozen design has **68 encoded features**, fixed before any inference outcome was used.

Retuning matters because the procedure we use chooses its penalty from the data; a fixed α would ignore the instability of tuning itself (the selected penalty ranges from 1.1×10^{-4} to 1.8×10^{-3}, moving the number of selected features from about 70 to 199; Q3.2). Grouped folds are needed because copies of a house in both fitting and validation folds would reward

memorizing houses and push the penalty too low. The 80% cutoff is a reproducibility rule, not a significance level.

Q3.2. Examine penalty variability

3 points

Each bootstrap fit made a tuning decision as well as a selection decision. Variation in the selected penalties can help explain why features enter or leave the model, especially when the cross-validation error curve is nearly flat around its minimum. The recorded penalties therefore provide useful context for the selection frequencies, even when different penalties give similar predictive performance.

Task: Reuse recorded bootstrap penalties and selected-set sizes. Relate their distribution to the earlier CV-error curve and describe how flat the CV curve is near its minimum and what that implies.

Artifacts: A distribution plot of the selected penalties on a logarithmic axis, a comparison with the number of nonzero features, and a short explanation relating the observed variability to the CV-error curve.

Constraints: No extra bootstrap runs. Descriptive tuning variability is not a confidence interval for a uniquely true penalty. Do not automatically use the bootstrap-average penalty for final fitting.

Output names for the guided checks: Use the `alpha` column of `selection_bootstrap_results`.

In [24]:
```
# Q3.2: tuning variability, reusing the Q3.1 penalties (no new fits).
alpha_counts = selection_bootstrap_results["alpha"].round(12).value_counts().sort_i
alpha_summary = selection_bootstrap_results["alpha"].describe(percentiles=[0.1, 0.2
display(alpha_summary.to_frame("bootstrap selected alpha").T)
print(f"Full-sample selected alpha (Q2.2): {lasso_tuning['alpha']:.4g}; "
      f"distinct grid values chosen by bootstrap: {len(alpha_counts)}")
display(selection_bootstrap_results.groupby("alpha")["nonzero"].agg(["size", "mean"
print("Correlation of log alpha with nonzero count:",
      round(np.corrcoef(np.log(selection_bootstrap_results.alpha), selection_bootst

curve = np.sqrt(lasso_tuning["mse"])
for tol in [0.002, 0.005]:
    near = LASSO_ALPHAS[curve <= curve.min() + tol]
    share = selection_bootstrap_results["alpha"].between(near.min() - 1e-15, near.m
    print(f"Q2.2 CV RMSE within {tol} of its minimum for alpha in [{near.min():.3g}
          f"{share:.0%} of bootstrap penalties fall in that range.")

fig, axes = plt.subplots(1, 3, figsize=(17, 4.5))
axes[0].bar(np.log10(alpha_counts.index), alpha_counts.values, width=0.12, color="t
axes[0].axvline(np.log10(lasso_tuning["alpha"]), color="black", ls="--", label="Q2.
axes[0].set_xlabel("Selected lasso penalty, log10(alpha)")
```

```
axes[0].set_ylabel("Number of bootstrap resamples")
axes[0].set_title(f"Penalties chosen in {B_SELECTION} retuned resamples")
axes[0].legend(fontsize=8)

jitter = np.random.default_rng(0).uniform(-0.03, 0.03, len(selection_bootstrap_resu
axes[1].scatter(np.log10(selection_bootstrap_results.alpha) + jitter, selection_boo
axes[1].scatter(np.log10(lasso_tuning["alpha"]), int((np.abs(lasso_model.coef_) > N
axes[1].set_xlabel("Selected lasso penalty, log10(alpha) (jittered)")
axes[1].set_ylabel("Nonzero encoded features")
axes[1].set_title("Larger penalties select fewer features")
axes[1].legend(fontsize=8)

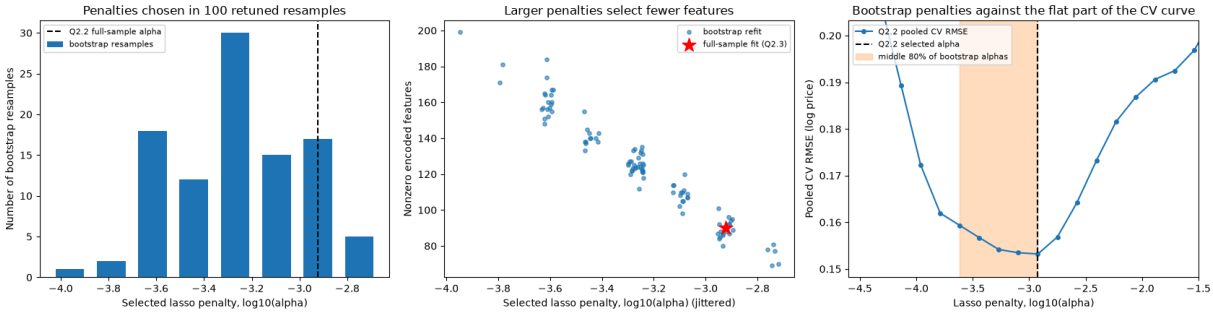
axes[2].plot(np.log10(LASSO_ALPHAS), curve, marker="o", ms=4, label="Q2.2 pooled CV
axes[2].axvline(np.log10(lasso_tuning["alpha"]), color="black", ls="--", label="Q2.
lo, hi = np.log10(selection_bootstrap_results.alpha.quantile([0.1, 0.9]))
axes[2].axvspan(lo, hi, color="tab:orange", alpha=0.25, label="middle 80% of bootst
axes[2].set_xlim(-4.6, -1.5)
axes[2].set_ylim(curve.min() - 0.005, curve.min() + 0.05)
axes[2].set_xlabel("Lasso penalty, log10(alpha)")
axes[2].set_ylabel("Pooled CV RMSE (log price)")
axes[2].set_title("Bootstrap penalties against the flat part of the CV curve")
axes[2].legend(fontsize=8)
plt.tight_layout()
plt.show()
```

	count	mean	std	min	10%	25%	50%	75%	9
bootstrap selected alpha	100.0	0.000665	0.000413	0.000109	0.000242	0.000361	0.000538	0.000801	0.001

```
Full-sample selected alpha (Q2.2): 0.001194; distinct grid values chosen by bootstra
p: 8
```

alpha	size	mean	min	max
0.000109	1	199.0	199	199
0.000163	2	176.0	171	181
0.000242	18	160.9	148	184
0.000361	12	140.8	133	155
0.000538	30	125.2	112	135
0.000801	15	108.7	98	120
0.001194	17	89.6	80	101
0.001778	5	75.0	69	81

```
Correlation of log alpha with nonzero count: -0.978
Q2.2 CV RMSE within 0.002 of its minimum for alpha in [0.000538, 0.00119]; 62% of bo
otstrap penalties fall in that range.
Q2.2 CV RMSE within 0.005 of its minimum for alpha in [0.000361, 0.00178]; 79% of bo
otstrap penalties fall in that range.
```



In [25]:
```
grader.check("q3_2")
```

Out[25]:

q3_2 passed! 🙌

Written response — Q3.2

The 100 retuned resamples chose 8 different grid penalties, from 1.1×10^{-4} to 1.8×10^{-3} (median 5.4×10^{-4}, below the full-sample 1.19×10^{-3}; only 22 chose the full-sample value or larger). The penalty almost fixes the number of features kept (correlation of log α with the nonzero count −0.98): about 75 at 1.8×10^{-3}, 125 at 5.4×10^{-4} and 199 at 1.1×10^{-4}.

This matches the flat Q2.2 CV curve: pooled CV RMSE is within 0.002 of its minimum for α from 5.4×10^{-4} to 1.19×10^{-3} and within 0.005 from 3.6×10^{-4} to 1.8×10^{-3}, a factor-of-five range holding 79% of the bootstrap penalties. Predictions are almost equally good across it while the selected sets differ greatly, so features that enter only at smaller penalties get intermediate frequencies, and the 80% rule keeps those that survive across most of the range. The spread describes unstable tuning, not a confidence interval for a "true" penalty, and we would not refit with the bootstrap-average penalty.

Q3.3. Quantify coefficient uncertainty

6 points

A feature can be selected frequently without its coefficient being estimated precisely. We now move from reproducibility of selection to uncertainty about the size and direction of the retained associations. Because prices in the selection sample helped determine which features survived, the independent inference sample gives us fresh outcomes for estimating the frozen model and assessing its coefficient uncertainty.

Task: Fit OLS with the frozen features on the independent inference rows and bootstrap those coefficient estimates. Express numeric effects per one selection-data standard deviation throughout the analysis, even when each bootstrap fitting sample has its own

scaler. Leave encoded indicator effects in their 0/1 units. Use the supplied preprocessing and fitting helpers.

Artifacts: A table for every frozen feature containing its coefficient estimate, bootstrap mean, standard deviation, and 95% percentile interval; a coefficient-interval plot with zero marked; and histograms for two coefficients you choose, with zero marked. Explain what these results support about the estimated associations.

Constraints: Use only the independent inference rows for coefficient estimation and its bootstrap. Apply the supplied support and rank checks, and state the assumptions of the imputation and regression procedure. These intervals describe this independent fit; they do not automatically apply to the later model refitted on all training rows.

In Section 5 the predictors were prescribed before coefficient analysis. Here they were selected using prices, so using the same outcomes for ordinary fixed-model intervals would ignore the selection step. Reserving independent inference outcomes addresses that reuse; it does not remove the imputation and regression assumptions.

When only some indicators from a categorical predictor are retained, an included indicator compares that level with the levels whose indicators were omitted, collectively. For example, retaining only one neighborhood indicator compares that neighborhood with the other neighborhoods, rather than with the original encoding reference alone. Other included predictors also enter the comparison.

Output names for the guided checks: `coefficient_summary`, one row per frozen feature with columns `term`, `coefficient` (the independent-fit estimate), `mean`, `sd`, `lower`, and `upper`, and `coefficient_draws`, one row per bootstrap.

Use `B_COEFFICIENT` resamples with `INFERENCE_BOOTSTRAP_SEED`, and save their positional index arrays as `bootstrap_indices`. Refit preprocessing on each sample. The supplied cell below defines `reference_sd` in the numeric-column order. For each retained numeric column `j`, multiply the fitted coefficient by `reference_sd[j] / fitted_prep.scaler_.scale_[j]`; use a multiplier of one for indicators. Apply the same conversion to the independent-fit point estimate. You may use `bootstrap_coefficient_maps(shared_preprocessor, X_inference, frozen_indices, reference_sd, bootstrap_indices)` to construct `coefficient_operators`; `coefficient_operators @ y_inference` gives the bootstrap coefficients. Keep those maps for Q3.6. Create the coefficient-interval plot yourself: show point estimates and their 95% bootstrap percentile intervals, label the predictors, and include a zero reference line. Use a descriptive title, a labeled effect axis, and a legend distinguishing the estimates from the intervals. If displaying a subset for readability, state how you chose it; the table must still include every frozen feature.

`checked_ols(shared_preprocessor, X_inference, y_inference, frozen_indices)` returns the fitted preprocessing, checked design, coefficients including

the intercept, residuals, and residual standard error. The returned dictionary uses keys `prep`, `design`, `coef`, `residuals`, and `sigma`.

With `evaluation=`, `checked_ols` also returns `prediction` and `evaluation_design` for those held-out rows.

In [26]:
```
# Supplied settings for coefficient uncertainty on the inference sample.
B_COEFFICIENT = 300
INFERENCE_BOOTSTRAP_SEED = 3109
# Express numeric coefficients per selection-sample SD; keep indicators on their 0/
reference_sd = (
    X_selection[prep_template.numeric].std(ddof=0).fillna(1).replace(0, 1).to_numpy
)
```

In [27]:
```
# Q3.3: OLS on the frozen features, fitted and bootstrapped on the independent infe
rng_coef = np.random.default_rng(INFERENCE_BOOTSTRAP_SEED)
n_inference = len(X_inference)
bootstrap_indices = rng_coef.integers(0, n_inference, size=(B_COEFFICIENT, n_infere
# Each map refits preprocessing on its resample and rescales numeric effects to ref
coefficient_operators = bootstrap_coefficient_maps(
    shared_preprocessor, X_inference, frozen_indices, reference_sd, bootstrap_indic
)
coefficient_draws = coefficient_operators @ y_inference  # (B_COEFFICIENT, number o

inference_fit = checked_ols(shared_preprocessor, X_inference, y_inference, frozen_i
n_numeric = len(prep_template.numeric)
reference_multiplier = np.array(
    [reference_sd[j] / inference_fit["prep"].scaler_.scale_[j] if j < n_numeric els
)
inference_estimate = inference_fit["coef"][1:] * reference_multiplier

coefficient_summary = pd.DataFrame(
    {
        "term": frozen_terms,
        "coefficient": inference_estimate,
        "mean": coefficient_draws.mean(axis=0),
        "sd": coefficient_draws.std(axis=0, ddof=1),
        "lower": np.percentile(coefficient_draws, 2.5, axis=0),
        "upper": np.percentile(coefficient_draws, 97.5, axis=0),
    }
)
coefficient_summary["units"] = ["per selection SD" if j < n_numeric else "indicator
coefficient_summary["interval_excludes_zero"] = (coefficient_summary.lower > 0) | (
print(f"Inference fit: n = {n_inference}, design columns = {inference_fit['design']
      f"residual SE = {inference_fit['sigma']:.4f} log price")
with pd.option_context("display.max_rows", 200, "display.width", 200):
    display(coefficient_summary.round(4))
print(f"{int(coefficient_summary.interval_excludes_zero.sum())} of {len(coefficient
      "95% bootstrap percentile intervals exclude zero.")

order = np.argsort(coefficient_summary["coefficient"].to_numpy())
ypos = np.arange(len(order))
cs = coefficient_summary.iloc[order]
fig, ax = plt.subplots(figsize=(9, 0.32 * len(order) + 1.5))
```

```
ax.hlines(ypos, cs["lower"], cs["upper"], color="tab:blue", lw=2, label="95% bootst
ax.plot(cs["coefficient"], ypos, "o", color="black", label="independent-fit estimat
ax.axvline(0, color="red", ls="--", label="zero (no association)")
ax.set_yticks(ypos, [f"{t} ({'SD' if u.startswith('per') else '0/1'})" for t, u in
ax.set_xlabel("Effect on log sale price (per selection SD for numeric; per 0/1 for
ax.set_title(f"Frozen-feature OLS on {n_inference} inference houses: {B_COEFFICIENT
ax.legend(fontsize=8, loc="lower right")
plt.tight_layout()
plt.show()

hist_terms = ["Gr_Liv_Area", "Lot_Frontage"]  # one clearly nonzero, one straddling
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, term in zip(axes, hist_terms):
    k = frozen_terms.index(term)
    ax.hist(coefficient_draws[:, k], bins=30, color="tab:blue", alpha=0.7, label="b
    ax.axvline(0, color="red", ls="--", label="zero")
    ax.axvline(inference_estimate[k], color="black", label="independent-fit estimat
    ax.set_xlabel(f"{term} coefficient (log price, {coefficient_summary.units[k]})"
    ax.set_ylabel("Number of bootstrap resamples")
    ax.set_title(f"Bootstrap distribution: {term}")
    ax.legend(fontsize=8)
plt.tight_layout()
plt.show()
```

```
Inference fit: n = 879, design columns = 69, residual SE = 0.1410 log price
```

	term	coefficient	mean	sd	lower	upper
0	Bldg_Type_Twnhs	-0.0540	-0.0510	0.0251	-0.1014	-0.0037
1	Bsmt_Full_Bath	0.0346	0.0309	0.0085	0.0148	0.0480
2	Central_Air_Y	0.1019	0.1016	0.0477	0.0207	0.2032
3	Condition_1_Norm	0.0377	0.0364	0.0150	0.0033	0.0645
4	Enclosed_Porch	0.0018	0.0021	0.0076	-0.0127	0.0157
5	Fireplaces	0.0353	0.0333	0.0067	0.0206	0.0462
6	Full_Bath	0.0152	0.0123	0.0105	-0.0068	0.0335
7	Garage_Cars	0.0378	0.0366	0.0075	0.0218	0.0506
8	Gr_Liv_Area	0.1399	0.1441	0.0182	0.1078	0.1788
9	Heating_QC_Typical	-0.0025	-0.0041	0.0133	-0.0272	0.0210
10	MS_SubClass_Two_Story_1945_and_Older	-0.0137	-0.0034	0.0431	-0.0913	0.0813
11	Neighborhood_Crawford	0.1419	0.1367	0.0286	0.0787	0.1935
12	Neighborhood_Edwards	-0.0363	-0.0315	0.0236	-0.0734	0.0172
13	Neighborhood_Northridge_Heights	0.0405	0.0348	0.0239	-0.0113	0.0824
14	Overall_Cond_Fair	-0.1958	-0.1951	0.0569	-0.3131	-0.0851
15	Overall_Cond_Good	0.0223	0.0253	0.0148	-0.0053	0.0522
16	Overall_Qual_Below_Average	-0.1424	-0.1379	0.0416	-0.2146	-0.0590

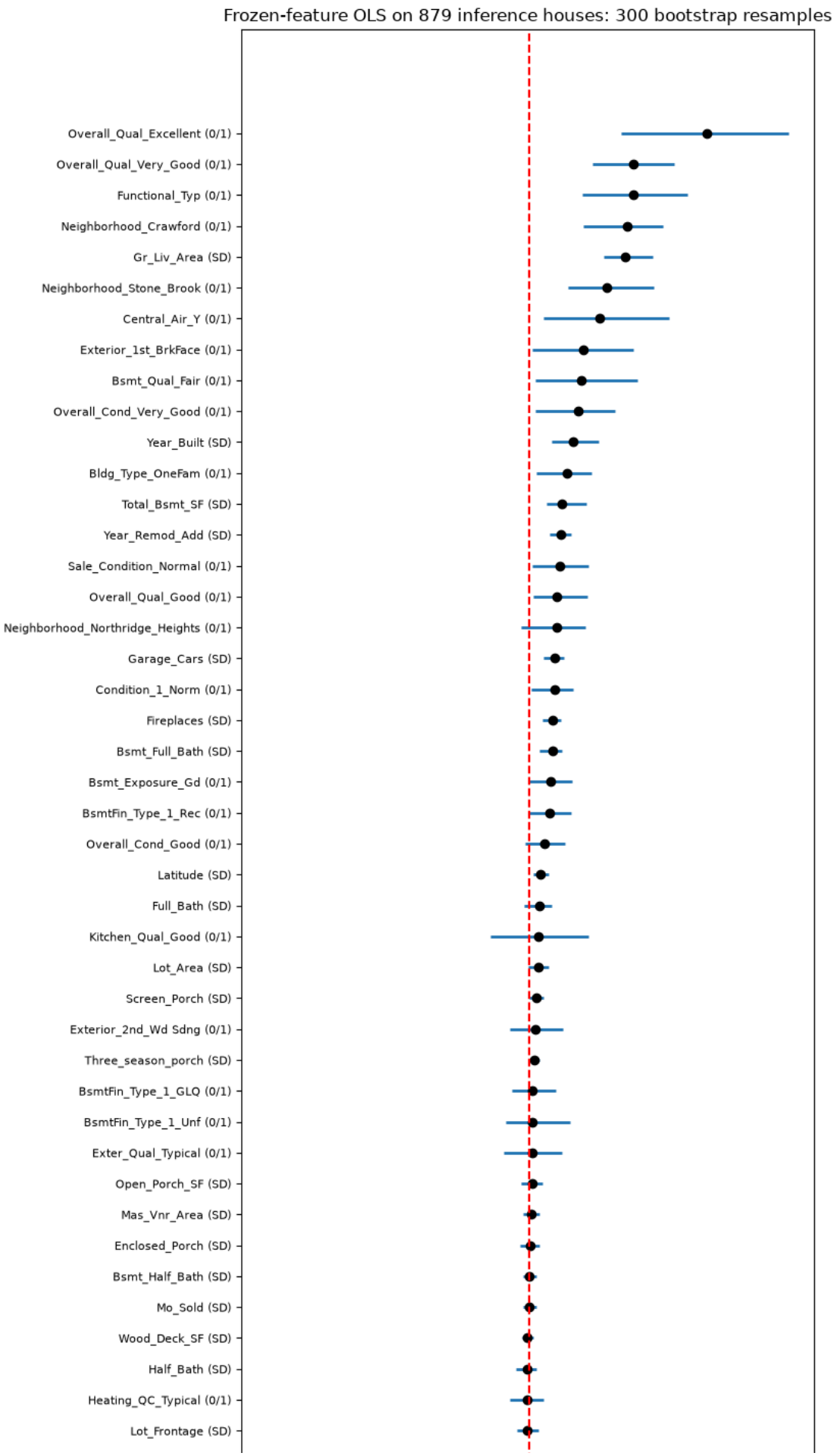
	term	coefficient	mean	sd	lower	upper
17	Overall_Qual_Excellent	0.2575	0.2430	0.0578	0.1337	0.3748
18	Overall_Qual_Good	0.0409	0.0408	0.0193	0.0067	0.0853
19	Overall_Qual_Very_Good	0.1516	0.1451	0.0309	0.0918	0.2100
20	Sale_Condition_Normal	0.0452	0.0409	0.0207	0.0046	0.0863
21	Screen_Porch	0.0106	0.0098	0.0052	0.0004	0.0212
22	Total_Bsmt_SF	0.0478	0.0557	0.0158	0.0262	0.0838
23	Wood_Deck_SF	-0.0022	-0.0013	0.0041	-0.0093	0.0065
24	Year_Built	0.0636	0.0678	0.0168	0.0331	0.1007
25	Year_Remod_Add	0.0464	0.0448	0.0084	0.0302	0.0612
26	Year_Sold	-0.0067	-0.0060	0.0047	-0.0148	0.0030
27	BsmtFin_Type_1_GLQ	0.0052	0.0078	0.0158	-0.0251	0.0393
28	Bsmt_Exposure_Gd	0.0322	0.0314	0.0166	-0.0014	0.0620
29	BsmtFin_Type_1_Unf	0.0049	0.0110	0.0231	-0.0327	0.0601
30	Functional_Typ	0.1507	0.1511	0.0403	0.0780	0.2287
31	Kitchen_AbvGr	-0.0113	-0.0139	0.0072	-0.0275	0.0006
32	Kitchen_Qual_Typical	-0.0031	-0.0064	0.0438	-0.0752	0.0771

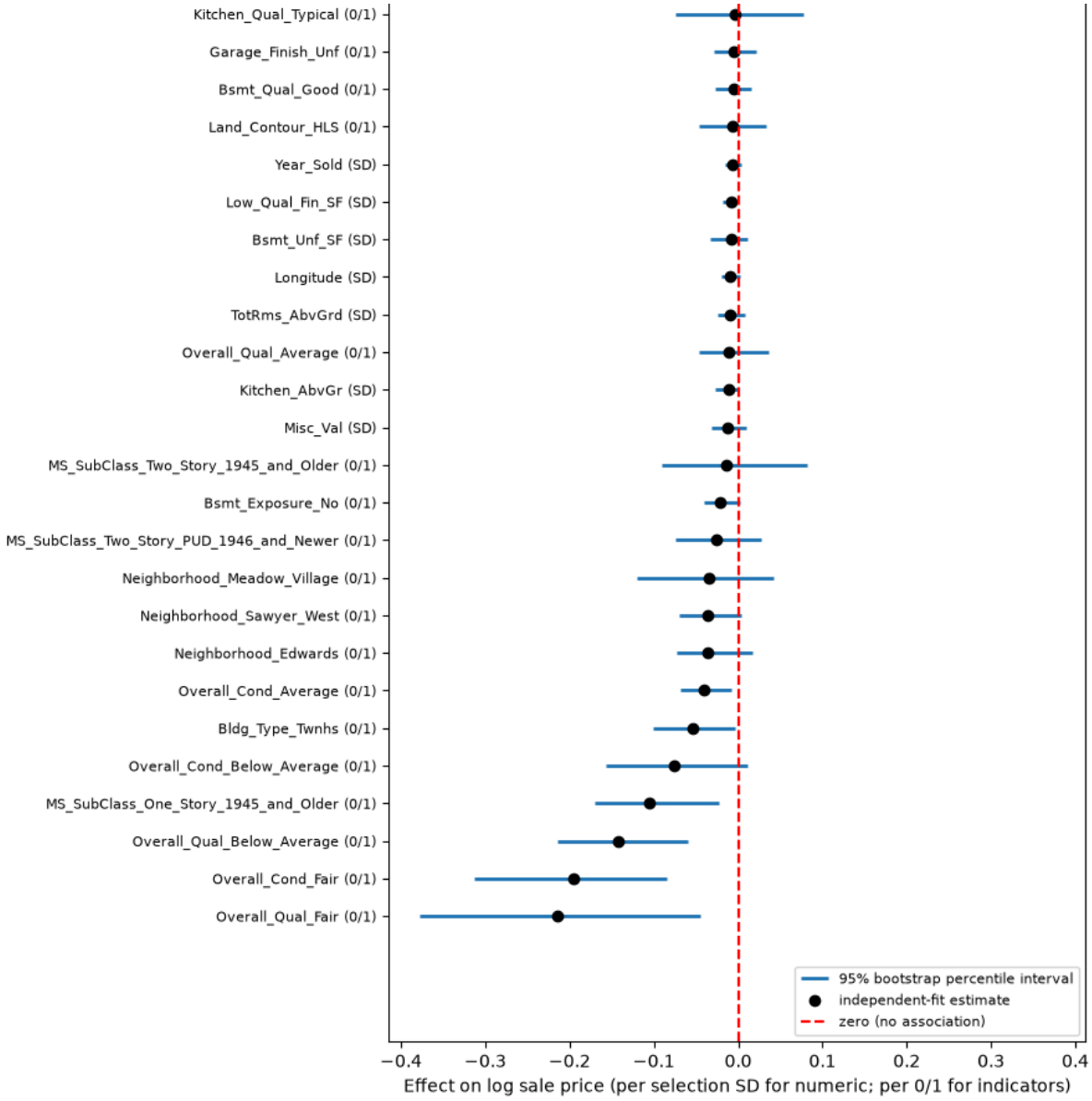
	term	coefficient	mean	sd	lower	upper
33	Latitude	0.0170	0.0161	0.0059	0.0064	0.0287
34	Open_Porch_SF	0.0045	0.0061	0.0078	-0.0107	0.0193
35	Bldg_Type_OneFam	0.0552	0.0479	0.0202	0.0112	0.0899
36	Land_Contour_HLS	-0.0060	-0.0078	0.0216	-0.0464	0.0335
37	Longitude	-0.0094	-0.0090	0.0060	-0.0201	0.0019
38	Bsmt_Unf_SF	-0.0086	-0.0142	0.0118	-0.0338	0.0113
39	Exterior_1st_BrkFace	0.0793	0.0816	0.0388	0.0048	0.1516
40	Exter_Qual_Typical	0.0048	0.0045	0.0229	-0.0368	0.0478
41	MS_SubClass_One_Story_1945_and_Older	-0.1059	-0.0966	0.0374	-0.1698	-0.0228
42	Overall_Cond_Average	-0.0399	-0.0390	0.0151	-0.0689	-0.0081
43	Overall_Qual_Average	-0.0106	-0.0082	0.0223	-0.0470	0.0363
44	TotRms_AbvGrd	-0.0099	-0.0074	0.0086	-0.0242	0.0087
45	Lot_Frontage	-0.0025	-0.0019	0.0079	-0.0164	0.0141
46	Half_Bath	-0.0025	-0.0037	0.0076	-0.0191	0.0103
47	Neighborhood_Stone_Brook	0.1131	0.1137	0.0327	0.0572	0.1807
48	Overall_Cond_Below_Average	-0.0763	-0.0796	0.0419	-0.1577	0.0107
49	Exterior_2nd_Wd Sdng	0.0092	0.0107	0.0183	-0.0274	0.0496

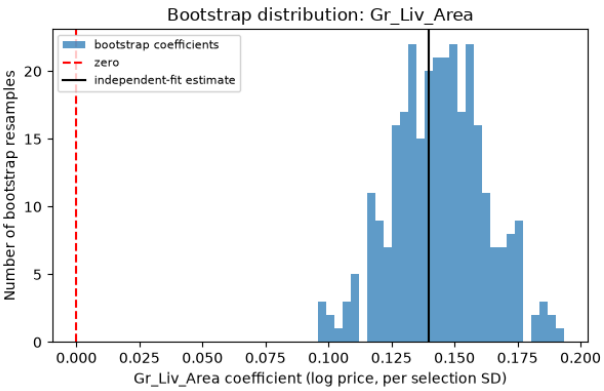
	term	coefficient	mean	sd	lower	upper
50	Low_Qual_Fin_SF	-0.0076	-0.0079	0.0046	-0.0184	0.0001
51	Mas_Vnr_Area	0.0031	0.0048	0.0058	-0.0076	0.0151
52	Bsmt_Half_Bath	0.0011	0.0021	0.0049	-0.0086	0.0114
53	Neighborhood_Sawyer_West	-0.0356	-0.0359	0.0195	-0.0705	0.0041
54	Lot_Area	0.0136	0.0147	0.0080	-0.0003	0.0290
55	Overall_Qual_Fair	-0.2144	-0.2087	0.0861	-0.3774	-0.0450
56	Three_season_porch	0.0074	0.0075	0.0030	0.0020	0.0149
57	Bsmt_Qual_Fair	0.0764	0.0771	0.0372	0.0091	0.1563
58	Garage_Finish_Unf	-0.0049	-0.0033	0.0131	-0.0295	0.0209
59	Kitchen_Qual_Good	0.0141	0.0108	0.0399	-0.0546	0.0860
60	Neighborhood_Meadow_Village	-0.0347	-0.0399	0.0423	-0.1202	0.0425
61	BsmtFin_Type_1_Rec	0.0304	0.0321	0.0165	-0.0005	0.0613
62	MS_SubClass_Two_Story_PUD_1946_and_Newer	-0.0259	-0.0262	0.0261	-0.0742	0.0273
63	Misc_Val	-0.0130	-0.0101	0.0124	-0.0316	0.0096
64	Bsmt_Exposure_No	-0.0215	-0.0198	0.0114	-0.0408	0.0022
65	Mo_Sold	0.0002	0.0013	0.0055	-0.0087	0.0115
66	Bsmt_Qual_Good	-0.0058	-0.0068	0.0121	-0.0277	0.0160

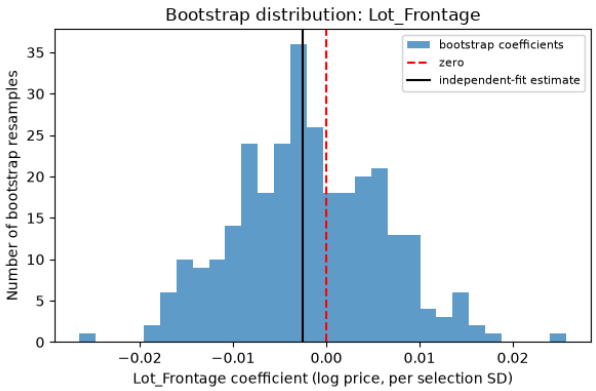
	term	coefficient	mean	sd	lower	upper
67	Overall_Cond_Very_Good	0.0716	0.0618	0.0288	0.0092	0.1251

29 of 68 95% bootstrap percentile intervals exclude zero.


Frozen-feature OLS on 879 inference houses: 300 bootstrap resamples
Overall_Qual_Excellent (0/1)
Overall_Qual_Very_Good (0/1)
Functional_Typ (0/1)
Neighborhood_Crawford (0/1)
Gr_Liv_Area (SD)
Neighborhood_Stone_Brook (0/1)
Central_Air_Y (0/1)
Exterior_1st_BrkFace (0/1)
Bsmt_Qual_Fair (0/1)
Overall_Cond_Very_Good (0/1)
Year_Built (SD)
Bldg_Type_OneFam (0/1)
Total_Bsmt_SF (SD)
Year_Remod_Add (SD)
Sale_Condition_Normal (0/1)
Overall_Qual_Good (0/1)
Neighborhood_Northridge_Heights (0/1)
Garage_Cars (SD)
Condition_1_Norm (0/1)
Fireplaces (SD)
Bsmt_Full_Bath (SD)
Bsmt_Exposure_Gd (0/1)
BsmtFin_Type_1_Rec (0/1)
Overall_Cond_Good (0/1)
Latitude (SD)
Full_Bath (SD)
Kitchen_Qual_Good (0/1)
Lot_Area (SD)
Screen_Porch (SD)
Exterior_2nd_Wd Sdng (0/1)
Three_season_porch (SD)
BsmtFin_Type_1_GLQ (0/1)
BsmtFin_Type_1_Unf (0/1)
Exter_Qual_Typical (0/1)
Open_Porch_SF (SD)
Mas_Vnr_Area (SD)
Enclosed_Porch (SD)
Bsmt_Half_Bath (SD)
Mo_Sold (SD)
Wood_Deck_SF (SD)
Half_Bath (SD)
Heating_QC_Typical (0/1)
Lot_Frontage (SD)







In [28]:
```
grader.check("q3_3")
```

Out[28]:

q3_3 passed! 🎉

Written response — Q3.3

We fitted OLS with the 68 frozen features on the 879 independent inference houses (residual SE 0.141 log-price units) and bootstrapped it 300 times, refitting preprocessing each time, with numeric effects per selection-sample SD. **29 of the 68 percentile intervals (95%) exclude zero**, for example `Overall_Qual_Excellent` +0.257 [0.134, 0.375] (about 30% higher price than the omitted levels), `Overall_Cond_Fair` −0.196, `Neighborhood_Crawford` +0.142 and `Gr_Liv_Area` +0.140 [0.108, 0.179] per SD (about 15% higher price; its histogram lies entirely right of zero). Many frequently selected features have intervals around zero: `Lot_Frontage` (−0.003 [−0.016, 0.014]; its histogram straddles zero), `Mo_Sold`, `Enclosed_Porch`, `Wood_Deck_SF`. Indicators compared with a small omitted group are imprecise (`Kitchen_Qual_Typical` spans about ±0.08).

These results support conditional associations: after adjusting for the other 67 features, larger living area, newer or remodeled houses, higher quality and condition, and certain neighborhoods go with higher log price. The inference prices played no part in selection, so the intervals are a fair assessment; none of this is causal. An included indicator compares its level with all omitted levels together, and correlated predictors share credit (`TotRms_AbvGrd` is slightly negative given living area).

Assumptions. Houses are independent draws from one population (rows resampled as pairs); log price is linear in the frozen features; neighborhood-median frontage imputation, relearned per resample, is adequate only if missingness is close to MAR given neighborhood; the percentile bootstrap is a large-sample approximation, weakest for rare indicators. The intervals describe this independent fit, not the Q3.8 refit.

Q3.4. Measure predictive importance

5 points

The coefficient intervals describe uncertainty about associations within the fitted model. They do not directly tell us how much the model relies on each input to make accurate predictions, particularly when predictors carry overlapping information. Measuring changes in held-out prediction error gives us a separate way to investigate that predictive reliance.

Task: Use CV within the inference data to evaluate OLS with the frozen features. Use `inference_folds`; for each fold, record the error on its held-out rows. Repeatedly shuffle each original predictor represented in the model among the held-out houses, pass the changed inputs through the fitted preprocessing, and calculate the increase in prediction loss. Keep the fitted model fixed during shuffling.

Artifacts: `importance_baselines`, with columns `fold`, `held_out_mse`, and `held_out_count`; `importance_summary`, indexed by original predictor, with columns `mean` and `std` for the MSE increase of log price; and distributions of those increases. MSE of log price is measured in squared log-price units.

Constraints: Shuffle an original categorical column as a whole, rather than shuffling its encoded indicators independently. Discuss how correlated inputs affect interpretation. Variation across shuffles does not capture all uncertainty from fitting the model. Do not use these inference diagnostics to revise the frozen predictor set.

Output names for the guided checks: `importance_summary`, `importance_baselines`, and `importance_repeats` (fold, source, replicate, degradation).

Use `N_SHUFFLES` perturbations per predictor per fold. `checked_ols(..., evaluation=X_inference.iloc[va])` supplies held-out predictions. Assess original predictors supplying retained features. If `Lot_Frontage` is among the frozen features, also assess `Neighborhood`, even if no neighborhood indicator survives, because the shared preprocessor uses neighborhood to fill unknown frontage. Record the original predictor for every encoded feature using `feature_sources`.

In [29]:
```
# Supplied folds and shuffle count for held-out predictive importance.
inference_folds = fixed_folds(len(X_inference), seed=110)
N_SHUFFLES = 30
```

In [30]:
```
# Q3.4: held-out permutation importance of each original predictor behind the froze
importance_sources = list(dict.fromkeys(feature_sources[j] for j in frozen_indices)
if "Lot_Frontage" in importance_sources and "Neighborhood" not in importance_source
    importance_sources.append("Neighborhood")  # neighborhood fills unknown frontag
print("Original predictors assessed:", importance_sources)

rng_importance = np.random.default_rng(4109)
baseline_rows, repeat_rows = [], []
for fold, (tr, va) in enumerate(inference_folds):
    x_va = X_inference.iloc[va]
    y_va = y_inference[va]
    fit = checked_ols(shared_preprocessor, X_inference.iloc[tr], y_inference[tr], f
    base_mse = float(np.mean((y_va - fit["prediction"]) ** 2))
    baseline_rows.append({"fold": fold, "held_out_mse": base_mse, "held_out_count":
    for source in importance_sources:
        for r in range(N_SHUFFLES):
            shuffled = x_va.copy()
            # Shuffle the raw column as a whole, then reuse the fitted preprocessin
            shuffled[source] = rng_importance.permutation(shuffled[source].to_numpy
            design = np.column_stack([np.ones(len(va)), fit["prep"].transform(shuff
            mse = float(np.mean((y_va - design @ fit["coef"]) ** 2))
            repeat_rows.append({"fold": fold, "source": source, "replicate": r, "de

importance_baselines = pd.DataFrame(baseline_rows)
importance_repeats = pd.DataFrame(repeat_rows)
importance_summary = (
    importance_repeats.groupby("source")["degradation"].agg(["mean", "std"]).sort_v
)
importance_summary.index.name = "original predictor"
importance_summary["fold means (min to max)"] = (
    importance_repeats.groupby(["source", "fold"])["degradation"].mean().groupby("s
    .agg(lambda s: f"{s.min():.4f} to {s.max():.4f}")
)
```

```
display(importance_baselines.round(5))
print(f"Pooled held-out MSE: {np.average(importance_baselines.held_out_mse, weights
display(importance_summary.round(5))

order = importance_summary.index.tolist()
fig, ax = plt.subplots(figsize=(10, 0.4 * len(order) + 1.5))
ax.boxplot([importance_repeats.loc[importance_repeats.source == s, "degradation"] f
           orientation="horizontal", tick_labels=order, showfliers=True)
ax.scatter(importance_summary["mean"], np.arange(1, len(order) + 1), color="red", z
ax.axvline(0, color="gray", ls="--", label="no change")
ax.invert_yaxis()
ax.set_xlabel("Increase in held-out MSE of log price after shuffling (squared log-p
ax.set_title(f"Permutation importance: {len(inference_folds)} folds x {N_SHUFFLES}
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()
```

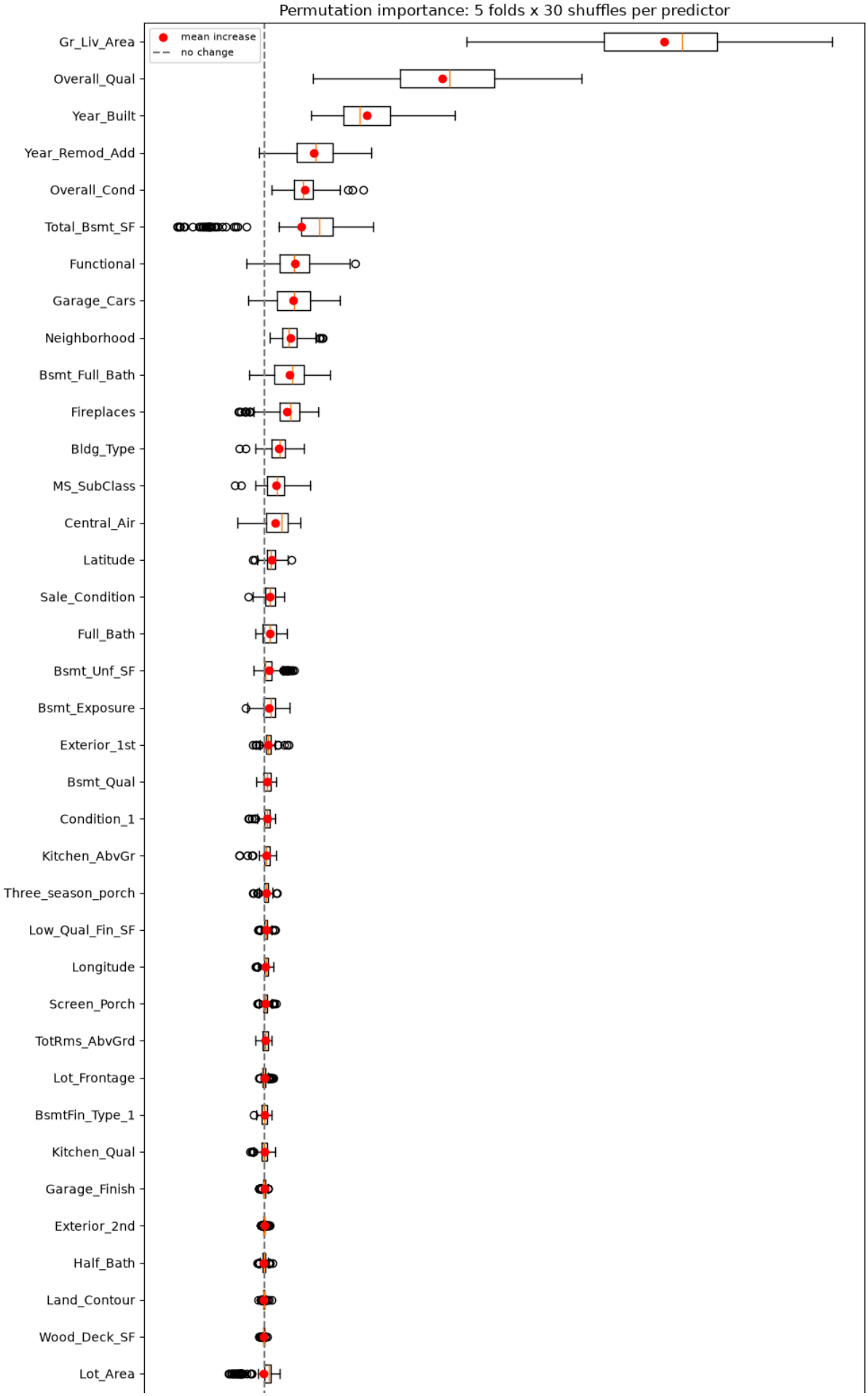
```
Original predictors assessed: ['Bldg_Type', 'Bsmt_Full_Bath', 'Central_Air', 'Condit
ion_1', 'Enclosed_Porch', 'Fireplaces', 'Full_Bath', 'Garage_Cars', 'Gr_Liv_Area',
'Heating_QC', 'MS_SubClass', 'Neighborhood', 'Overall_Cond', 'Overall_Qual', 'Sale_C
ondition', 'Screen_Porch', 'Total_Bsmt_SF', 'Wood_Deck_SF', 'Year_Built', 'Year_Remo
d_Add', 'Year_Sold', 'BsmtFin_Type_1', 'Bsmt_Exposure', 'Functional', 'Kitchen_AbvG
r', 'Kitchen_Qual', 'Latitude', 'Open_Porch_SF', 'Land_Contour', 'Longitude', 'Bsmt_
Unf_SF', 'Exterior_1st', 'Exter_Qual', 'TotRms_AbvGrd', 'Lot_Frontage', 'Half_Bath',
'Exterior_2nd', 'Low_Qual_Fin_SF', 'Mas_Vnr_Area', 'Bsmt_Half_Bath', 'Lot_Area', 'Th
ree_season_porch', 'Bsmt_Qual', 'Garage_Finish', 'Misc_Val', 'Mo_Sold']
```

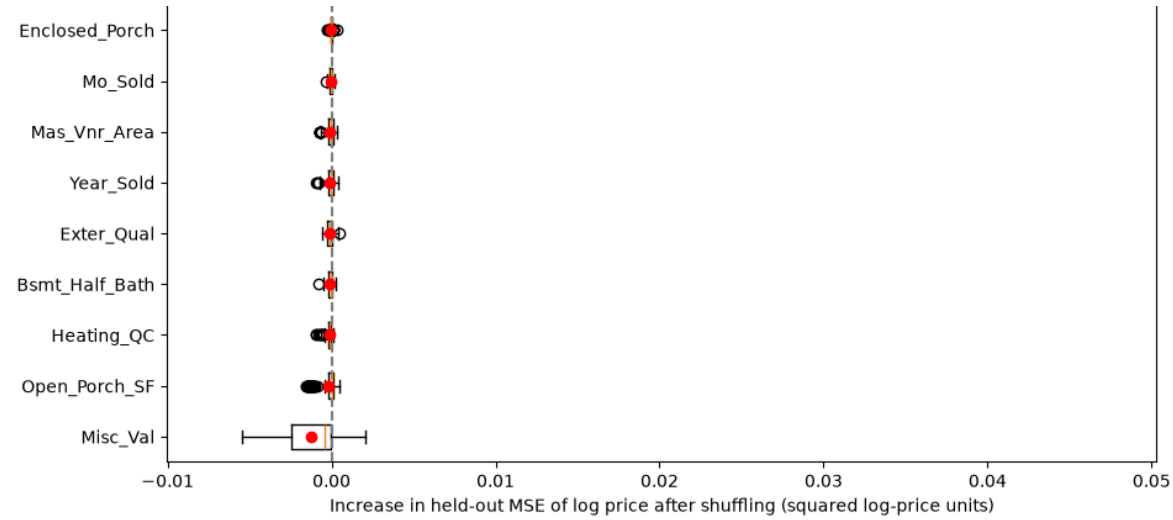
	fold	held_out_mse	held_out_count
0	0	0.02885	176
1	1	0.01697	176
2	2	0.04117	176
3	3	0.02121	176
4	4	0.03903	175

```
Pooled held-out MSE: 0.02944
```

original predictor	mean	std	fold means (min to max)
Gr_Liv_Area	0.03349	0.00673	0.0247 to 0.0401
Overall_Qual	0.01495	0.00557	0.0068 to 0.0211
Year_Built	0.00863	0.00269	0.0061 to 0.0122
Year_Remod_Add	0.00419	0.00208	0.0026 to 0.0066
Overall_Cond	0.00341	0.00135	0.0026 to 0.0050
Total_Bsmt_SF	0.00312	0.00426	-0.0049 to 0.0061
Functional	0.00261	0.00167	0.0010 to 0.0045
Garage_Cars	0.00245	0.00177	0.0002 to 0.0042
Neighborhood	0.00222	0.00098	0.0015 to 0.0034
Bsmt_Full_Bath	0.00216	0.00164	0.0000 to 0.0039
Fireplaces	0.00194	0.00152	-0.0005 to 0.0030
Bldg_Type	0.00124	0.00091	0.0003 to 0.0020
MS_SubClass	0.00102	0.00096	0.0000 to 0.0021
Central_Air	0.00097	0.00133	-0.0010 to 0.0020
Latitude	0.00063	0.00054	0.0003 to 0.0012
Sale_Condition	0.00049	0.00059	-0.0000 to 0.0011
Full_Bath	0.00047	0.00067	-0.0003 to 0.0012
Bsmt_Unf_SF	0.00042	0.00070	-0.0002 to 0.0015
Bsmt_Exposure	0.00037	0.00081	-0.0008 to 0.0011
Exterior_1st	0.00036	0.00039	0.0002 to 0.0006
Bsmt_Qual	0.00026	0.00038	-0.0001 to 0.0006
Condition_1	0.00024	0.00041	-0.0001 to 0.0006
Kitchen_AbvGr	0.00021	0.00043	-0.0001 to 0.0005
Three_season_porch	0.00018	0.00030	-0.0002 to 0.0004
Low_Qual_Fin_SF	0.00016	0.00023	0.0001 to 0.0003
Longitude	0.00014	0.00030	-0.0001 to 0.0004
Screen_Porch	0.00012	0.00026	-0.0001 to 0.0003
TotRms_AbvGrd	0.00009	0.00030	-0.0001 to 0.0004
Lot_Frontage	0.00004	0.00022	-0.0001 to 0.0004

original predictor	mean	std	fold means (min to max)
BsmtFin_Type_1	0.00002	0.00028	-0.0002 to 0.0003
Kitchen_Qual	0.00002	0.00040	-0.0005 to 0.0004
Garage_Finish	0.00001	0.00015	-0.0001 to 0.0002
Exterior_2nd	0.00000	0.00012	-0.0001 to 0.0001
Half_Bath	-0.00001	0.00019	-0.0002 to 0.0002
Land_Contour	-0.00002	0.00010	-0.0000 to 0.0000
Wood_Deck_SF	-0.00003	0.00009	-0.0001 to 0.0000
Lot_Area	-0.00003	0.00107	-0.0020 to 0.0006
Enclosed_Porch	-0.00003	0.00006	-0.0001 to -0.0000
Mo_Sold	-0.00005	0.00008	-0.0001 to 0.0000
Mas_Vnr_Area	-0.00009	0.00029	-0.0006 to 0.0002
Year_Sold	-0.00010	0.00025	-0.0003 to 0.0001
Exter_Qual	-0.00012	0.00021	-0.0003 to 0.0001
Bsmt_Half_Bath	-0.00013	0.00018	-0.0004 to 0.0000
Heating_QC	-0.00013	0.00017	-0.0003 to -0.0000
Open_Porch_SF	-0.00022	0.00056	-0.0013 to 0.0002
Misc_Val	-0.00121	0.00171	-0.0040 to -0.0000


Permutation importance: 5 folds x 30 shuffles per predictor
mean increase
no change
Gr_Liv_Area
Overall_Qual
Year_Built
Year_Remod_Add
Overall_Cond
Total_Bsmt_SF
Functional
Garage_Cars
Neighborhood
Bsmt_Full_Bath
Fireplaces
Bldg_Type
MS_SubClass
Central_Air
Latitude
Sale_Condition
Full_Bath
Bsmt_Unf_SF
Bsmt_Exposure
Exterior_1st
Bsmt_Qual
Condition_1
Kitchen_AbvGr
Three_season_porch
Low_Qual_Fin_SF
Longitude
Screen_Porch
TotRms_AbvGrd
Lot_Frontage
BsmtFin_Type_1
Kitchen_Qual
Garage_Finish
Exterior_2nd
Half_Bath
Land_Contour
Wood_Deck_SF
Lot_Area



In [31]:
```
grader.check("q3_4")
```

Out[31]:

q3_4 passed! 🌈

Written response — Q3.4

Pooled over the five inference folds, the frozen OLS model's held-out MSE is 0.0294 (RMSE ≈ 0.17; folds 0.017–0.041). Shuffling `Gr_Liv_Area` among held-out houses raises MSE by 0.0335 on average, more than doubling it, so the model relies on living area most; next come `Overall_Qual` (0.0150, all indicators shuffled together), `Year_Built` (0.0086), `Year_Remod_Add` (0.0042), `Overall_Cond` (0.0034) and `Neighborhood` (0.0022, including its role in filling frontage). Most other inputs barely matter given the rest of the model: 21 of the 46 assessed predictors change MSE by less than 0.0002 in absolute value, and some means are slightly negative (`Lot_Area` ≈ 0; `Misc_Val` −0.0012, negative in every fold, consistent with an overfit term, see Extension 1), which on small held-out folds is mostly noise.

Correlated inputs limit interpretation. Living area and rooms above ground overlap, as do `Year_Built` and `Year_Remod_Add` : the fitted model splits the weight, and shuffling one leaves the other carrying the information, so `TotRms_AbvGrd` scores near zero (0.0001) despite a clear price relationship. Conversely, shuffling one of a correlated pair creates unrealistic houses, which can inflate importance. These scores measure how much this fitted model relies on an input, not the input's own association with price. Spread across shuffles understates uncertainty because coefficients are fixed within a fold; fold spread shows some fitting variability (`Total_Bsmt_SF` : −0.005 to 0.006). None of this changed the frozen set.

Q3.5. Quantify prediction uncertainty

6 points

We have examined the inputs used by the model, but a reported price prediction also needs a statement of uncertainty. Uncertainty in the fitted coefficients affects our estimate of the expected log price for a house with given characteristics. An individual sale can also differ from that expectation, so an interval for one future sale answers a different question from an interval for the conditional mean.

Task: Following Lecture 8, construct 95% intervals using `inference_folds` and `B_INTERVAL` bootstrap fits per training fold. The expected-response draws are bootstrap predictions of log price. For individual-outcome draws, add normal noise with mean zero and standard deviation equal to each bootstrap fit's residual standard error; use `NOISE_DRAWS_PER_FIT` noise draws per fit. Compare both interval types and assess held-out individual-interval coverage and average width in log-price units. State the assumptions of this noise model.

Artifacts: A table of interval bounds for every held-out inference house; a plot comparing both interval types and observed prices for the eight inference houses with the smallest row IDs; distributions of expected-response and individual-outcome draws for one of those houses; and a summary of prediction-interval coverage and average width.

Constraints: Exclude each assessed house from all fits used to construct its intervals. Use residual degrees of freedom equal to the fitting-sample size minus the design-matrix rank, including the intercept. Observed sale-price coverage assesses individual prediction intervals, not confidence intervals for the conditional mean. If you exponentiate bounds for expected log price, do not label them as intervals for arithmetic mean price.

Output names for the guided checks: `interval_results`, one row per inference observation.

Use the smallest eight labels in `X_inference.index` for the comparison plot. `interval_results` should include `row_id`, `fold`, `actual`, `prediction`, `mean_low`, `mean_high`, `prediction_low`, and `prediction_high`. These bounds are on the log-price scale; label any exponentiated display accordingly.

In [32]:
```
# Supplied counts for prediction intervals; reuse inference_folds from Q3.4.
B_INTERVAL = 200
NOISE_DRAWS_PER_FIT = 20
```

In [33]:
```
# Q3.5: bootstrap confidence and prediction intervals for every held-out inference
rng_interval = np.random.default_rng(5109)
interval_frames = []
draw_store = {}
redraws = 0
for fold, (tr, va) in enumerate(inference_folds):
    x_tr, y_tr = X_inference.iloc[tr], y_inference[tr]
    x_va = X_inference.iloc[va]
    point = checked_ols(shared_preprocessor, x_tr, y_tr, frozen_indices, evaluation
    mean_draws = np.empty((B_INTERVAL, len(va)))
    sigmas = np.empty(B_INTERVAL)
```

```
    b = 0
    while b < B_INTERVAL:
        idx = rng_interval.integers(0, len(tr), size=len(tr))
        try:
            fit = checked_ols(shared_preprocessor, x_tr.iloc[idx], y_tr[idx], froze
        except ValueError:
            redraws += 1  # a rare indicator vanished from this resample; draw agai
            continue
        mean_draws[b] = fit["prediction"]
        sigmas[b] = fit["sigma"]  # residual SE with df = resample size - design ra
        b += 1
    noise = rng_interval.normal(size=(B_INTERVAL, NOISE_DRAWS_PER_FIT, len(va))) *
    outcome_draws = (mean_draws[:, None, :] + noise).reshape(-1, len(va))
    frame = pd.DataFrame(
        {
            "row_id": x_va.index,
            "fold": fold,
            "actual": y_inference[va],
            "prediction": point["prediction"],
            "mean_low": np.percentile(mean_draws, 2.5, axis=0),
            "mean_high": np.percentile(mean_draws, 97.5, axis=0),
            "prediction_low": np.percentile(outcome_draws, 2.5, axis=0),
            "prediction_high": np.percentile(outcome_draws, 97.5, axis=0),
        }
    )
    interval_frames.append(frame)
    for k, row_id in enumerate(x_va.index):
        draw_store[row_id] = (mean_draws[:, k], outcome_draws[:, k])

interval_results = pd.concat(interval_frames).sort_values("row_id").reset_index(dro
print("Resamples redrawn because of an unidentified coefficient:", redraws)
display(interval_results.head(10).round(4))

covered = interval_results.actual.between(interval_results.prediction_low, interval
mean_covered = interval_results.actual.between(interval_results.mean_low, interval_
interval_summary = pd.DataFrame(
    {
        "interval": ["prediction (individual sale)", "confidence (expected log pric
        "observed-price coverage": [covered.mean(), mean_covered.mean()],
        "average width (log price)": [
            (interval_results.prediction_high - interval_results.prediction_low).me
            (interval_results.mean_high - interval_results.mean_low).mean(),
        ],
    }
)
display(interval_summary.round(4))
print("Prediction-interval coverage by fold:", interval_results.assign(c=covered).g
print(f"Average prediction-interval width {interval_summary.iloc[0, 2]:.3f} log uni
      f"is a factor of about {np.exp(interval_summary.iloc[0, 2]):.2f} between the

first8 = interval_results.nsmallest(8, "row_id")
x = np.arange(len(first8))
fig, ax = plt.subplots(figsize=(11, 4.5))
ax.vlines(x - 0.1, first8.prediction_low, first8.prediction_high, color="tab:orange
ax.vlines(x + 0.1, first8.mean_low, first8.mean_high, color="tab:blue", lw=6, label
ax.plot(x, first8.prediction, "_", color="black", ms=20, mew=2, label="point predic
```

```
ax.plot(x, first8.actual, "D", color="red", label="observed log sale price")
ax.set_xticks(x, [f"row {r}\n(fold {f})" for r, f in zip(first8.row_id, first8.fold
ax.set_xlabel("Inference house (eight smallest row IDs)")
ax.set_ylabel("log(sale price in dollars)")
ax.set_title("Held-out intervals: expected log price versus one future sale")
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()

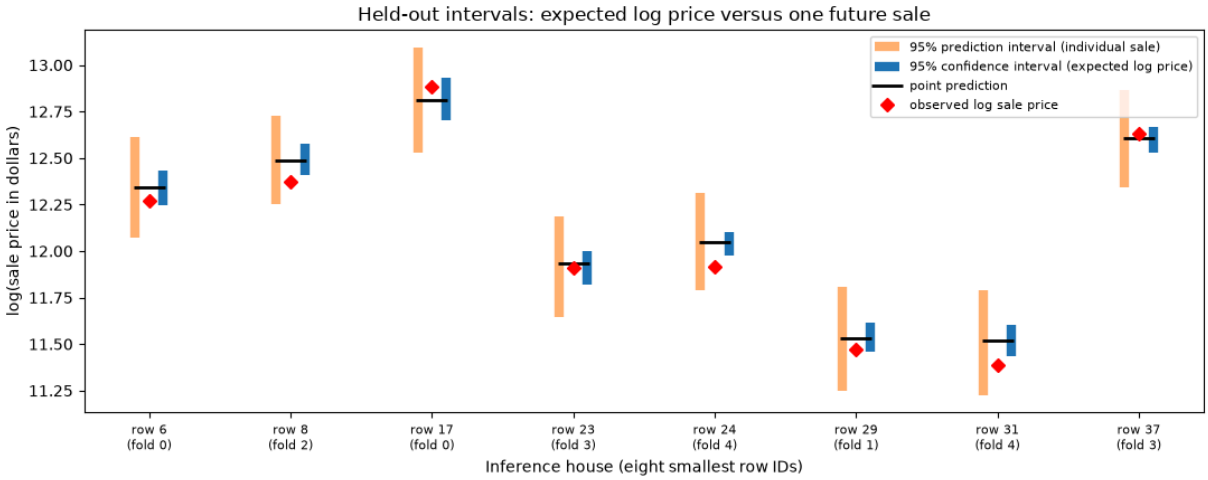
example_id = int(first8.row_id.iloc[0])
mean_d, outcome_d = draw_store[example_id]
example = interval_results.set_index("row_id").loc[example_id]
fig, ax = plt.subplots(figsize=(9, 4))
bins = np.linspace(outcome_d.min(), outcome_d.max(), 60)
ax.hist(outcome_d, bins=bins, density=True, alpha=0.5, color="tab:orange", label=f"
ax.hist(mean_d, bins=bins, density=True, alpha=0.7, color="tab:blue", label=f"expec
ax.axvline(example.actual, color="red", lw=2, label="observed log sale price")
ax.set_xlabel("log(sale price in dollars)")
ax.set_ylabel("Density")
ax.set_title(f"Bootstrap draws for inference house row {example_id}")
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()
```

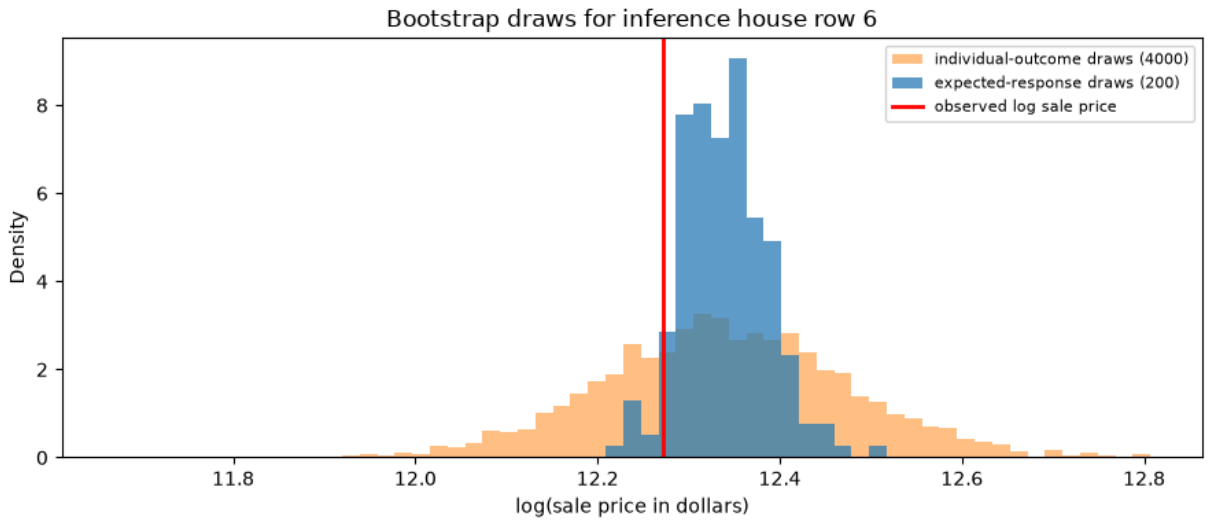
```
Resamples redrawn because of an unidentified coefficient: 0
```

	row_id	fold	actual	prediction	mean_low	mean_high	prediction_low	prediction_high
0	6	0	12.2714	12.3448	12.2445	12.4321	12.0744	12.6144
1	8	2	12.3737	12.4883	12.4058	12.5760	12.2538	12.7260
2	17	0	12.8852	12.8134	12.7019	12.9337	12.5263	13.0946
3	23	3	11.9117	11.9345	11.8215	11.9978	11.6441	12.1878
4	24	4	11.9177	12.0476	11.9777	12.1023	11.7875	12.3135
5	29	1	11.4721	11.5339	11.4584	11.6150	11.2508	11.8105
6	31	4	11.3851	11.5212	11.4375	11.6026	11.2269	11.7876
7	37	3	12.6313	12.6078	12.5312	12.6679	12.3398	12.8632
8	40	1	12.3014	12.2736	12.1736	12.3540	11.9890	12.5469
9	42	4	12.4646	12.5592	12.4582	12.6642	12.2767	12.8286

	interval	observed-price coverage	average width (log price)
0	prediction (individual sale)	0.9625	0.5457
1	confidence (expected log price)	0.6121	0.1957

```
Prediction-interval coverage by fold: {0: 0.966, 1: 0.972, 2: 0.938, 3: 0.96, 4: 0.9
77}
Average prediction-interval width 0.546 log units is a factor of about 1.73 between
the upper and lower price bounds.
```





In [34]:
```
grader.check("q3_5")
```

Out[34]:

q3_5 passed! 🚀

Written response — Q3.5

The two intervals answer different questions. The confidence interval for the **expected** log price reflects coefficient uncertainty only (average width 0.196 log-price units). The prediction interval for an **individual sale** adds house-to-house noise (average width 0.546: a factor of 1.73 between the upper and lower price bounds, about +31% / −24% around the prediction). Noise dominates. In the plot of the eight smallest row IDs every confidence interval sits inside a much longer prediction interval, and rows 8, 24 and 31 fall outside the confidence interval but inside the prediction interval; the row 6 histograms show concentrated expected-response draws against much wider individual-outcome draws.

Coverage. 96.2% of the 879 held-out log prices fall inside their 95% prediction intervals (folds 93.8–97.7%), close to nominal. The confidence intervals cover only 61%, as expected: they target the mean, not individual sales.

Noise-model assumptions. Each log price equals its expected value plus independent normal noise with mean zero and a constant SD for all houses, estimated by each bootstrap fit's residual standard error (df = resample size − design rank). The frozen model must also be correct on the log scale, frontage imputation adequate and houses independent. If expensive or abnormal sales have larger noise, their intervals are too narrow and others' too wide, which overall coverage can hide.

Q3.6. Assess global statistical evidence

4 points

The preceding analyses describe selection stability, estimated associations, and predictive reliance. We can also ask how unusual the overall coefficient evidence would be if prices had no relationship with the predictor set. Permuting the responses provides a reference distribution for that global question, which differs from testing one coefficient conditional on the others.

Task: Reuse the Q3.3 bootstrap rows and `coefficient_operators`. Calculate observed coefficient draws as `coefficient_operators @ y_inference`, and compute each coefficient's bootstrap ratio using the lecture notation, $$\hat{t}_j = \frac{\overline{\beta^*_j}}{\mathrm{SD}(\beta^*_j)}.$$ For our single global test, take the largest absolute ratio, $$\hat{t}_{\max,\mathrm{obs}} = \max_j |\hat{t}_j|,$$ excluding the intercept. Shuffle `y_inference` using `N_PERMUTATIONS` permutations. For each shuffled response vector, use the same maps to recompute coefficient draws and the same maximum statistic. Report $$p=\frac{1+\#\{\hat{t}_{\max,\mathrm{null}}\ge \hat{t}_{\max,\mathrm{obs}}\}}{1+N_{\mathrm{PERMUTATIONS}}}.$$

Artifacts: `null_statistics`, an array of permutation statistics; `global_pvalue`; a null-distribution plot with the observed statistic marked; and an explanation of the null, p-value, and minimum possible p-value, $1/(1+N_{\mathrm{PERMUTATIONS}})$.

Constraints: State the exchangeability assumption for response permutation. If a coefficient's bootstrap standard deviation is zero or below `1e-12`, identify that feature and stop to investigate before calculating a ratio. Use the same maps and sample count for observed and null calculations. Their preprocessing is learned from predictors only, so the maps apply to every shuffled response without refitting tens of thousands of models. This global test does not test each coefficient conditional on the others.

In [35]:
```
# Supplied number of response permutations for the global-null test.
N_PERMUTATIONS = 199
```

In [36]:
```
# Q3.6: global permutation test using the Q3.3 bootstrap maps.
def max_bootstrap_ratio(response):
    """Largest |mean/SD| of the bootstrap coefficient draws (intercept already excl
    draws = coefficient_operators @ response
```

```
    sd = draws.std(axis=0, ddof=1)
    if np.any(sd < 1e-12):
        bad = [frozen_terms[k] for k in np.flatnonzero(sd < 1e-12)]
        raise ValueError(f"Zero bootstrap SD for {bad}; investigate before computin
    return float(np.max(np.abs(draws.mean(axis=0) / sd)))


observed_draws = coefficient_operators @ y_inference
bootstrap_ratio = observed_draws.mean(axis=0) / observed_draws.std(axis=0, ddof=1)
t_max_observed = max_bootstrap_ratio(y_inference)
print(f"Observed max |t| = {t_max_observed:.2f}, from {frozen_terms[int(np.argmax(n
display(pd.DataFrame({"term": frozen_terms, "bootstrap_ratio": bootstrap_ratio}).se
        .sort_values("bootstrap_ratio", key=np.abs, ascending=False).round(2).head(

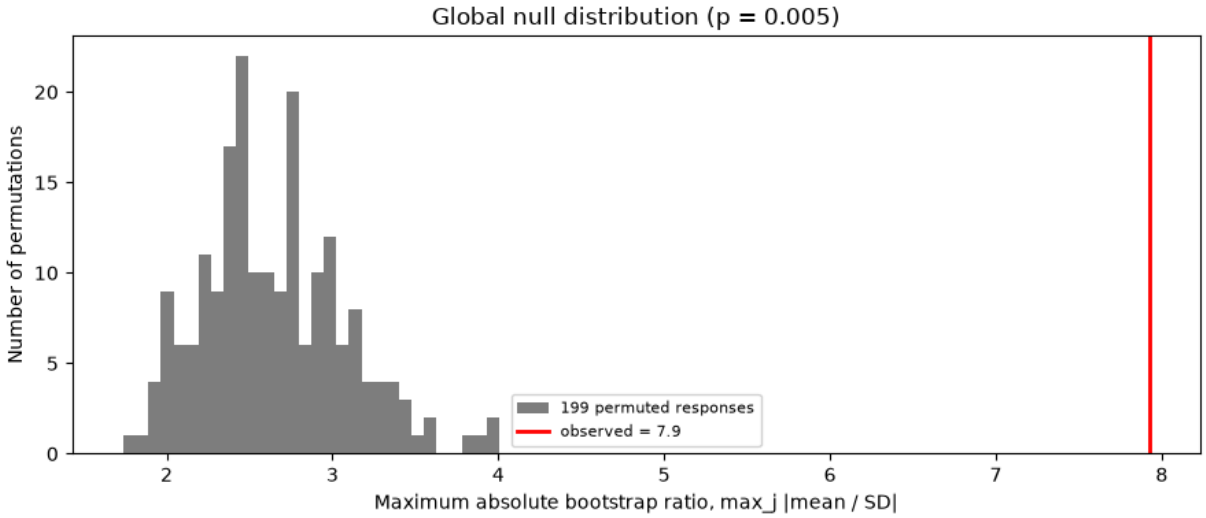
rng_perm = np.random.default_rng(6109)
null_statistics = np.array(
    [max_bootstrap_ratio(rng_perm.permutation(y_inference)) for _ in range(N_PERMUT
)
global_pvalue = (1 + np.sum(null_statistics >= t_max_observed)) / (1 + N_PERMUTATIO
print(f"Null max |t|: median {np.median(null_statistics):.2f}, 95th percentile {np.
      f"max {null_statistics.max():.2f}")
print(f"Permutation p-value = {global_pvalue:.4f} (smallest possible = {1 / (1 + N_

fig, ax = plt.subplots(figsize=(9, 4))
ax.hist(null_statistics, bins=30, color="tab:gray", label=f"{N_PERMUTATIONS} permut
ax.axvline(t_max_observed, color="red", lw=2, label=f"observed = {t_max_observed:.1
ax.set_xlabel("Maximum absolute bootstrap ratio, max_j |mean / SD|")
ax.set_ylabel("Number of permutations")
ax.set_title(f"Global null distribution (p = {global_pvalue:.3f})")
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()
```

```
Observed max |t| = 7.92, from Gr_Liv_Area
```

term	Gr_Liv_Area	Year_Remod_Add	Fireplaces	Garage_Cars	Neighborhood_Crawf
bootstrap_ratio	7.92	5.36	4.96	4.86	4

```
Null max |t|: median 2.60, 95th percentile 3.40, max 4.01
Permutation p-value = 0.0050 (smallest possible = 0.0050)
```



In [37]:
```
grader.check("q3_6")
```

Out[37]:

q3_6 passed! ✨

Written response — Q3.6

The global null is that log price has no relationship with the frozen predictors: given the predictor values, every assignment of the observed prices to houses is equally likely, so permuting `y_inference` yields data as plausible as the observed. This requires exchangeable houses (independent and identically distributed, with no grouping such as sale year or block that makes some permutations unrealistic).

The observed statistic is 7.92 (the largest absolute bootstrap ratio, `Gr_Liv_Area`). The 199 permuted statistics have median 2.60, 95th percentile 3.40 and maximum 4.01, so none comes close, and p = (1 + 0)/(1 + 199) = **0.005**, the smallest possible with 199 permutations. We reject the global null: at least one frozen feature is associated with price in the independent sample ($p \leq 0.005$; more permutations would be needed to measure the probably far smaller true tail probability). The test does not say which features and is not a conditional test of any single coefficient. Taking the maximum over all 68 ratios makes it one global question, but a large individual ratio still has no individual p-value here.

Q3.7. Synthesize the model evidence

3 points

We now have several kinds of evidence about the model, and they need not give identical impressions of every predictor. A feature might be selected often but have an uncertain coefficient, or have a clear association while adding little predictive information beyond other inputs. Combining these results helps us distinguish conclusions supported by the analyses from conclusions that remain uncertain.

Task: Choose three features from `frozen_indices` and compare the evidence about their selection stability, coefficient uncertainty, and predictive importance. Explain what the overall permutation result adds to the model interpretation and what it does not establish about those individual features.

Artifacts: Compact synthesis table and qualified interpretation.

Constraints: Do not assign the global p-value to individual features or make causal claims. If the analyses suggest different conclusions, explain why their questions differ; do not assume their rankings must agree.

For each feature, report its Q3.1 selection frequency, Q3.3 coefficient and interval, and Q3.4 original-predictor importance. For an indicator, use the importance of its whole original categorical predictor and label that distinction; Q3.4 does not give a separate importance score for each encoded level.

In [38]:
```
# Q3.7: three frozen features compared across selection, coefficient and importance
synthesis_terms = ["Gr_Liv_Area", "Overall_Qual_Excellent", "Lot_Frontage"]
synthesis_rows = []
for term in synthesis_terms:
    j = feature_names.index(term)
    k = frozen_terms.index(term)
    source = feature_sources[j]
    cs_row = coefficient_summary.iloc[k]
    synthesis_rows.append({
        "feature": term,
        "type": "numeric (per SD)" if j < n_numeric else f"indicator of {source}",
        "Q3.1 selection frequency": selection_frequencies[j],
        "Q3.3 coefficient": cs_row["coefficient"],
        "Q3.3 95% interval": f"[{cs_row['lower']:.3f}, {cs_row['upper']:.3f}]",
        "Q3.4 importance of": source + (" (whole categorical predictor)" if j >= n_
        "Q3.4 mean MSE increase": importance_summary.loc[source, "mean"],
        "Q3.4 importance rank": int(importance_summary.index.get_loc(source)) + 1,
    })
synthesis_table = pd.DataFrame(synthesis_rows).set_index("feature")
print(f"Global permutation p-value (Q3.6) for the whole frozen set: {global_pvalue:
      f"importance ranks out of {len(importance_summary)} original predictors.")
display(synthesis_table.round(4))
```

```
Global permutation p-value (Q3.6) for the whole frozen set: 0.005; importance ranks
out of 46 original predictors.
```

feature	type	Q3.1 selection frequency	Q3.3 coefficient	Q3.3 95% interval	Q3.4 importance of	Q3.4 mean MSE increase	i
Gr_Liv_Area	numeric (per SD)	1.00	0.1399	[0.108, 0.179]	Gr_Liv_Area	0.0335	
Overall_Qual_Excellent	indicator of Overall_Qual	1.00	0.2575	[0.134, 0.375]	Overall_Qual (whole categorical predictor)	0.0150	
Lot_Frontage	numeric (per SD)	0.91	-0.0025	[-0.016, 0.014]	Lot_Frontage	0.0000	

Written response — Q3.7

feature	Q3.1 selection frequency	Q3.3 coefficient [95% interval]	Q3.4 importance (MSE increase, rank of 46)
`Gr_Liv_Area` (per SD)	1.00	+0.140 [0.108, 0.179]	0.0335, rank 1
`Overall_Qual_Excellent` (0/1)	1.00	+0.257 [0.134, 0.375]	0.0150, rank 2, for **all of** `Overall_Qual`
`Lot_Frontage` (per SD)	0.91	−0.003 [−0.016, 0.014]	0.00004, rank 29

`Gr_Liv_Area` : all three agree. Selected in every resample, precisely estimated away from zero in fresh data, and the most damaging input to shuffle.

`Overall_Qual_Excellent` : stable and clearly positive, but imprecise (only 38 inference houses carry it). Its rank-2 importance belongs to the whole `Overall_Qual` predictor (Q3.4 shuffles all levels together), so it shows reliance on overall quality, not on the Excellent level.

`Lot_Frontage` : frequent selection is not association or predictive value. Selected in 91% of resamples, yet its inference coefficient is indistinguishable from zero and shuffling it leaves held-out error unchanged. The analyses ask different questions (reproducibility of selection, size of a conditional association, reliance of this fit), so their rankings need not agree.

Global test. The p-value of 0.005 (the minimum for 199 permutations) is strong evidence that the frozen set as a whole is associated with log price in data that played no part in selection. It does not show that any individual feature, including `Lot_Frontage`, is associated with price, and nothing here is causal.

AC209A Extension 1. Individual classical coefficient tests (AC209A only)

6 points

The global permutation result concerns the predictor set as a whole. Extend the interpretation to individual coefficients conditional on the other included features.

Task: Use statsmodels on the independent inference fit to obtain standard errors, t-statistics, p-values and intervals. Use $t_j=\hat\beta_j/\mathrm{SE}(\hat\beta_j)$, with $\nu=n-\mathrm{rank}(X)$ degrees of freedom and two-sided $p_j=2[1-F_{t_\nu}(|t_j|)]$. Interpret and compare with the global result. In this question's own answer cell, write an addendum explaining how these tests qualify or reinforce your Q3.7 synthesis.

Artifacts: Individual-test table, aligned classical/bootstrap intervals, and synthesis addendum.

Constraints: Explain classical model assumptions and individual rather than simultaneous testing. No mathematical derivation or custom t-distribution implementation required. Independent-fit inference does not transfer automatically to final refit coefficients.

Before comparing with Q3.3, convert numeric coefficient estimates, standard errors, and interval bounds to the same `reference_sd` units. The positive scaling factor does not change t-statistics or p-values. Indicators remain in 0/1 units. CS1090A students skip this extension.

In [39]:
```
# AC209A Extension 1: classical t-tests for each coefficient in the independent inf
import statsmodels.api as sm

ols_sm = sm.OLS(y_inference, inference_fit["design"]).fit()
rank_design = np.linalg.matrix_rank(inference_fit["design"])
print(f"n = {len(y_inference)}, rank(X) = {rank_design}, residual df = {ols_sm.df_r
      f"(= n - rank: {len(y_inference) - rank_design})")
ci = ols_sm.conf_int(alpha=0.05)[1:]  # drop the intercept
classical_tests = pd.DataFrame(
    {
        "term": frozen_terms,
        "coefficient": ols_sm.params[1:] * reference_multiplier,
        "se": ols_sm.bse[1:] * reference_multiplier,
        "t": ols_sm.tvalues[1:],
        "p_value": ols_sm.pvalues[1:],
        "classical_lower": ci[:, 0] * reference_multiplier,
        "classical_upper": ci[:, 1] * reference_multiplier,
    }
)
# Align with the Q3.3 bootstrap results (same reference_sd units).
classical_tests["bootstrap_sd"] = coefficient_summary["sd"].to_numpy()
classical_tests["bootstrap_lower"] = coefficient_summary["lower"].to_numpy()
classical_tests["bootstrap_upper"] = coefficient_summary["upper"].to_numpy()
```

```
classical_tests["se_ratio (bootstrap/classical)"] = classical_tests["bootstrap_sd"]
print("Point estimates match Q3.3:", bool(np.allclose(classical_tests["coefficient"
with pd.option_context("display.max_rows", 200, "display.width", 250):
    display(classical_tests.round(4))
print(f"p < 0.05 for {int((classical_tests.p_value < 0.05).sum())} of {len(classica
      f"Bonferroni threshold {0.05 / len(classical_tests):.2g} passed by {int((clas
both_exclude = ((classical_tests.classical_lower > 0) | (classical_tests.classical_
print(f"Classical and bootstrap 95% intervals agree on excluding zero for {int(both
print("Features where they disagree:")
display(classical_tests.loc[~both_exclude.to_numpy(), ["term", "coefficient", "p_va
                                                       "bootstrap_lower", "bootstr
print("Largest bootstrap/classical SE ratios:")
display(classical_tests.nlargest(5, "se_ratio (bootstrap/classical)")[["term", "se"

order = np.argsort(classical_tests["coefficient"].to_numpy())
ypos = np.arange(len(order))
ct = classical_tests.iloc[order]
fig, ax = plt.subplots(figsize=(9, 0.32 * len(order) + 1.5))
ax.hlines(ypos + 0.15, ct.classical_lower, ct.classical_upper, color="tab:orange",
ax.hlines(ypos - 0.15, ct.bootstrap_lower, ct.bootstrap_upper, color="tab:blue", lw
ax.plot(ct.coefficient, ypos, "o", color="black", ms=4, label="independent-fit esti
ax.axvline(0, color="red", ls="--", label="zero")
ax.set_yticks(ypos, ct.term, fontsize=8)
ax.set_xlabel("Effect on log sale price (per selection SD for numeric; per 0/1 for
ax.set_title("Classical versus bootstrap 95% intervals, independent inference fit")
ax.legend(fontsize=8, loc="lower right")
plt.tight_layout()
plt.show()
```

```
n = 879, rank(X) = 69, residual df = 810 (= n - rank: 810)
Point estimates match Q3.3: True
```

	term	coefficient	se	t	p_value	classica
0	Bldg_Type_Twnhs	-0.0540	0.0348	-1.5501	0.1215	
1	Bsmt_Full_Bath	0.0346	0.0073	4.7592	0.0000	
2	Central_Air_Y	0.1019	0.0241	4.2256	0.0000	
3	Condition_1_Norm	0.0377	0.0150	2.5050	0.0124	
4	Enclosed_Porch	0.0018	0.0063	0.2798	0.7797	
5	Fireplaces	0.0353	0.0066	5.3774	0.0000	
6	Full_Bath	0.0152	0.0083	1.8269	0.0681	
7	Garage_Cars	0.0378	0.0073	5.1571	0.0000	
8	Gr_Liv_Area	0.1399	0.0136	10.3029	0.0000	
9	Heating_QC_Typical	-0.0025	0.0127	-0.1950	0.8454	
10	MS_SubClass_Two_Story_1945_and_Older	-0.0137	0.0292	-0.4675	0.6402	
11	Neighborhood_Crawford	0.1419	0.0308	4.6045	0.0000	
12	Neighborhood_Edwards	-0.0363	0.0229	-1.5820	0.1140	
13	Neighborhood_Northridge_Heights	0.0405	0.0267	1.5166	0.1298	
14	Overall_Cond_Fair	-0.1958	0.0392	-4.9930	0.0000	
15	Overall_Cond_Good	0.0223	0.0172	1.2981	0.1946	
16	Overall_Qual_Below_Average	-0.1424	0.0239	-5.9586	0.0000	
17	Overall_Qual_Excellent	0.2575	0.0359	7.1781	0.0000	
18	Overall_Qual_Good	0.0409	0.0172	2.3794	0.0176	
19	Overall_Qual_Very_Good	0.1516	0.0231	6.5762	0.0000	
20	Sale_Condition_Normal	0.0452	0.0136	3.3197	0.0009	
21	Screen_Porch	0.0106	0.0054	1.9494	0.0516	
22	Total_Bsmt_SF	0.0478	0.0101	4.7462	0.0000	
23	Wood_Deck_SF	-0.0022	0.0049	-0.4379	0.6616	
24	Year_Built	0.0636	0.0121	5.2463	0.0000	
25	Year_Remod_Add	0.0464	0.0076	6.1151	0.0000	
26	Year_Sold	-0.0067	0.0050	-1.3307	0.1837	
27	BsmtFin_Type_1_GLQ	0.0052	0.0166	0.3109	0.7559	
28	Bsmt_Exposure_Gd	0.0322	0.0206	1.5615	0.1188	
29	BsmtFin_Type_1_Unf	0.0049	0.0182	0.2702	0.7871	

	term	coefficient	se	t	p_value	classica
30	Functional_Typ	0.1507	0.0216	6.9653	0.0000	
31	Kitchen_AbvGr	-0.0113	0.0078	-1.4517	0.1470	
32	Kitchen_Qual_Typical	-0.0031	0.0236	-0.1325	0.8946	
33	Latitude	0.0170	0.0062	2.7500	0.0061	
34	Open_Porch_SF	0.0045	0.0048	0.9282	0.3536	
35	Bldg_Type_OneFam	0.0552	0.0219	2.5210	0.0119	
36	Land_Contour_HLS	-0.0060	0.0264	-0.2267	0.8207	
37	Longitude	-0.0094	0.0070	-1.3416	0.1801	
38	Bsmt_Unf_SF	-0.0086	0.0094	-0.9148	0.3605	
39	Exterior_1st_BrkFace	0.0793	0.0295	2.6833	0.0074	
40	Exter_Qual_Typical	0.0048	0.0171	0.2827	0.7775	
41	MS_SubClass_One_Story_1945_and_Older	-0.1059	0.0264	-4.0078	0.0001	
42	Overall_Cond_Average	-0.0399	0.0148	-2.7031	0.0070	
43	Overall_Qual_Average	-0.0106	0.0158	-0.6692	0.5035	
44	TotRms_AbvGrd	-0.0099	0.0100	-0.9933	0.3209	
45	Lot_Frontage	-0.0025	0.0076	-0.3333	0.7390	
46	Half_Bath	-0.0025	0.0072	-0.3405	0.7336	
47	Neighborhood_Stone_Brook	0.1131	0.0433	2.6113	0.0092	
48	Overall_Cond_Below_Average	-0.0763	0.0276	-2.7662	0.0058	
49	Exterior_2nd_Wd Sdng	0.0092	0.0162	0.5690	0.5695	
50	Low_Qual_Fin_SF	-0.0076	0.0046	-1.6339	0.1027	
51	Mas_Vnr_Area	0.0031	0.0061	0.5011	0.6164	
52	Bsmt_Half_Bath	0.0011	0.0057	0.2002	0.8414	
53	Neighborhood_Sawyer_West	-0.0356	0.0270	-1.3198	0.1873	
54	Lot_Area	0.0136	0.0069	1.9642	0.0499	
55	Overall_Qual_Fair	-0.2144	0.0485	-4.4188	0.0000	
56	Three_season_porch	0.0074	0.0033	2.2232	0.0265	
57	Bsmt_Qual_Fair	0.0764	0.0331	2.3060	0.0214	
58	Garage_Finish_Unf	-0.0049	0.0127	-0.3902	0.6965	
59	Kitchen_Qual_Good	0.0141	0.0222	0.6346	0.5259	

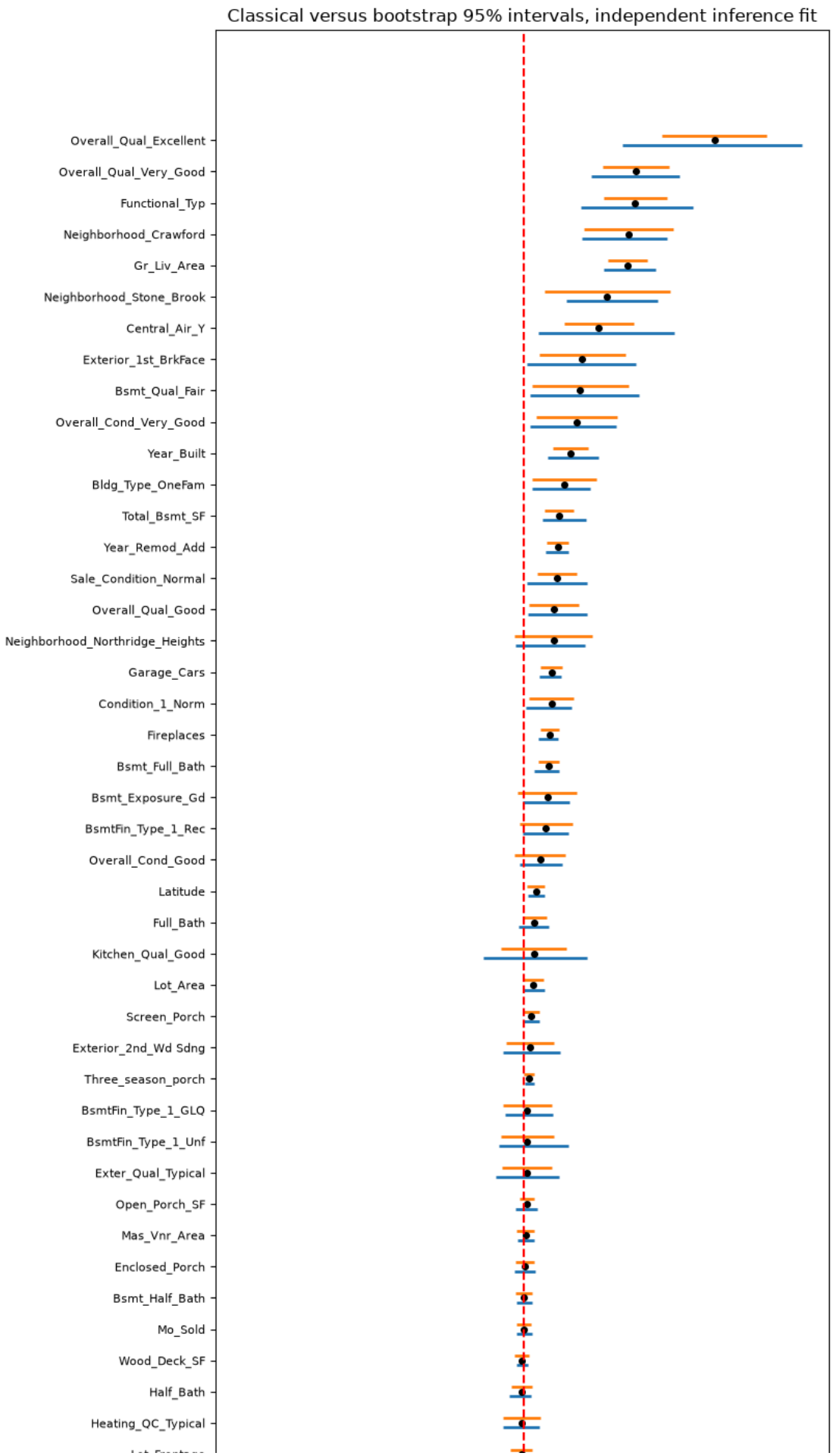
	term	coefficient	se	t	p_value	classica
60	Neighborhood_Meadow_Village	-0.0347	0.0533	-0.6510	0.5152	
61	BsmtFin_Type_1_Rec	0.0304	0.0184	1.6541	0.0985	
62	MS_SubClass_Two_Story_PUD_1946_and_Newer	-0.0259	0.0343	-0.7541	0.4510	
63	Misc_Val	-0.0130	0.0019	-6.7338	0.0000	
64	Bsmt_Exposure_No	-0.0215	0.0130	-1.6553	0.0983	
65	Mo_Sold	0.0002	0.0051	0.0297	0.9763	
66	Bsmt_Qual_Good	-0.0058	0.0144	-0.4011	0.6885	
67	Overall_Cond_Very_Good	0.0716	0.0277	2.5796	0.0101	

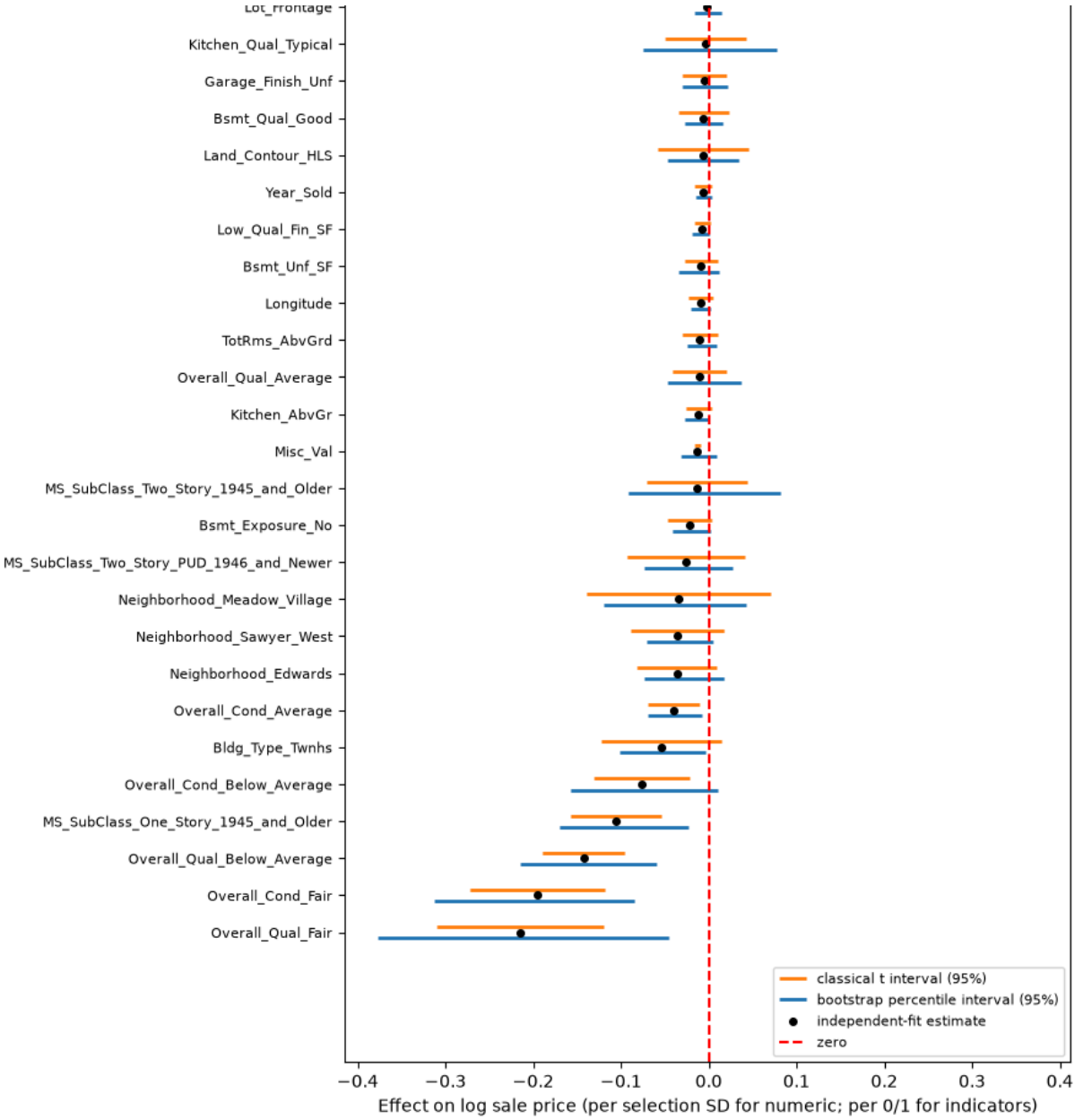
```
p < 0.05 for 30 of 68 coefficients; Bonferroni threshold 0.00074 passed by 17.
Classical and bootstrap 95% intervals agree on excluding zero for 63 of 68 features.
Features where they disagree:
```

	term	coefficient	p_value	classical_lower	classical_upper	bootstra
0	Bldg_Type_Twnhs	-0.0540	0.1215	-0.1223	0.0144	
21	Screen_Porch	0.0106	0.0516	-0.0001	0.0213	
48	Overall_Cond_Below_Average	-0.0763	0.0058	-0.1304	-0.0221	
54	Lot_Area	0.0136	0.0499	0.0000	0.0272	
63	Misc_Val	-0.0130	0.0000	-0.0168	-0.0092	

```
Largest bootstrap/classical SE ratios:
```

	term	se	bootstrap_sd	se_ratio (bootstrap/classical)
63	Misc_Val	0.0019	0.0124	6.4211
2	Central_Air_Y	0.0241	0.0477	1.9783
30	Functional_Typ	0.0216	0.0403	1.8637
32	Kitchen_Qual_Typical	0.0236	0.0438	1.8586
59	Kitchen_Qual_Good	0.0222	0.0399	1.7963


Classical versus bootstrap 95% intervals, independent inference fit
Overall_Qual_Excellent
Overall_Qual_Very_Good
Functional_Typ
Neighborhood_Crawford
Gr_Liv_Area
Neighborhood_Stone_Brook
Central_Air_Y
Exterior_1st_BrkFace
Bsmt_Qual_Fair
Overall_Cond_Very_Good
Year_Built
Bldg_Type_OneFam
Total_Bsmt_SF
Year_Remod_Add
Sale_Condition_Normal
Overall_Qual_Good
Neighborhood_Northridge_Heights
Garage_Cars
Condition_1_Norm
Fireplaces
Bsmt_Full_Bath
Bsmt_Exposure_Gd
BsmtFin_Type_1_Rec
Overall_Cond_Good
Latitude
Full_Bath
Kitchen_Qual_Good
Lot_Area
Screen_Porch
Exterior_2nd_Wd Sdng
Three_season_porch
BsmtFin_Type_1_GLQ
BsmtFin_Type_1_Unf
Exter_Qual_Typical
Open_Porch_SF
Mas_Vnr_Area
Enclosed_Porch
Bsmt_Half_Bath
Mo_Sold
Wood_Deck_SF
Half_Bath
Heating_QC_Typical



Written response — AC209A Extension 1

Refitting the independent inference model in statsmodels (n = 879, rank 69, ν = 810), with coefficients, SEs and intervals converted to Q3.3's `reference_sd` units (the positive factor leaves t and p unchanged), **30 of 68 coefficients have p < 0.05** and 17 pass the Bonferroni threshold 0.05/68 ≈ 0.00074. Classical and bootstrap intervals agree on excluding zero for 63 of 68; three disagreements are near the boundary (`Screen_Porch` p = 0.052 and `Lot_Area` p = 0.0499 classically; `Bldg_Type_Twnhs` bootstrap upper bound −0.004) and two are substantive:

- `Misc_Val` : classical p < 0.0001, [−0.017, −0.009], but bootstrap [−0.032, 0.010]; its bootstrap SD is **6.4 times** the classical SE. It is zero for almost every house and large for a few, so the estimate rests on a few high-leverage sales that the classical SE (fixed predictors, constant variance) misses.

- `Overall_Cond_Below_Average`: classical p = 0.006, bootstrap [−0.158, 0.011], SD 1.5 times larger.

Assumptions and scope. Classical tests assume a correct linear mean on the log scale, independent constant-variance errors and normality (or large n), with the design chosen without these outcomes (true here). Each p-value tests one coefficient conditional on the other 67 at a 5% rate for that test alone, so about 3 false rejections are expected among 68 tests even if every coefficient were zero; Q3.6's single global test answers a different question. Results describe the independent fit, not the Q3.8 refit.

Addendum to Q3.7. The tests reinforce `Gr_Liv_Area` (t = 10.3, p < 0.0001) and `Overall_Qual_Excellent` (t = 7.2, p < 0.0001; classical [0.187, 0.328] against bootstrap [0.134, 0.375], plausibly narrower because the bootstrap better reflects the 38 houses carrying the level). For `Lot_Frontage` (t = −0.33, p = 0.74) they confirm that frequent selection gave no detectable conditional association. Q3.6 showed only that the set matters; these tests show which coefficients carry individual evidence, while `Misc_Val` warns that a decisive p-value can rest on a few unusual houses.

Q3.8. Final predictive evaluation

4 points

The development analyses have informed our modeling choices without using the reserved test houses. Once those choices are fixed, we can refit the predictive procedures on all non-test houses and assess their performance on new observations. This refit uses more data for prediction, but its coefficients do not inherit the intervals or p-values from the independent inference fit.

Task: Freeze choices, concatenate selection rows followed by inference rows as all training data, refit preprocessing and the model, and evaluate the tuned lasso and fixed-subset OLS models on test. For final lasso, apply the specified CV tuning rule on all training data; for fixed-subset OLS, retain the frozen features with no penalty. Quantify appropriate performance-score uncertainty.

Artifacts: Final predictive comparison reporting test RMSE in log-price units; a scoped uncertainty summary; and one sentence explaining which earlier inference results apply to the refitted model.

Constraints: Freeze the model choices before examining test performance. Do not retune or select models using test results. Earlier coefficient intervals and p-values describe the independent inference fit. State whether your performance uncertainty reflects the test sample, repeated model development, or both.

Run this question only after finishing the earlier Ames questions and freezing choices. For the final lasso fit, use `all_train_folds`, `LASSO_ALPHAS`, and the Q2.2 minimum pooled-

error rule on all training rows. Test outcomes must not enter any fit, tuning choice, or diagnostic until this evaluation.

Build all-training IDs with `np.concatenate([selection_ids, inference_ids])` and test IDs with `partitions["test"]`. Take each group's predictors from `ames_x.loc[ids]` and its response from `np.log(raw_ames.loc[ids, "Sale_Price"])`. Preserve this row order when using the supplied folds.

In [40]:
```
# Fixed final-fit folds assume selection houses followed by inference houses.
all_train_folds = fixed_folds(len(selection_ids) + len(inference_ids), seed=109)
```

In [41]:
```
# Q3.8: refit the frozen procedures on all non-test houses and score them once on t
train_ids = np.concatenate([selection_ids, inference_ids])
test_ids = partitions["test"]
X_train_all = ames_x.loc[train_ids]
y_train_all = np.log(raw_ames.loc[train_ids, "Sale_Price"].to_numpy())
X_test = ames_x.loc[test_ids]
y_test = np.log(raw_ames.loc[test_ids, "Sale_Price"].to_numpy())

# Tuned lasso: same pooled minimum-MSE rule on all training rows.
final_lasso_tuning = tune_penalty("lasso", LASSO_ALPHAS, X_train_all, y_train_all,
final_prep = clone(shared_preprocessor).fit(X_train_all)
final_lasso = make_penalized("lasso", final_lasso_tuning["alpha"]).fit(final_prep.t
lasso_test_pred = final_lasso.predict(final_prep.transform(X_test))
# Fixed-subset OLS: the frozen features, no penalty.
final_ols = checked_ols(shared_preprocessor, X_train_all, y_train_all, frozen_indic
ols_test_pred = final_ols["prediction"]

final_comparison = pd.DataFrame(
    {
        "alpha": [final_lasso_tuning["alpha"], np.nan],
        "features used": [int((np.abs(final_lasso.coef_) > NONZERO_TOL).sum()), len
        "training CV RMSE": [final_lasso_tuning["cv_rmse"], np.nan],
        "test RMSE": [rmse(y_test, lasso_test_pred), rmse(y_test, ols_test_pred)],
    },
    index=["tuned lasso", "fixed-subset OLS"],
)
display(final_comparison.round(4))

# Uncertainty from the finite test sample: paired bootstrap over test houses (model
rng_test = np.random.default_rng(8110)
test_boot = []
for _ in range(5000):
    i = rng_test.integers(0, len(y_test), size=len(y_test))
    r_lasso, r_ols = rmse(y_test[i], lasso_test_pred[i]), rmse(y_test[i], ols_test_
    test_boot.append((r_lasso, r_ols, r_ols - r_lasso))
test_boot = np.array(test_boot)
test_uncertainty = pd.DataFrame(
    {"estimate": [final_comparison.loc["tuned lasso", "test RMSE"], final_compariso
                  final_comparison.loc["fixed-subset OLS", "test RMSE"] - final_com
     "2.5%": np.percentile(test_boot, 2.5, axis=0), "97.5%": np.percentile(test_boo
    index=["tuned lasso RMSE", "fixed-subset OLS RMSE", "difference (OLS - lasso)"]
)
```

```
display(test_uncertainty.round(4))
print(f"Share of test resamples where lasso has the lower RMSE: {np.mean(test_boot[
```

	alpha	features used	training CV RMSE	test RMSE
tuned lasso	0.0005	130	0.1495	0.1219
fixed-subset OLS	NaN	68	NaN	0.1315

	estimate	2.5%	97.5%
tuned lasso RMSE	0.1219	0.1093	0.1350
fixed-subset OLS RMSE	0.1315	0.1197	0.1434
difference (OLS - lasso)	0.0096	0.0040	0.0152

```
Share of test resamples where lasso has the lower RMSE: 0.999
```

Written response — Q3.8

model (refitted on 2,197 non-test houses)	penalty	features	test RMSE (log price)	95% bootstrap interval
tuned lasso	5.4×10^{-4}	130	**0.122**	[0.109, 0.135]
fixed-subset OLS (68 frozen features)	—	68	0.131	[0.120, 0.143]
difference (OLS − lasso)			0.0096	[0.0040, 0.0152]

Choices were frozen before the test houses were used: the pooled-MSE rule, grid, folds and 68-feature subset. On all training rows the rule chose a smaller penalty than on the selection rows alone (5.4×10^{-4} against 1.19×10^{-3}), keeping 130 features (training-CV RMSE 0.149). On the 733 test houses the tuned lasso is more accurate (typical error about 13% in price); the paired bootstrap gives the RMSE difference a 95% interval of 0.004 to 0.015, with the lasso ahead in 99.9% of resamples.

Scope. The intervals resample the 733 test houses with both models fixed, so they reflect only the finite test sample. They exclude the variability of repeating model development (retuning, reselecting, refitting), which Q3.1–Q3.2 showed is substantial, and so understate the uncertainty of each *procedure*. The Q3.3 and Extension 1 intervals and p-values describe only the independent inference fit, not the refitted models, whose training data include the selection houses that chose the features.

Part 4. Principal component analysis

PCA for visualization

The Ames analyses used predictors with familiar meanings. Before modeling high-dimensional spectra, we will use PCA on familiar animal traits so that you can interpret directions and loading arrows using knowledge you already have. The supplied UCI Zoo dataset describes 101 animals with 15 yes/no traits and a leg count. Names and seven animal classes are metadata used for annotation, not PCA inputs.

In [42]:
```
# Supplied taxonomy labels; neither names nor classes enter PCA.
classes = {
    1: "Mammals",
    2: "Birds",
    3: "Reptiles",
    4: "Fish",
    5: "Amphibians",
    6: "Insects",
    7: "Other invertebrates",
}
```

In [43]:
```
# Supplied animal data; names and classes are metadata.
zoo = pd.read_csv("data/zoo.csv")
traits = [c for c in zoo if c not in ["animal_name", "class_type"]]
```

Q4.1. Construct and interpret an animal biplot

5 points

The Ames analysis used predictors to model a response. PCA instead summarizes variation among the predictors themselves, which can help us visualize a dataset with many measurements. Animal traits provide a familiar setting for connecting the positions of observations and the directions of the loading vectors to concrete differences in the recorded features.

Task: Use the Zoo trait predictors to fit PCA and plot the first two components. Color points by the supplied animal classes, place animal names near representative points, overlay labeled trait-loading arrows on the same scatter plot, and include each component's explained-variance percentage in its axis label. Use the arrows to interpret two directions of variation and explain two selected animal comparisons.

Artifacts: One readable annotated biplot and focused interpretation.

Constraints: Names/classes are metadata, excluded from fitting PCA. Document preprocessing and arrow scaling. Handle identical-profile overlap with readable representative labels. No clustering or required scree plot.

Output names for the guided checks: `scores_sd`, the two standardized score columns, and `traits`, the predictor-column list.

`zoo` and `traits` are supplied above. Standardize the 16 trait columns before PCA. For the checked `scores_sd` output, divide each of the first two score columns by its standard deviation (`ddof=0`). Label representative animals where profiles overlap, and explain your labeling choice. Choose two pairs of animals for interpretation; no domain knowledge beyond the named traits is required.

You may multiply all arrow coordinates by one common positive factor to make the overlay readable. Report the factor in the title or caption, and choose axis limits that include every displayed arrow endpoint and its label. Do not scale individual arrows separately: that would change their relative lengths. State whether your arrows show PCA loadings or feature–component correlations; these quantities describe the displayed PCA directions, not importance for predicting a response.

In [44]:
```
# Q4.1: standardized PCA biplot of the 16 animal traits (names/classes are labels o
zoo_scaler = StandardScaler().fit(zoo[traits])
zoo_z = zoo_scaler.transform(zoo[traits])
zoo_pca = PCA(n_components=2).fit(zoo_z)
zoo_scores = zoo_pca.transform(zoo_z)
scores_sd = zoo_scores / zoo_scores.std(axis=0, ddof=0)
evr = zoo_pca.explained_variance_ratio_
loadings = pd.DataFrame(zoo_pca.components_.T, index=traits, columns=["PC1", "PC2"]
print(f"Explained variance: PC1 {evr[0]:.1%}, PC2 {evr[1]:.1%}, together {evr.sum()
display(loadings.round(3).sort_values("PC1"))

# Animals with identical trait profiles land on the same point; label one represent
profile_groups = zoo.groupby(traits, sort=False).indices
representatives = {}
for rows in profile_groups.values():
    names = zoo.animal_name.iloc[rows].tolist()
    representatives[rows[0]] = names[0] + (f" (+{len(names) - 1})" if len(names) >
print(f"{len(zoo)} animals share {len(profile_groups)} distinct trait profiles.")
print("Shared profiles:", [zoo.animal_name.iloc[r].tolist() for r in profile_groups
# Label every shared profile plus a readable set of distinctive single animals.
extra = ["platypus", "penguin", "octopus", "frog", "scorpion", "tortoise", "honeybe
         "kiwi", "seal", "duck", "gorilla", "ostrich", "pitviper", "newt"]
label_rows = [r for r in representatives if "(+" in representatives[r]]
label_rows += [zoo.index[zoo.animal_name == n][0] for n in extra if n in set(zoo.an

ARROW_SCALE = 5.0
fig, ax = plt.subplots(figsize=(13, 10))
for code, name in classes.items():
    m = zoo["class_type"] == code
    ax.scatter(scores_sd[m, 0], scores_sd[m, 1], s=60, alpha=0.75, label=name)
# Merge labels of profiles that land almost on the same point so the text stays rea
placed = []  # [x, y, text]
for r in label_rows:
    x, y = scores_sd[r]
    near = [p for p in placed if abs(p[0] - x) < 0.15 and abs(p[1] - y) < 0.15]
    if near:
        near[0][2] += " / " + representatives[r]
    else:
        placed.append([x, y, representatives[r]])
```

```
for x, y, text in placed:
    ax.annotate(text, (x, y), xytext=(4, 3), textcoords="offset points", fontsize=8
for trait, (l1, l2) in loadings.iterrows():
    ax.arrow(0, 0, ARROW_SCALE * l1, ARROW_SCALE * l2, color="black", alpha=0.6, wi
    ax.text(ARROW_SCALE * l1 * 1.1, ARROW_SCALE * l2 * 1.1, trait, color="darkred",
ends = np.vstack([scores_sd, ARROW_SCALE * 1.2 * loadings.to_numpy()])
ax.set_xlim(ends[:, 0].min() - 0.4, ends[:, 0].max() + 1.2)
ax.set_ylim(ends[:, 1].min() - 0.4, ends[:, 1].max() + 0.4)
ax.axhline(0, color="gray", lw=0.5)
ax.axvline(0, color="gray", lw=0.5)
ax.set_xlabel(f"PC1 score, standardized (explains {evr[0]:.1%} of trait variance)")
ax.set_ylabel(f"PC2 score, standardized (explains {evr[1]:.1%} of trait variance)")
ax.set_title(f"Zoo animals on the first two principal components\n"
             f"Arrows: PCA loadings of the standardized traits, all multiplied by {
             "'name (+n)' marks n other animals with the identical profile")
ax.legend(title="Animal class (not used in PCA)", fontsize=8, loc="lower left")
plt.tight_layout()
plt.show()

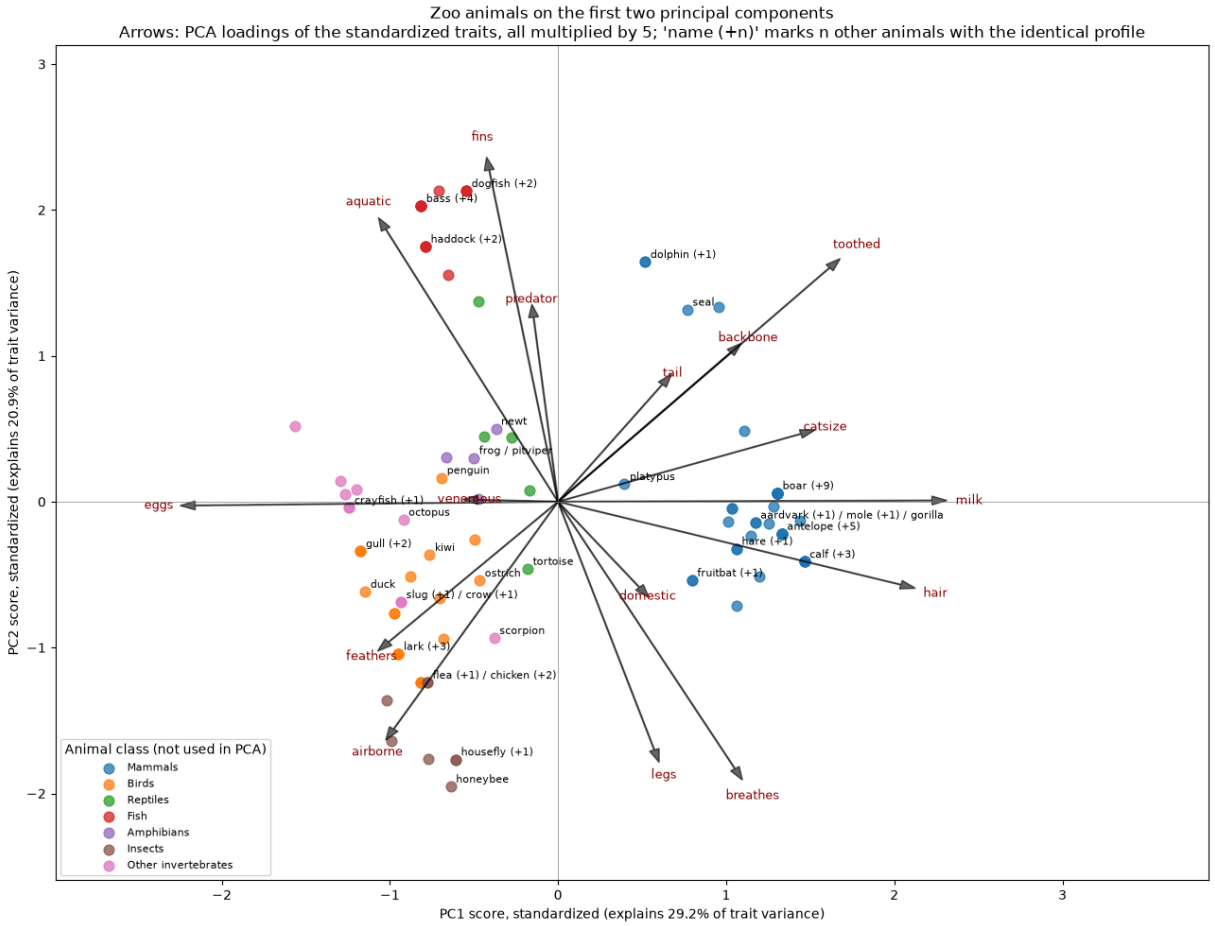
def zoo_profile(*names):
    """Trait values and standardized scores for named animals."""
    rows = [zoo.index[zoo.animal_name == n][0] for n in names]
    out = zoo.loc[rows, ["animal_name"] + traits].set_index("animal_name")
    out["PC1_sd"], out["PC2_sd"] = scores_sd[rows, 0], scores_sd[rows, 1]
    return out

display(zoo_profile("lion", "chicken", "bass", "housefly", "dolphin", "platypus").r
```

```
Explained variance: PC1 29.2%, PC2 20.9%, together 50.1%
```

	PC1	PC2
eggs	-0.432	-0.005
aquatic	-0.205	0.374
feathers	-0.202	-0.193
airborne	-0.195	-0.312
venomous	-0.096	0.003
fins	-0.082	0.455
predator	-0.029	0.253
domestic	0.097	-0.117
legs	0.115	-0.340
tail	0.124	0.161
backbone	0.205	0.205
breathes	0.210	-0.366
catsize	0.289	0.093
toothed	0.323	0.321
hair	0.408	-0.114
milk	0.445	0.002

```
101 animals share 59 distinct trait profiles.
Shared profiles: [['aardvark', 'bear'], ['boar', 'cheetah', 'leopard', 'lion', 'lyn
x', 'mongoose', 'polecat', 'puma', 'raccoon', 'wolf'], ['mole', 'opossum'], ['antelo
pe', 'buffalo', 'deer', 'elephant', 'giraffe', 'oryx'], ['hare', 'vole'], ['calf',
'goat', 'pony', 'reindeer'], ['fruitbat', 'vampire'], ['housefly', 'moth'], ['dolphi
n', 'porpoise'], ['slug', 'worm'], ['flea', 'termite'], ['dogfish', 'pike', 'tuna'],
['bass', 'catfish', 'chub', 'herring', 'piranha'], ['crayfish', 'lobster'], ['haddoc
k', 'seahorse', 'sole'], ['crow', 'hawk'], ['lark', 'pheasant', 'sparrow', 'wren'],
['chicken', 'dove', 'parakeet'], ['gull', 'skimmer', 'skua']]
```



animal_name	hair	feathers	eggs	milk	airborne	aquatic	predator	toothed	backbone	b
lion	1	0	0	1	0	0	1	1	1	
chicken	0	1	1	0	1	0	0	0	1	
bass	0	0	1	0	0	1	1	1	1	
housefly	1	0	1	0	1	0	0	0	0	
dolphin	0	0	0	1	0	1	1	1	1	
platypus	1	0	1	1	0	1	1	0	1	

In [45]:
```
grader.check("q4_1")
```

Out[45]:

q4_1 passed! ✨

Written response — Q4.1

Preprocessing and arrows. We standardized all 16 traits (mean 0, SD 1) so the leg count (0–8) does not dominate the 0/1 traits; names and classes only color and label points. Points are the two PC scores, each divided by its SD. Arrows are the **PCA loadings** of the standardized

traits, all multiplied by one factor of 5 (relative lengths preserved). The 101 animals have 59 distinct trait profiles, and identical profiles plot on one point, so we label one representative per shared profile as "name (+n)" (for example "boar (+9)" for ten predator mammals) and merge labels that almost coincide with "/".

Two directions. PC1 (29.2% of trait variance) runs from egg-laying animals to mammals: `milk` (+0.45), `hair` (+0.41), `toothed` (+0.32) and `catsize` (+0.29) point right, `eggs` (−0.43) left. PC2 (20.9%) runs from finned water animals at the top (`fins` +0.46, `aquatic` +0.37) to breathing, legged, flying animals at the bottom (`breathes` −0.37, `legs` −0.34, `airborne` −0.31).

Two comparisons. *Lion vs chicken:* the lion (with nine other predator mammals) is far right (PC1 ≈ +1.3), the chicken (with dove and parakeet) left (≈ −0.8), matching hair, milk and teeth against feathers and eggs; the chicken is also lower on PC2 (−1.2 against +0.1), mainly because it flies and lacks teeth and predator traits. *Bass vs housefly:* similar PC1 (−0.8, −0.6; both lay eggs and give no milk) but opposite ends of PC2 (+2.0 against −1.8): the bass has fins, lives in water and has teeth; the housefly breathes, has six legs and flies. Loadings describe directions of greatest trait variation, not prediction of a response.

Q4.2. Explain what the projection leaves out

3 points

The animal biplot gives us a two-dimensional view of data described by many traits. A readable plot can nevertheless leave substantial information out, so its apparent similarities and differences need to be interpreted in light of the variance represented by the displayed components.

Task: Explain why PCA axes should report explained variance and whether animals close in the projection must be close in full predictor space.

Artifacts: Focused written explanation.

Constraints: Discuss both visible separation and apparent proximity, and distinguish statements about the plotted coordinates from statements about all 16 traits. No additional scree plot is required.

In [46]:
```
# Q4.2: compare distances in the 2-D projection with distances over all 16 standard
from scipy.spatial.distance import pdist, squareform
full_distance = squareform(pdist(zoo_z))
plot_distance = squareform(pdist(zoo_scores))  # unscaled PC scores, same units as
iu = np.triu_indices(len(zoo), k=1)
different = full_distance[iu] > 1e-9
print(f"Variance shown by PC1+PC2: {evr.sum():.1%}; left out: {1 - evr.sum():.1%}")
print(f"Correlation of 2-D and 16-D pairwise distances: {np.corrcoef(plot_distance[

# Pairs that look close in the plot but differ in many traits.
```

```
pairs = pd.DataFrame({"a": zoo.animal_name.to_numpy()[iu[0]], "b": zoo.animal_name.
                      "plot distance": plot_distance[iu], "16-trait distance": full
                      "traits that differ": (zoo[traits].to_numpy()[iu[0]] != zoo[t
close_in_plot = pairs[different & (pairs["plot distance"] < 0.3)].sort_values("16-t
print("Pairs within 0.3 units in the plot, ranked by full-space distance:")
display(close_in_plot.head(8).round(2))
print(f"Median 16-trait distance over all pairs: {np.median(full_distance[iu]):.2f}
```

```
Variance shown by PC1+PC2: 50.1%; left out: 49.9%
Correlation of 2-D and 16-D pairwise distances: 0.791
Pairs within 0.3 units in the plot, ranked by full-space distance:
```

	a	b	plot distance	16-trait distance	traits that differ
1121	chicken	termite	0.08	6.06	6
1057	chicken	flea	0.08	6.06	6
2156	flea	parakeet	0.08	6.06	6
1877	dove	termite	0.08	6.06	6
1813	dove	flea	0.08	6.06	6
4134	parakeet	termite	0.08	6.06	6
2306	frog	penguin	0.26	5.87	6
3077	hawk	slug	0.17	5.38	6

```
Median 16-trait distance over all pairs: 5.85
```

Written response — Q4.2

Axes should report explained variance because it says how much of the data the picture shows: PC1 and PC2 together show 50.1% of the standardized-trait variance, so 49.9%, spread over 14 more directions, is invisible.

Visible separation. In unscaled PC units a projection can only shrink distances, so a large separation in the plot is real in 16-trait space. (The plotted scores are divided by SDs above 1, which shrinks the axes but keeps this true.)

Apparent proximity. Closeness in the plot does not imply closeness in 16-D, because animals can differ along dropped directions. The chicken/dove/parakeet group and the flea/termite group are 0.08 apart in unscaled PC units yet 6.06 apart across all 16 standardized traits (the median pair distance is 5.85), differing in feathers, airborne, backbone, legs, tail and domestic. Hawk and slug are 0.17 apart in the plot but 5.38 apart in 16-D, and over all pairs the two distances correlate only 0.79. Statements about plotted coordinates are safe; statements about all traits need the omitted variance checked.

Principal component regression

PCA for regression: a new dataset

For the final section of the notebook, we switch from animal traits to measurements of cookie dough. The animal example used PCA to visualize relationships among observations. For modeling, PCA offers a related possibility: we can replace many original predictors with a smaller set of principal components and use those components as the inputs to a regression model. This approach is called **principal component regression (PCR)**. We will investigate whether it helps us predict when the original dataset has many more predictors than observations, and how we should decide which components to retain.

The dataset contains near-infrared spectra of cookie-dough samples and their measured sucrose content. Sucrose is sugar; our prediction target is its percentage in each sample. A spectrum consists of measurements taken at different wavelengths of light. Each sample has 700 recorded wavelength measurements, from 1100 through 2498 nm. These measurements are our predictors, and sucrose percentage is the response.

There are many more predictors than samples, making this a useful setting for examining what PCA can contribute to regression. The source provides 40 calibration samples for developing models and 32 separate prediction samples for final evaluation. The supplied loader excludes calibration observation 23, documented as an outlier, leaving 39 development samples. We will use cross-validation within those 39 samples to compare and select models, keeping the prediction batch reserved until Q4.10. See the fds dataset documentation, citing Brown, Fearn, and Vannucci (2001).

The loading cell below creates `cookie_X`, the sample-by-wavelength predictor array; `cookie_y`, the matching sucrose percentages; and `wavelengths`, the wavelength corresponding to each predictor column. It also supplies `cookie_folds` for calibration cross-validation. These are the data used in Q4.3–Q4.9.

Each wavelength is centered using its fitting-sample mean; wavelengths share a measurement scale and are not individually scaled to unit variance. Validation/test observations reuse those fitted means. Names/classes in the preceding animal example were metadata; here sucrose is the response and is excluded from PCA fitting.

In Q4.5–Q4.9, adapt earlier methods where appropriate or justify an alternative. State the question answered, assumptions, evidence, and limitations. Chemical expertise or mechanism explanations are not expected. A valid analysis need not beat the baseline or give a clean original-input interpretation. Null-based significance is optional. Automated checks do not enforce a specific algorithm or selected set.

In [47]:
```
# Supplied loading and fixed calibration folds; no prediction-batch access.
# The loader omits documented calibration observation 23 (one-based).
cookie_X, cookie_y, wavelengths = load_cookie()
cookie_folds = fixed_folds(len(cookie_y), seed=109)
```

Q4.3. Diagnose the original regression and coordinate change

2 points

The animal example showed how PCA can summarize predictors for visualization. With the cookie-dough spectra, we now consider whether those coordinates can also help us build a regression model when there are many more measurements than calibration samples. Comparing original-space OLS with OLS using all available principal components helps distinguish a change of coordinates from a reduction in model complexity.

Task: Examine original-space OLS training/CV behavior and dimensions/rank. Show that two least-squares fits give the same predictions: one on the centered original spectra, and one on all principal components whose singular values exceed the tolerance below. Use matching minimum-norm numerical conventions.

Artifacts: Diagnosis table, prediction-equivalence comparison, and an explanation of what equivalence shows about whether a change to PCA coordinates by itself constrains the fit.

Constraints: Describe the observed training and CV errors without assuming that original-space regression must perform poorly. Use matching centering, rank tolerance, and minimum-norm conventions in the equivalence comparison. With more predictors than fitting observations, individual unregularized coefficients need not be uniquely determined.

Output names for the guided checks: `equivalence`, a list of floats: the largest absolute prediction difference for the full-calibration fit and for each fold's held-out predictions.

Use the supplied `cookie_folds` for calibration cross-validation. Use `np.linalg.lstsq(..., rcond=1e-10)` for the original centered design. For each fitting sample, retain only singular values above `largest_singular_value * 1e-10` when forming its PCA regression. Centering 39 rows leaves at most 38 nonzero directions. In this setting the original coefficients are not unique; the supplied least-squares convention returns the solution with the smallest sum of squared coefficients. This comparison is about equivalent predictions under a specified numerical convention.

In [48]:
```
# Q4.3: original-space least squares versus regression on every nonzero principal c
def center_fit(x):
    """Return the column means of the fitting spectra."""
    return x.mean(axis=0)


def original_ols(x_fit, y_fit, x_eval):
    """Minimum-norm least squares on centered spectra; intercept = mean response."""
    mean = center_fit(x_fit)
    beta = np.linalg.lstsq(x_fit - mean, y_fit - y_fit.mean(), rcond=1e-10)[0]
    return (x_eval - mean) @ beta + y_fit.mean(), beta
```

```
def fit_pca(x_fit, tol=1e-10):
    """Centering and SVD of the fitting spectra; keep singular values above tol * l
    mean = center_fit(x_fit)
    _, s, vt = np.linalg.svd(x_fit - mean, full_matrices=False)
    keep = s > s[0] * tol
    return {"mean": mean, "components": vt[keep], "singular_values": s[keep],
            "share": s[keep] ** 2 / np.sum(s ** 2)}


def pcr_all(x_fit, y_fit, x_eval):
    """Least squares on all retained principal-component scores."""
    pca = fit_pca(x_fit)
    scores = (x_fit - pca["mean"]) @ pca["components"].T
    gamma = np.linalg.lstsq(scores, y_fit - y_fit.mean(), rcond=1e-10)[0]
    return (x_eval - pca["mean"]) @ pca["components"].T @ gamma + y_fit.mean(), pca


centered = cookie_X - cookie_X.mean(axis=0)
full_pred, _ = original_ols(cookie_X, cookie_y, cookie_X)
pcr_full_pred, pca_full_all = pcr_all(cookie_X, cookie_y, cookie_X)
equivalence = [float(np.max(np.abs(full_pred - pcr_full_pred)))]
diagnosis_rows = [{"fit": "all calibration", "fitting rows": len(cookie_y), "predic
                   "rank (centered)": int(np.linalg.matrix_rank(centered, tol=np.li
                   "PCs kept": len(pca_full_all["singular_values"]),
                   "train RMSE": rmse(cookie_y, full_pred), "held-out RMSE": np.nan
cv_pred = np.empty(len(cookie_y))
for fold, (tr, va) in enumerate(cookie_folds):
    orig_va, _ = original_ols(cookie_X[tr], cookie_y[tr], cookie_X[va])
    orig_tr, _ = original_ols(cookie_X[tr], cookie_y[tr], cookie_X[tr])
    pcr_va, pca_fold = pcr_all(cookie_X[tr], cookie_y[tr], cookie_X[va])
    cv_pred[va] = orig_va
    equivalence.append(float(np.max(np.abs(orig_va - pcr_va))))
    c_tr = cookie_X[tr] - cookie_X[tr].mean(axis=0)
    diagnosis_rows.append({"fit": f"fold {fold}", "fitting rows": len(tr), "predict
                           "rank (centered)": int(np.linalg.matrix_rank(c_tr, tol=n
                           "PCs kept": len(pca_fold["singular_values"]),
                           "train RMSE": rmse(cookie_y[tr], orig_tr), "held-out RMS
cookie_diagnosis = pd.DataFrame(diagnosis_rows)
display(cookie_diagnosis)
print(f"Original-space OLS: pooled CV RMSE = {rmse(cookie_y, cv_pred):.3f} sucrose
      f"(response SD = {cookie_y.std():.3f}, range {cookie_y.min():.2f}-{cookie_y.m
print("Largest |prediction difference| (full fit, then folds 0-4):", [f"{e:.2e}" fo
```

	fit	fitting rows	predictors	rank (centered)	PCs kept	train RMSE	held-out RMSE
0	all calibration	39	700	38	38	4.431773e-14	NaN
1	fold 0	31	700	30	30	1.528416e-13	1.083646
2	fold 1	31	700	30	30	5.531420e-14	0.988737
3	fold 2	31	700	30	30	6.919779e-14	1.499385
4	fold 3	31	700	30	30	3.058490e-14	1.529392
5	fold 4	32	700	31	31	2.977551e-14	1.689041

```
Original-space OLS: pooled CV RMSE = 1.376 sucrose percentage points (response SD =
3.907, range 9.95-23.11%)
Largest |prediction difference| (full fit, then folds 0-4): ['4.80e-14', '7.78e-13',
'4.97e-14', '5.68e-14', '3.20e-14', '7.28e-14']
```

In [49]:
```
grader.check("q4_3")
```

Out[49]:

q4_3 passed! 🍀

Written response — Q4.3

Diagnosis. The design has 39 spectra and 700 wavelengths; after centering its rank is 38 (30 or 31 in the CV fitting samples). Least squares fits every training sample exactly (training RMSE about 10^{-13}) and the coefficients are not unique: infinitely many vectors interpolate, and `lstsq` returns the one with the smallest sum of squares. That minimum-norm fit still predicts held-out samples reasonably (pooled CV RMSE 1.38 sucrose points, folds 0.99–1.69, against a response SD of 3.91), so original-space regression is not hopeless here.

Equivalence. Least squares on the centered spectra and on all principal components with singular values above 10^{-10} × the largest give the same predictions: the largest absolute differences are 4.8×10^{-14} (full fit) and 3.2×10^{-14} to 7.8×10^{-13} (the five held-out folds), which is rounding error. Keeping every nonzero component only rotates the same column space (scores = XV), leaving residuals and coefficient length unchanged. A change to PCA coordinates therefore does **not** by itself constrain the fit; PCR simplifies the model only when components are dropped.

Q4.4. Establish the 95%-variance PCR baseline

3 points

Using every available principal component does not by itself simplify the fitted regression. Retaining fewer components offers a way to reduce the number of directions used for prediction. A common starting rule is to retain enough components to explain 95% of predictor variance; the scree and cumulative-variance plots let us see what that rule keeps and what it discards.

Task: Plot the variance explained by each component and the cumulative explained variance. Find the smallest number of leading components that explains at least 95% on the full calibration data. Evaluate the corresponding PCR procedure using `cookie_folds`, relearning PCA and the 95% component count within each fitting fold.

Artifacts: Variance plot, full-calibration component count, CV performance, and an explanation of what information the 95% rule uses and what it ignores.

Constraints: Refit centering, PCA, and the 95% component count inside each fitting fold. Do not score models on the prediction batch.

Output names for the guided checks: `share`, the explained-variance ratios, and `k95`, the smallest qualifying component count.

In [50]:
```
# Q4.4: explained variance and the 95%-variance PCR baseline.
share = pca_full_all["share"]
k95 = int(np.searchsorted(np.cumsum(share), 0.95) + 1)
print(f"Full calibration: k95 = {k95} components explain {np.cumsum(share)[k95 - 1]
display(pd.DataFrame({"share": share[:10], "cumulative": np.cumsum(share)[:10]},
                     index=pd.RangeIndex(1, 11, name="component")).round(5))


def pcr_k95(x_fit, y_fit, x_eval):
    """Relearn centering, PCA and the 95% component count from the fitting rows, th
    pca = fit_pca(x_fit)
    k = int(np.searchsorted(np.cumsum(pca["share"]), 0.95) + 1)
    z_fit = (x_fit - pca["mean"]) @ pca["components"][:k].T
    model = LinearRegression().fit(z_fit, y_fit)
    return model.predict((x_eval - pca["mean"]) @ pca["components"][:k].T), k


baseline_pred = np.empty(len(cookie_y))
baseline_rows = []
for fold, (tr, va) in enumerate(cookie_folds):
    baseline_pred[va], k_fold = pcr_k95(cookie_X[tr], cookie_y[tr], cookie_X[va])
    baseline_rows.append({"fold": fold, "k95": k_fold, "held-out RMSE": rmse(cookie
display(pd.DataFrame(baseline_rows).round(3))
baseline_cv_rmse = rmse(cookie_y, baseline_pred)
print(f"95%-variance PCR: pooled CV RMSE = {baseline_cv_rmse:.3f} sucrose percentag
      f"(predicting the mean gives about {cookie_y.std():.3f})")

fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
comp = np.arange(1, len(share) + 1)
axes[0].bar(comp, share, color="tab:blue", label="per component")
axes[0].set_yscale("log")
```

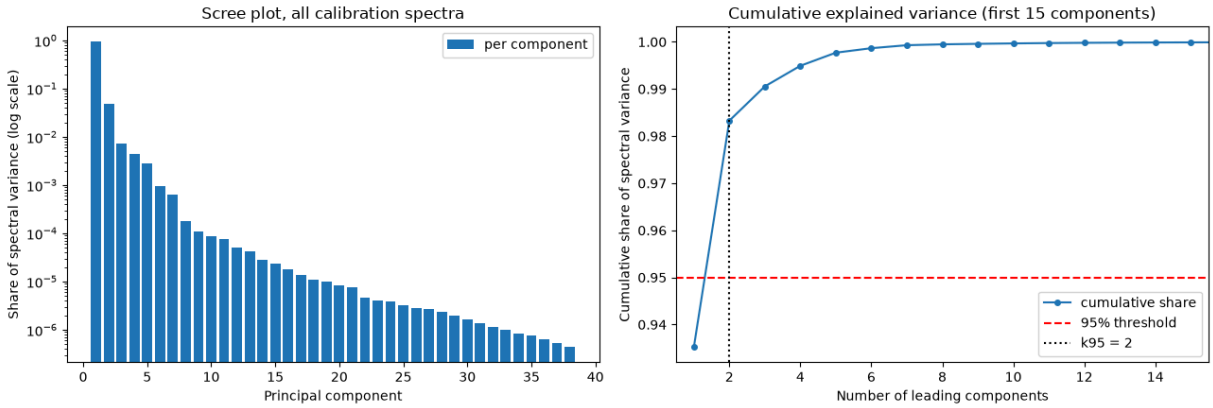
```
axes[0].set_xlabel("Principal component")
axes[0].set_ylabel("Share of spectral variance (log scale)")
axes[0].set_title("Scree plot, all calibration spectra")
axes[0].legend()
axes[1].plot(comp, np.cumsum(share), marker="o", ms=4, label="cumulative share")
axes[1].axhline(0.95, color="red", ls="--", label="95% threshold")
axes[1].axvline(k95, color="black", ls=":", label=f"k95 = {k95}")
axes[1].set_xlim(0.5, 15.5)
axes[1].set_xlabel("Number of leading components")
axes[1].set_ylabel("Cumulative share of spectral variance")
axes[1].set_title("Cumulative explained variance (first 15 components)")
axes[1].legend()
plt.tight_layout()
plt.show()
```

```
Full calibration: k95 = 2 components explain 98.33% of spectral variance
```

component	share	cumulative
1	0.93521	0.93521
2	0.04807	0.98328
3	0.00728	0.99056
4	0.00437	0.99493
5	0.00278	0.99771
6	0.00095	0.99866
7	0.00063	0.99930
8	0.00018	0.99948
9	0.00011	0.99959
10	0.00009	0.99968

	fold	k95	held-out RMSE
0	0	2	3.205
1	1	2	3.223
2	2	2	4.078
3	3	2	4.269
4	4	2	2.843

```
95%-variance PCR: pooled CV RMSE = 3.583 sucrose percentage points (predicting the m
ean gives about 3.907)
```



In [51]:
```
grader.check("q4_4")
```

Out[51]:

q4_4 passed! ✨

Written response — Q4.4

Spectral variance is extremely concentrated: PC1 explains 93.5%, PC2 4.8% and PC3–PC6 together about 1.5%. On the full calibration data **k95 = 2** (98.3%), and every CV fitting fold also chose 2. The resulting PCR predicts poorly: pooled CV RMSE is **3.58** sucrose points (folds 2.84–4.27), barely better than predicting the mean (SD 3.91) and far worse than minimum-norm regression on all components (1.38).

The 95% rule uses only the spectra: it keeps the directions of greatest absorbance variation, presumably overall intensity and baseline differences between doughs, and ignores sucrose entirely. Nothing guarantees that the leading components carry the sucrose signal: using all components cuts CV RMSE from 3.58 to 1.38 (Q4.3), so the useful information also sits in discarded components that each explain under 1% of the variance. A variance threshold describes the predictors; it is not a model-selection criterion for a response.

Q4.5. Select components using predictive evidence

4 points

The 95% rule chooses components using the spectra alone, without considering sucrose percentage. Directions that explain substantial variation in the measurements need not be the directions most useful for predicting the response. We can now draw on the earlier modeling work to develop and evaluate a component-selection approach that uses predictive evidence.

Task: Propose, justify and implement a predictive component-selection strategy and compare predictions on the same outer held-out observations in `cookie_folds` as the baseline. Any tuning must occur inside each outer fitting sample.

Artifacts: Selection rationale, selected component count or set and predictive evidence.

Constraints: You may adapt a method used earlier or propose another suitable method. Explain how prediction performance informs your component choice. You may consider components beyond the 95% cutoff. Fit transformations and tune choices using only the relevant fitting data, and compare methods on common held-out observations. A justified method can receive full credit even if it does not improve on the baseline.

Written response — Q4.5

Method. We select components with the lasso on the **standardized PC scores** of all available components (30–31 per outer fitting sample, 38 on the full calibration). It adapts Q2.2 and can keep a lower-variance component while skipping a leading one, which matters because Q4.4 showed useful information beyond the leading two components. Dividing each score by its fitting-sample SD gives every direction equal footing under the penalty (PC1's raw scores are about 100 times larger than PC10's). Within each outer fitting sample the penalty is tuned over `logspace(0, -4, 41)` by 5-fold inner CV (minimum pooled-MSE rule), refitting centering, PCA and score scaling in every inner fold, so outer held-out samples never influence the transformation, penalty or selection.

Evidence (same outer `cookie_folds`):

procedure	pooled CV RMSE (sucrose %)
original-space min-norm OLS	1.38
95%-variance PCR (k = 2)	3.58
lasso on standardized PC scores	**1.10**

The lasso beats the baseline in every fold (for example 0.71 against 3.21 in fold 0) and has smaller squared error on 35 of 39 held-out samples. Inner CV chose penalties of 0.13–0.20 and kept 12–18 components per fold, always PCs 1–7 except PC4 in fold 4. On all 39 samples the frozen procedure chose α = 0.079 and **20 components**: PCs 1–7, 10, 11, 13, 14, 17, 20, 22, 24, 25, 31, 32, 33 and 36. Prediction performance drives the choice at both levels (the penalty minimizes inner-CV error; a component stays only if it improves the penalized fit to sucrose), at the cost that tiny-variance components, which may be mostly noise, get the same chance as real signal; Q4.6 checks the high-index selections.

In [52]:
```
# Q4.5: lasso on standardized PC scores, penalty tuned by inner CV inside each oute
COOKIE_ALPHAS = np.logspace(0, -4, 41)
COOKIE_LASSO = {"max_iter": 200_000, "tol": 1e-8}


def score_transform(pca, scale, x):
    """Project spectra onto fixed components and divide by fixed score SDs."""
    return ((x - pca["mean"]) @ pca["components"].T) / scale


def tune_score_lasso(z, y, folds, alphas=COOKIE_ALPHAS, x=None):
    """Pooled-MSE lasso tuning. If x is given, refit PCA and scaling inside each in
```

```
    squared = np.zeros(len(alphas))
    for itr, iva in folds:
        if x is not None:
            pca_i = fit_pca(x[itr])
            raw = (x[itr] - pca_i["mean"]) @ pca_i["components"].T
            scale_i = raw.std(axis=0)
            z_tr, z_va = raw / scale_i, score_transform(pca_i, scale_i, x[iva])
        else:
            z_tr, z_va = z[itr], z[iva]
        model = Lasso(alpha=alphas[0], warm_start=True, **COOKIE_LASSO)
        for i, a in enumerate(alphas):
            model.set_params(alpha=a)
            model.fit(z_tr, y[itr])
            squared[i] += np.sum((y[iva] - model.predict(z_va)) ** 2)
    mse = squared / sum(len(iva) for _, iva in folds)
    return float(alphas[int(np.argmin(mse))]), mse


def fit_pcr_lasso(x_fit, y_fit, inner_folds):
    """Full procedure on one fitting sample: PCA, score scaling, inner-CV penalty,
    pca = fit_pca(x_fit)
    raw = (x_fit - pca["mean"]) @ pca["components"].T
    scale = raw.std(axis=0)
    alpha, mse = tune_score_lasso(None, y_fit, inner_folds, x=x_fit)
    model = Lasso(alpha=alpha, **COOKIE_LASSO).fit(raw / scale, y_fit)
    return {"pca": pca, "scale": scale, "alpha": alpha, "model": model, "mse": mse,
            "selected": np.flatnonzero(np.abs(model.coef_) > 1e-8)}


def predict_pcr_lasso(fit, x):
    """Predict sucrose for spectra x with a fitted PCA-lasso procedure."""
    return fit["model"].predict(score_transform(fit["pca"], fit["scale"], x))


lasso_pred = np.empty(len(cookie_y))
outer_fits = []
outer_rows = []
for fold, (tr, va) in enumerate(cookie_folds):
    fit = fit_pcr_lasso(cookie_X[tr], cookie_y[tr], fixed_folds(len(tr), seed=209))
    outer_fits.append(fit)
    lasso_pred[va] = predict_pcr_lasso(fit, cookie_X[va])
    outer_rows.append({"fold": fold, "alpha": fit["alpha"], "components selected":
                       "selected (1-based)": (fit["selected"] + 1).tolist(),
                       "lasso RMSE": rmse(cookie_y[va], lasso_pred[va]),
                       "95% baseline RMSE": rmse(cookie_y[va], baseline_pred[va])})
with pd.option_context("display.max_colwidth", 120):
    display(pd.DataFrame(outer_rows).round(3))
lasso_cv_rmse = rmse(cookie_y, lasso_pred)
pcr_comparison = pd.DataFrame(
    {"pooled CV RMSE": [rmse(cookie_y, cv_pred), baseline_cv_rmse, lasso_cv_rmse],
     "components": ["all (min-norm OLS)", "leading k95 per fold", "lasso-selected p
    index=["original-space OLS", "95%-variance PCR", "lasso on PC scores"],
)
display(pcr_comparison.round(3))
paired = (cookie_y - baseline_pred) ** 2 - (cookie_y - lasso_pred) ** 2
print(f"Lasso has smaller squared error on {int((paired > 0).sum())} of {len(cookie
```

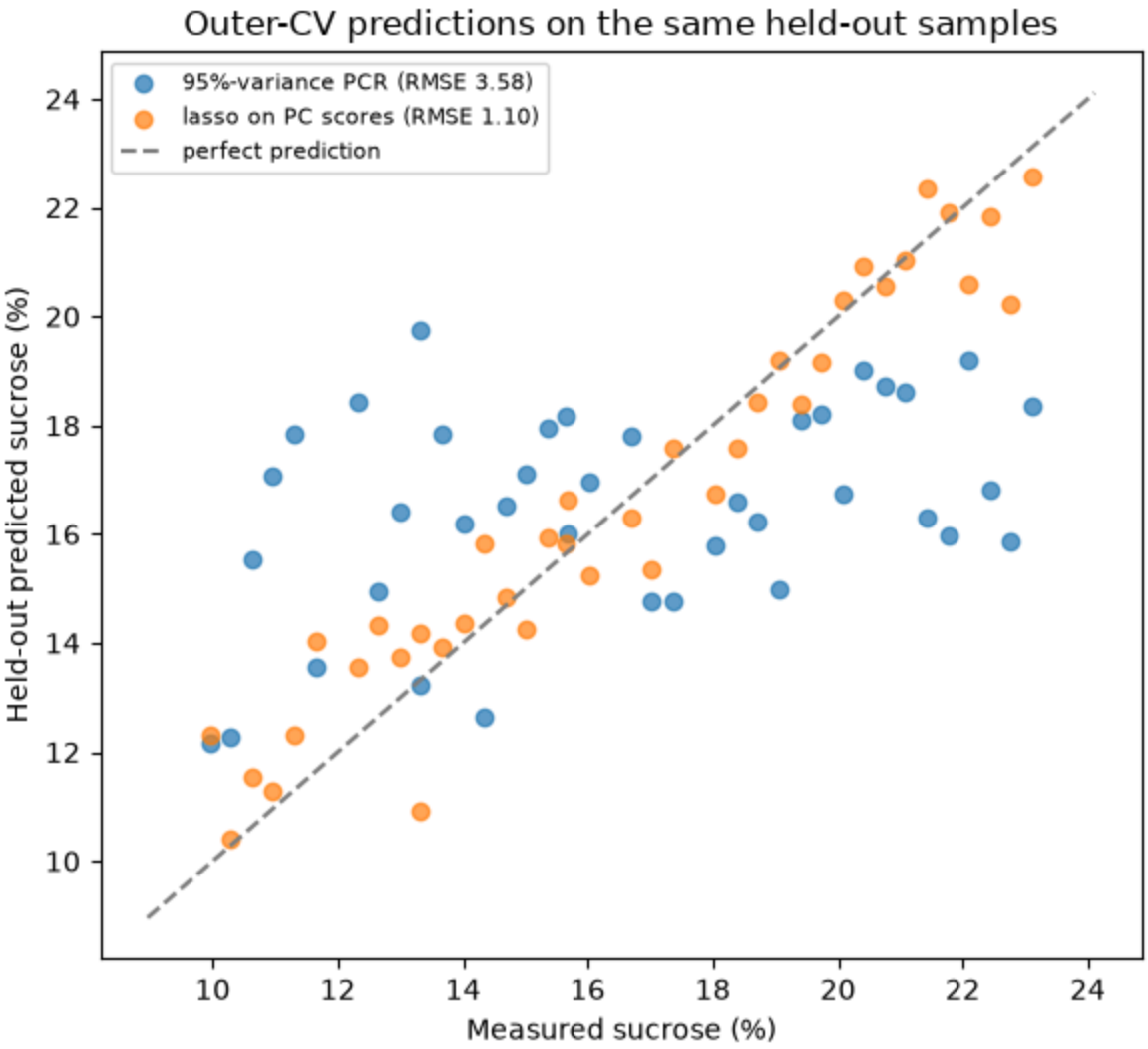
```
# The frozen procedure refitted on all calibration samples (used in Q4.6-Q4.10).
cookie_final = fit_pcr_lasso(cookie_X, cookie_y, fixed_folds(len(cookie_y), seed=20
print(f"All calibration: alpha = {cookie_final['alpha']:.4g}; {len(cookie_final['se
      f"(1-based): {(cookie_final['selected'] + 1).tolist()}")

fig, ax = plt.subplots(figsize=(6, 5.5))
ax.scatter(cookie_y, baseline_pred, label=f"95%-variance PCR (RMSE {baseline_cv_rms
ax.scatter(cookie_y, lasso_pred, label=f"lasso on PC scores (RMSE {lasso_cv_rmse:.2
lims = [cookie_y.min() - 1, cookie_y.max() + 1]
ax.plot(lims, lims, color="gray", ls="--", label="perfect prediction")
ax.set_xlabel("Measured sucrose (%)")
ax.set_ylabel("Held-out predicted sucrose (%)")
ax.set_title("Outer-CV predictions on the same held-out samples")
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()
```

	fold	alpha	components selected	selected (1-based)	lasso RMSE	95% baseline RMSE
0	0	0.126	18	[1, 2, 3, 4, 5, 6, 7, 8, 11, 12, 13, 17, 20, 22, 24, 25, 26, 27]	0.710	3.205
1	1	0.200	12	[1, 2, 3, 4, 5, 6, 7, 10, 13, 19, 22, 26]	0.748	3.223
2	2	0.158	12	[1, 2, 3, 4, 5, 6, 7, 11, 12, 13, 22, 28]	1.428	4.078
3	3	0.158	13	[1, 2, 3, 4, 5, 6, 7, 9, 12, 19, 21, 25, 30]	1.204	4.269
4	4	0.126	16	[1, 2, 3, 5, 6, 7, 9, 10, 12, 13, 14, 16, 22, 25, 27, 30]	1.225	2.843

	pooled CV RMSE	components
original-space OLS	1.376	all (min-norm OLS)
95%-variance PCR	3.583	leading k95 per fold
lasso on PC scores	1.097	lasso-selected per fold

```
Lasso has smaller squared error on 35 of 39 held-out samples.
All calibration: alpha = 0.07943; 20 components selected (1-based): [1, 2, 3, 4, 5,
6, 7, 10, 11, 13, 14, 17, 20, 22, 24, 25, 31, 32, 33, 36]
```



Q4.6. Assess selection stability

3 points

The response-informed procedure selected components using one calibration sample. Before interpreting that choice, we need evidence about whether it is sensitive to changes in the observations used for fitting. A stability assessment appropriate to your chosen method can help establish which selection conclusions are reproducible and which depend strongly on this sample.

Task: Choose, justify, and implement a stability assessment appropriate to your component-selection method. You may adapt the earlier analysis or propose an alternative. If you use resampling (e.g., bootstrap), hold the fitted PCA transformation and any score scaling your method uses fixed and repeat the downstream selection or tuning appropriate to your method.

Artifacts: Method-appropriate stability plot/table and interpretation.

Constraints: When calculating selection frequencies, allow the selected subset to vary across resamples; hold the PCA transformation fixed, not the selection result. Specify what is repeated and what remains fixed.

If your resampling assessment includes CV tuning, use `bootstrap_folds` as in Q3.1 so copies of a calibration sample stay in the same fold. You may use `B_SELECTION` as the supplied resampling count. Methods without discrete selection may assess variation in their chosen model size or predictions instead; explain what your assessment measures.

If you use bootstrap resampling, pass the resampled calibration-row positions `idx` as the `ids` argument to `bootstrap_folds`; these positions identify original samples, including repeated copies.

Written response — Q4.6

Fixed and repeated. We held the full-calibration PCA fixed (centering, 38 directions, score SDs). In each of 100 bootstrap resamples of the 39 samples we retuned the penalty by 5-fold CV with `bootstrap_folds` (copies of a sample share a fold), refit the lasso and recorded which fixed components are nonzero, letting the set vary.

Results. Only the six leading components are reproducible: PC3 and PC5 in 100% of resamples, PC1 99%, PC6 84%, PC2 83%, PC4 80%. Frequencies then fall to 68% (PC10), 60% (PC7, which every Q4.5 outer fold kept, and PC32) and below 60% for the rest. The set size ranges from 3 to 27 (median 17) and the penalty spans three orders of magnitude (10th–90th percentile 2.5×10^{-4} to 0.32), so the CV curve on 39 samples is poorly determined. Of the full-calibration model's 20 components, six are stable and fourteen (including PCs 31, 32, 33 and 36, which explain almost no variance) depend strongly on which samples are drawn.

Conclusion. "The model relies on PCs 1–6" is reproducible; the higher-order set should not be interpreted. Holding the PCA fixed means these frequencies understate total instability (Extension 2).

In [53]:
```
# Q4.6: bootstrap stability of the lasso component selection, PCA and score scaling
fixed_pca = cookie_final["pca"]
fixed_scale = cookie_final["scale"]
fixed_scores = score_transform(fixed_pca, fixed_scale, cookie_X)
n_components = fixed_scores.shape[1]


def cookie_selection_bootstrap():
    """Resample calibration rows; retune and refit the lasso on the fixed standardi
    rng = np.random.default_rng(SELECTION_BOOTSTRAP_SEED)
    n = len(cookie_y)
    alphas = np.empty(B_SELECTION)
    selected = np.zeros((B_SELECTION, n_components), dtype=bool)
    for b in range(B_SELECTION):
        idx = rng.integers(0, n, size=n)
        folds_b = bootstrap_folds(idx, seed=109 + b)  # copies of a sample share a
        alpha_b, _ = tune_score_lasso(fixed_scores[idx], cookie_y[idx], folds_b)
```

```
        model_b = Lasso(alpha=alpha_b, **COOKIE_LASSO).fit(fixed_scores[idx], cooki
        alphas[b] = alpha_b
        selected[b] = np.abs(model_b.coef_) > 1e-8
    return {"alpha": alphas, "selected": selected}

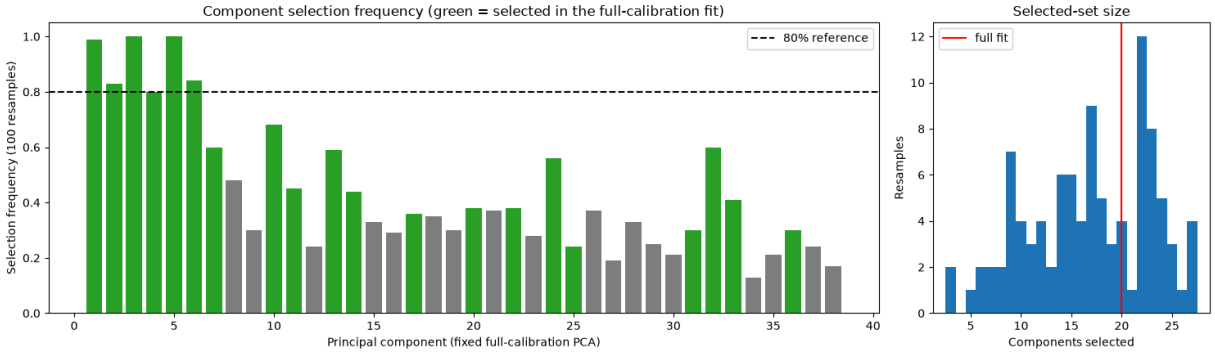

cookie_cache = cached_bootstrap_results(
    cookie_selection_bootstrap,
    cache_path="cache/q4_6_cookie_selection_bootstrap.npz",
    X=cookie_X,
    y=cookie_y,
    settings={
        "implementation_version": "q4_6-v1",
        "B_SELECTION": B_SELECTION,
        "seed": SELECTION_BOOTSTRAP_SEED,
        "fold_seed_base": 109,
        "COOKIE_ALPHAS": COOKIE_ALPHAS,
        "COOKIE_LASSO": COOKIE_LASSO,
        "fixed_scale": fixed_scale,
        "pca_tol": 1e-10,
    },
    dependencies=[tune_score_lasso, score_transform, fit_pca, center_fit, bootstrap
    force=REFRESH_BOOTSTRAP_CACHE,
)
component_frequency = cookie_cache["selected"].mean(axis=0)
component_nonzero = cookie_cache["selected"].sum(axis=1)
cookie_stability = pd.DataFrame(
    {"component": np.arange(1, n_components + 1), "variance share": fixed_pca["shar
     "selection frequency": component_frequency,
     "selected in full fit": np.isin(np.arange(n_components), cookie_final["selecte
)
display(cookie_stability[cookie_stability["selection frequency"] > 0].sort_values("
print(f"Components selected in at least 80% of resamples: "
      f"{(np.flatnonzero(component_frequency >= 0.8) + 1).tolist()}")
print(f"Selected-set size across resamples: median {np.median(component_nonzero):.0
      f"range {component_nonzero.min()}-{component_nonzero.max()}; full fit {len(co
print(f"Bootstrap penalties: median {np.median(cookie_cache['alpha']):.3g}, "
      f"10-90% range {np.percentile(cookie_cache['alpha'], 10):.3g}-{np.percentile(
      f"full fit {cookie_final['alpha']:.3g}")

fig, axes = plt.subplots(1, 2, figsize=(15, 4.5), gridspec_kw={"width_ratios": [3,
colors_sel = np.where(cookie_stability["selected in full fit"], "tab:green", "tab:g
axes[0].bar(cookie_stability.component, component_frequency, color=colors_sel)
axes[0].axhline(0.8, color="black", ls="--", label="80% reference")
axes[0].set_xlabel("Principal component (fixed full-calibration PCA)")
axes[0].set_ylabel(f"Selection frequency ({B_SELECTION} resamples)")
axes[0].set_title("Component selection frequency (green = selected in the full-cali
axes[0].legend()
axes[1].hist(component_nonzero, bins=np.arange(component_nonzero.min() - 0.5, compo
axes[1].axvline(len(cookie_final["selected"]), color="red", label="full fit")
axes[1].set_xlabel("Components selected")
axes[1].set_ylabel("Resamples")
axes[1].set_title("Selected-set size")
axes[1].legend()
plt.tight_layout()
plt.show()
```

```
Saved fresh bootstrap results to cache.
```

	component	variance share	selection frequency	selected in full fit
4	5	0.0028	1.00	True
2	3	0.0073	1.00	True
0	1	0.9352	0.99	True
5	6	0.0010	0.84	True
1	2	0.0481	0.83	True
3	4	0.0044	0.80	True
9	10	0.0001	0.68	True
6	7	0.0006	0.60	True
31	32	0.0000	0.60	True
12	13	0.0000	0.59	True
23	24	0.0000	0.56	True
7	8	0.0002	0.48	False
10	11	0.0001	0.45	True
13	14	0.0000	0.44	True
32	33	0.0000	0.41	True
19	20	0.0000	0.38	True
21	22	0.0000	0.38	True
20	21	0.0000	0.37	False
25	26	0.0000	0.37	False
16	17	0.0000	0.36	True
17	18	0.0000	0.35	False
14	15	0.0000	0.33	False
27	28	0.0000	0.33	False
30	31	0.0000	0.30	True
8	9	0.0001	0.30	False

```
Components selected in at least 80% of resamples: [1, 2, 3, 4, 5, 6]
Selected-set size across resamples: median 17, range 3-27; full fit 20
Bootstrap penalties: median 0.1, 10-90% range 0.000246-0.316; full fit 0.0794
```



AC209A Extension 2. Explain the staff's stability procedure (AC209A only)

4 points

The staff deliberately specified a different approach to the bootstrap stability analysis here than in the earlier dataset. What would be problematic about applying the earlier approach directly in this setting? Explain your reasoning, and explain what the specified analysis measures and what it leaves out.

Artifact: A short conceptual explanation.

In [54]:
```
# AC209A Extension 2 (illustration): what happens if PCA is refitted in every boots
rng_ext2 = np.random.default_rng(8109)
n_check = 12
match_rows = []
for b in range(50):
    idx = rng_ext2.integers(0, len(cookie_y), size=len(cookie_y))
    pca_b = fit_pca(cookie_X[idx])
    # |cosine| between each refitted component and each fixed component.
    cos = np.abs(pca_b["components"] @ fixed_pca["components"].T)
    for j in range(n_check):
        best = int(np.argmax(cos[j]))
        match_rows.append({"resample": b, "component": j + 1, "distinct samples": l
                           "components available": len(pca_b["singular_values"]),
                           "|cos| with same-numbered fixed PC": cos[j, j], "best-ma
                           "sign flipped": float(np.sign(pca_b["components"][j] @ f
matches = pd.DataFrame(match_rows)
matches["same index"] = (matches["best-matching fixed PC"] == matches["component"])
summary_ext2 = matches.groupby("component").agg(
    mean_abs_cos=("|cos| with same-numbered fixed PC", "mean"),
    share_same_index=("same index", "mean"),
    share_sign_flipped=("sign flipped", "mean"),
)
display(summary_ext2.round(3))
print(f"Distinct calibration samples per resample: median {matches['distinct sample
      f"components available after refitting: {matches['components available'].min(
      f"(fixed PCA has {n_components})")
```

component	mean_abs_cos	share_same_index	share_sign_flipped
1	0.999	1.00	0.58
2	0.992	1.00	0.46
3	0.948	1.00	0.36
4	0.791	0.80	0.58
5	0.787	0.80	0.50
6	0.777	0.80	0.52
7	0.782	0.80	0.46
8	0.712	0.74	0.46
9	0.513	0.44	0.40
10	0.421	0.30	0.40
11	0.481	0.44	0.42
12	0.469	0.42	0.56

```
Distinct calibration samples per resample: median 25; components available after ref
itting: 19-28 (fixed PCA has 38)
```

Written response — AC209A Extension 2

In Q3.1 every resample refitted all preprocessing, which worked because Ames encoded features have fixed identities: `Gr_Liv_Area` is the same column every time. Principal components do not: they are *learned* from the fitting rows, so refitting PCA in each resample changes what "component j" means. Refitting PCA on 50 resamples shows three problems.

- **Identity changes.** PCs 1–3 are recovered almost exactly (mean |cos| 0.95–1.00 with the same-numbered fixed component), PCs 4–8 only at |cos| ≈ 0.71–0.79 (a different fixed component is the best match in 20–26% of resamples), and by PCs 9–12 |cos| falls to 0.42–0.51 with the same index the best match less than half the time. Counting how often "PC10" is selected would mix different directions.
- **Signs are arbitrary.** About half of the refitted components flip sign, so coefficients could not be pooled either.
- **Duplicates shrink the basis.** A resample of 39 has a median of only 25 distinct spectra and 19–28 nonzero components, not 38, so high-index components often cannot exist and their selection frequencies would be meaningless.

The staff's analysis holds the fitted PCA and score scaling fixed and repeats only tuning and lasso selection. It measures how sensitive the downstream choice is to which calibration samples are drawn, given one set of directions. It leaves out uncertainty in the directions themselves, estimated from 39 spectra: a different sample would give different loadings and

might not find the higher-order components at all. The fixed-basis frequencies are conditional on this PCA and understate the instability of the full procedure, which is why Q4.7 bootstraps the whole procedure.

Q4.7. Quantify uncertainty

3 points

Selection stability addresses whether the procedure repeatedly retains similar components. It does not establish how precisely their effects, or the resulting predictions, are estimated. An uncertainty assessment helps determine how strongly the data support the claims you intend to make about your chosen model.

Task: Choose and implement an uncertainty analysis for a clearly stated claim about coefficients or predictions. You may adapt an earlier method or propose a suitable alternative. Explain why the method answers your question and use its results to qualify the claim.

Artifacts: Interval or distribution evidence and interpretation of what it captures.

Constraints: State which fitting choices remain fixed and which sources of uncertainty your analysis includes. Intervals from OLS fitted after outcome-based selection on the same observations do not automatically account for that selection.

Written response — Q4.7

Claim. For a calibration spectrum, the frozen procedure's prediction typically moves by about ±1 sucrose percentage point when the whole procedure is rerun on a different calibration sample of the same size, and by about ±1.3 for a spectrum the rerun sample did not contain.

Method. We bootstrapped the **entire procedure** 200 times: each resample of the 39 samples refits centering, PCA and score scaling, retunes the penalty by grouped CV, reselects components and refits the lasso, then predicts all 39 spectra (95% intervals from the 2.5th–97.5th percentiles, also computed from only the resamples that excluded the spectrum). Fixed: the penalty grid, 5-fold grouped CV with the minimum pooled-MSE rule, and the lasso on standardized scores. Because selection is repeated in every resample, the intervals include the outcome-based component selection.

Results. The median interval width is **2.1** points (range 1.3–5.3; median bootstrap SD 0.55), supporting ±1 for a typical spectrum, and 2.6 using only out-of-bag resamples. Widths tend to be larger at the ends of the sucrose range, with bootstrap means pulled toward the middle (a shrinkage bias).

Scope. Included: calibration-sampling variability through every fitting choice. Not included: measurement noise in a new sample's sucrose (so these are not prediction intervals for true sucrose), systematic bias, and future batches that differ. The calibration spectra were part of the frozen fit and of about 63% of the resamples, so the pooled ±1 is optimistic for new spectra (out-of-bag RMSE 1.43 against 1.10 from outer CV); resamples with only about 25 distinct spectra push the other way.

In [55]:
```
# Q4.7: claim about predictions. Bootstrap the whole procedure (centering, PCA, sca
B_PROCEDURE = 200
rng_proc = np.random.default_rng(7109)
procedure_preds = np.empty((B_PROCEDURE, len(cookie_y)))
procedure_sizes = np.empty(B_PROCEDURE, dtype=int)
oob_sq, oob_n = 0.0, 0
in_bag = np.zeros((B_PROCEDURE, len(cookie_y)), dtype=bool)
for b in range(B_PROCEDURE):
    idx = rng_proc.integers(0, len(cookie_y), size=len(cookie_y))
    fit_b = fit_pcr_lasso(cookie_X[idx], cookie_y[idx], bootstrap_folds(idx, seed=1
    procedure_preds[b] = predict_pcr_lasso(fit_b, cookie_X)
    procedure_sizes[b] = len(fit_b["selected"])
    in_bag[b, idx] = True
    oob = np.setdiff1d(np.arange(len(cookie_y)), idx)
    oob_sq += np.sum((cookie_y[oob] - procedure_preds[b, oob]) ** 2)
    oob_n += len(oob)

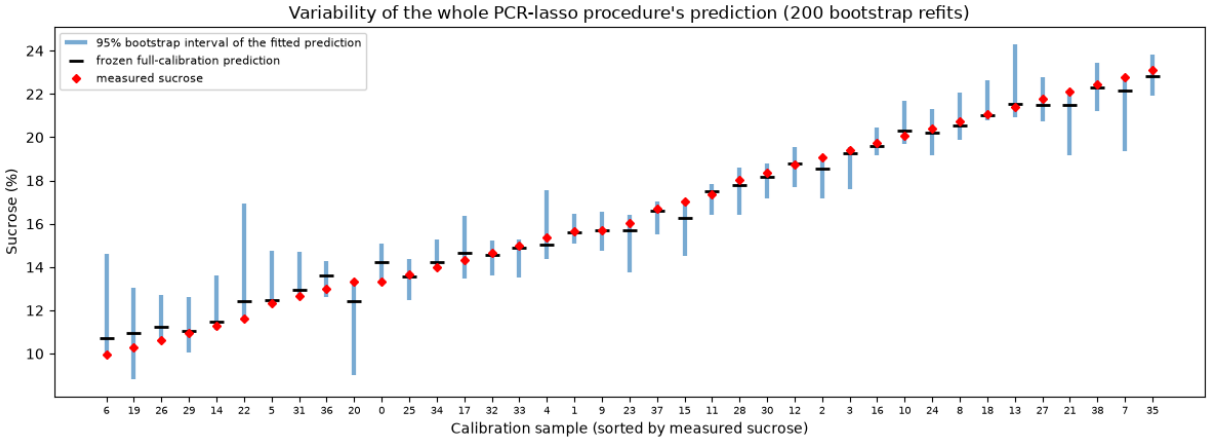
final_cal_pred = predict_pcr_lasso(cookie_final, cookie_X)
prediction_uncertainty = pd.DataFrame(
    {"sample": np.arange(len(cookie_y)), "measured": cookie_y, "frozen-fit predicti
     "bootstrap mean": procedure_preds.mean(axis=0), "bootstrap sd": procedure_pred
     "lower": np.percentile(procedure_preds, 2.5, axis=0), "upper": np.percentile(p
)
prediction_uncertainty["width"] = prediction_uncertainty.upper - prediction_uncerta
oob_preds = np.where(in_bag, np.nan, procedure_preds)  # each spectrum, only from r
oob_width = np.nanpercentile(oob_preds, 97.5, axis=0) - np.nanpercentile(oob_preds,
display(prediction_uncertainty.round(3).head(10))
print(f"95% procedure-bootstrap interval width: median {prediction_uncertainty.widt
      f"range {prediction_uncertainty.width.min():.2f}-{prediction_uncertainty.widt
print(f"Out-of-bag-only interval width: median {np.median(oob_width):.2f} percentag
print(f"Bootstrap SD of a fitted prediction: median {prediction_uncertainty['bootst
print(f"Out-of-bag RMSE of the bootstrap procedures: {np.sqrt(oob_sq / oob_n):.2f}
      f"(outer-CV RMSE {lasso_cv_rmse:.2f})")
print(f"Components selected across procedure resamples: median {np.median(procedure
      f"range {procedure_sizes.min()}-{procedure_sizes.max()}")

order = np.argsort(cookie_y)
fig, ax = plt.subplots(figsize=(12, 4.5))
xs = np.arange(len(order))
pu = prediction_uncertainty.iloc[order]
ax.vlines(xs, pu.lower, pu.upper, color="tab:blue", lw=3, alpha=0.6, label="95% boo
ax.plot(xs, pu["frozen-fit prediction"], "_", color="black", ms=10, mew=2, label="f
ax.plot(xs, pu.measured, "D", color="red", ms=4, label="measured sucrose")
ax.set_xticks(xs, pu["sample"], fontsize=7)
ax.set_xlabel("Calibration sample (sorted by measured sucrose)")
ax.set_ylabel("Sucrose (%)")
ax.set_title(f"Variability of the whole PCR-lasso procedure's prediction ({B_PROCED
```

```
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()
```

	sample	measured	frozen-fit prediction	bootstrap mean	bootstrap sd	lower	upper	width
0	0	13.32	14.220	13.999	0.527	13.320	15.065	1.745
1	1	15.65	15.617	15.754	0.362	15.095	16.469	1.374
2	2	19.06	18.548	18.309	0.565	17.169	19.060	1.891
3	3	19.40	19.286	18.865	0.565	17.624	19.535	1.912
4	4	15.35	15.024	15.571	0.747	14.375	17.562	3.187
5	5	12.31	12.488	13.034	0.666	12.307	14.735	2.428
6	6	9.95	10.735	11.234	1.335	9.951	14.632	4.681
7	7	22.77	22.159	21.375	1.018	19.366	22.769	3.403
8	8	20.75	20.526	20.728	0.502	19.872	22.085	2.212
9	9	15.69	15.723	15.621	0.460	14.762	16.563	1.801

```
95% procedure-bootstrap interval width: median 2.08, range 1.27-5.31 percentage poin
ts
Out-of-bag-only interval width: median 2.57 percentage points
Bootstrap SD of a fitted prediction: median 0.55 points
Out-of-bag RMSE of the bootstrap procedures: 1.43 points (outer-CV RMSE 1.10)
Components selected across procedure resamples: median 16, range 6-26
```



Q4.8. Assess original-measurement importance

3 points

Our models use principal components, but the original observations are measurements at particular wavelengths. To connect predictive performance back to those measurements, we need an importance analysis in the original input space. Nearby wavelengths can carry

overlapping information, and perturbing them can produce unrealistic spectra, so the assessment also needs to explain the limits of what its results support.

Task: Choose, justify, and implement an importance analysis for original wavelengths or meaningful wavelength groups. You may adapt an earlier method or propose an alternative. If you use permutation importance, perturb held-out original measurements and pass them through the fitted preprocessing and model without refitting.

Artifacts: Importance/variability evidence and limitations.

Constraints: Correlated spectra can make perturbations unrealistic and rankings unstable. A large loading indicates that a wavelength contributes strongly to a component's score. Its predictive role also depends on how the regression uses that component.

Written response — Q4.8

Method. We measured held-out permutation importance for **20 contiguous bands of 35 wavelengths** (70 nm each), not single wavelengths: neighbors are almost perfectly correlated, so shuffling one creates spikes no real spectrum has. For each outer fold we kept the Q4.5 fit fixed, swapped a whole band between that fold's held-out samples, passed the altered spectra through the fitted transformation and model, and recorded the increase in pooled held-out MSE over all 39 samples (30 repeats per band).

Results. Baseline held-out MSE is 1.20 (sucrose %^{2}). The four most important bands lie within 1800–2150 nm: 1870–1938 nm raises MSE by 84 on average, 2080–2148 nm by 76, 1800–1868 nm by 68 and 2010–2078 nm by 44. Bands near 1520–1660 nm and 1940–2008 nm add 28–30; bands below 1300 nm or above 2220 nm add less than 10. The implied coefficient curve oscillates sharply between neighboring wavelengths and does not mirror these importances.

Limitations. The top-band increases are 37–70 times the baseline MSE because swapping a band splices together spectra with different absorbance levels, unlike any real dough, so only the ranking is informative; with 7–8 held-out samples per fold few distinct permutations exist, and adjacent bands overlap, so neighboring ranks are unstable. A large loading only means a wavelength contributes strongly to a component's score; its predictive role depends on how the regression uses that component. Wavelength-level attribution is unsupported (the coefficient curve is jagged, driven by unstable high-order components); band-level patterns are the most these data support.

In [56]:
```
# Q4.8: held-out permutation importance of contiguous wavelength bands.
BAND_WIDTH = 35  # 35 wavelengths x 2 nm = 70 nm per band, 20 bands
band_edges = np.arange(0, len(wavelengths) + 1, BAND_WIDTH)
bands = [np.arange(a, b) for a, b in zip(band_edges[:-1], band_edges[1:])]
band_labels = [f"{wavelengths[b[0]]:.0f}-{wavelengths[b[-1]]:.0f}" for b in bands]
rng_band = np.random.default_rng(9109)

base_sq = np.sum((cookie_y - lasso_pred) ** 2)
```

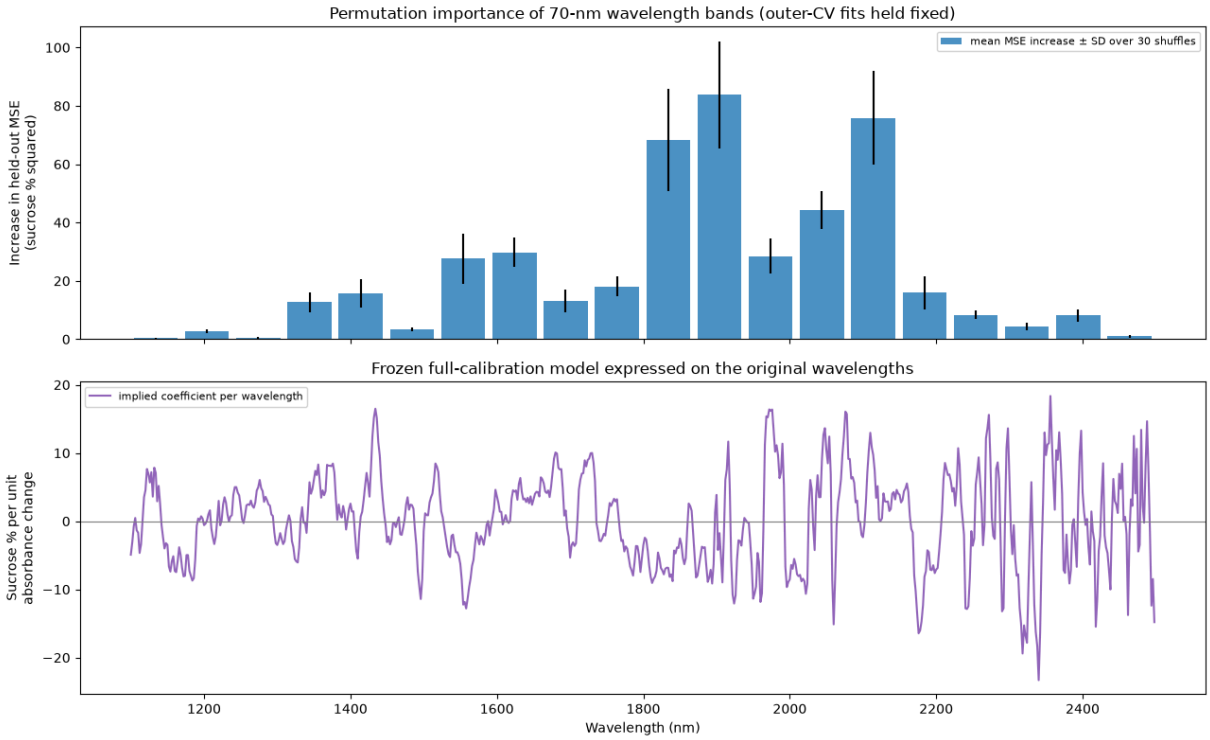
```
band_rows = []
for r in range(N_SHUFFLES):
    for band, label in zip(bands, band_labels):
        perturbed_pred = np.empty(len(cookie_y))
        for fit, (tr, va) in zip(outer_fits, cookie_folds):
            x_va = cookie_X[va].copy()
            # Swap the whole band between held-out samples; the outer-fold fit is n
            x_va[:, band] = x_va[rng_band.permutation(len(va))][:, band]
            perturbed_pred[va] = predict_pcr_lasso(fit, x_va)
        band_rows.append({"band (nm)": label, "replicate": r,
                          "degradation": (np.sum((cookie_y - perturbed_pred) ** 2)
band_repeats = pd.DataFrame(band_rows)
band_importance = band_repeats.groupby("band (nm)", sort=False)["degradation"].agg(
display(band_importance.sort_values("mean", ascending=False).round(3))
print(f"Baseline pooled held-out MSE: {base_sq / len(cookie_y):.3f} (sucrose percen

# Wavelength coefficient profile implied by the frozen full-calibration fit.
wavelength_coef = cookie_final["pca"]["components"].T @ (cookie_final["model"].coef

fig, axes = plt.subplots(2, 1, figsize=(13, 8), sharex=True)
centers = [wavelengths[b].mean() for b in bands]
axes[0].bar(centers, band_importance["mean"], width=60, yerr=band_importance["std"]
            alpha=0.8, label=f"mean MSE increase ± SD over {N_SHUFFLES} shuffles")
axes[0].set_ylabel("Increase in held-out MSE\n(sucrose % squared)")
axes[0].set_title("Permutation importance of 70-nm wavelength bands (outer-CV fits
axes[0].legend(fontsize=8)
axes[1].plot(wavelengths, wavelength_coef, color="tab:purple", label="implied coeff
axes[1].axhline(0, color="gray", lw=0.8)
axes[1].set_xlabel("Wavelength (nm)")
axes[1].set_ylabel("Sucrose % per unit\nabsorbance change")
axes[1].set_title("Frozen full-calibration model expressed on the original waveleng
axes[1].legend(fontsize=8)
plt.tight_layout()
plt.show()
```

band (nm)	mean	std
1870-1938	83.766	18.281
2080-2148	75.827	16.104
1800-1868	68.317	17.505
2010-2078	44.218	6.475
1590-1658	29.801	5.092
1940-2008	28.497	5.897
1520-1588	27.593	8.547
1730-1798	18.125	3.372
2150-2218	15.888	5.759
1380-1448	15.632	4.881
1660-1728	13.075	3.793
1310-1378	12.687	3.362
2220-2288	8.322	1.508
2360-2428	8.246	2.119
2290-2358	4.286	1.355
1450-1518	3.307	0.708
1170-1238	2.646	0.707
2430-2498	0.969	0.352
1240-1308	0.564	0.290
1100-1168	0.445	0.162

```
Baseline pooled held-out MSE: 1.203 (sucrose percentage points squared)
```



Q4.9. Interpret and synthesize

2 points

The stability, uncertainty, and importance analyses provide evidence for interpreting the fitted model. That interpretation is less direct than in the Ames example because each component combines many original measurements. Connecting the retained components to their loadings lets us explore useful wavelength contrasts while recognizing when the data do not support a clear attribution or a chemical explanation.

Task: Plot loadings for components relevant to your model and relate them to the stability, uncertainty, and importance evidence from Q4.6–Q4.8. Explain which wavelength-region patterns the evidence supports and which interpretations it cannot justify.

Artifacts: Justified loading plots and concise synthesis.

Constraints: No chemical mechanisms required. Limited interpretability is a valid conclusion. Null-based significance is optional, not a prerequisite for descriptive loading interpretation.

Written response — Q4.9

We plotted the loadings of the six components selected in at least 80% of fixed-PCA resamples (Q4.6), ordered by full-fit coefficient, with the three most important Q4.8 bands shaded.

- **PC3** (frequency 1.00, coefficient +2.46 per score SD, the largest): a trough near 1880–1990 nm against a peak near 2080 nm.

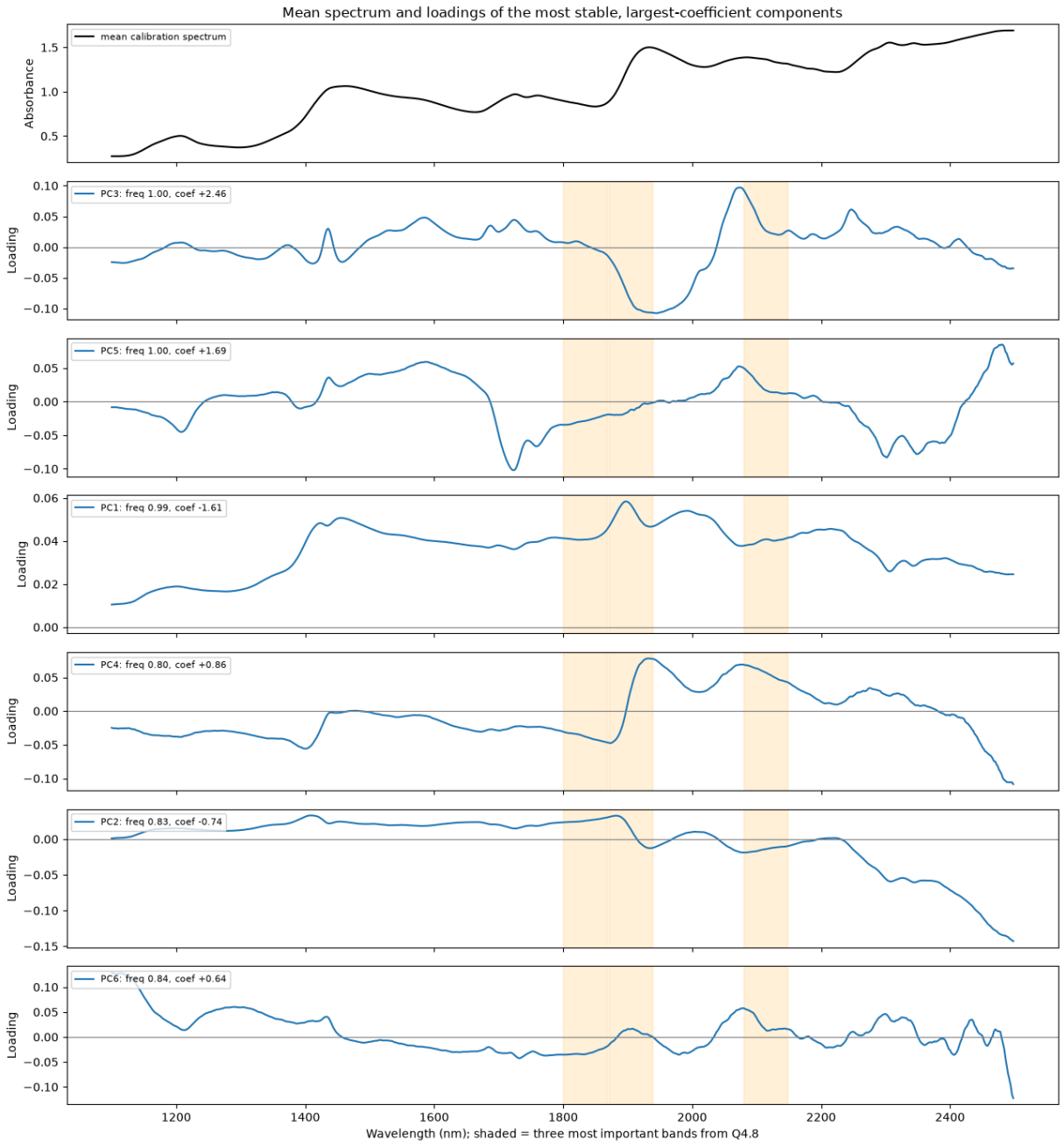
- **PC5** (1.00, +1.69): negative at 1700–1760 and 2300–2400 nm, positive near 1590 nm, 2080 nm and above 2450 nm.
- **PC1** (0.99, −1.61): positive at every wavelength, so it tracks overall absorbance.
- **PC4** (0.80, +0.86): a step from negative below about 1900 nm to positive at 1930–2150 nm. **PC2** (0.83, −0.74) and **PC6** (0.84, +0.64): weaker terms with broad tilts toward the long-wavelength end.

Supported: an overall-absorbance term plus contrasts around 1800–2150 nm: PC2–PC4 change sign within about 1840–2230 nm, PC3 and PC5 are selected in every resample (Q4.6), and shuffling bands there hurts held-out predictions most (Q4.8). **Not supported:** single-wavelength or narrow-peak attribution (the implied coefficient curve oscillates); the exact ranking and size of band importances (Q4.8 includes unstable higher-order components); interpreting the 14 less stable components; reading a loading's sign alone (component signs are arbitrary, only loading × coefficient matters); chemical mechanisms.

In [57]:
```
# Q4.9: loadings of the components the model relies on, alongside the Q4.6-Q4.8 evi
stable_components = np.flatnonzero(component_frequency >= 0.8)
coef_by_component = pd.Series(cookie_final["model"].coef_, index=np.arange(1, n_com
display(pd.DataFrame({
    "selection frequency": component_frequency[stable_components],
    "variance share": fixed_pca["share"][stable_components],
    "full-fit coefficient (per score SD)": coef_by_component.iloc[stable_components
}, index=pd.Index(stable_components + 1, name="component")).round(4))

show = stable_components[np.argsort(-np.abs(cookie_final["model"].coef_[stable_comp
fig, axes = plt.subplots(len(show) + 1, 1, figsize=(13, 2.0 * (len(show) + 1)), sha
axes[0].plot(wavelengths, cookie_X.mean(axis=0), color="black", label="mean calibra
axes[0].set_ylabel("Absorbance")
axes[0].set_title("Mean spectrum and loadings of the most stable, largest-coefficie
axes[0].legend(fontsize=8, loc="upper left")
for ax, j in zip(axes[1:], show):
    ax.plot(wavelengths, fixed_pca["components"][j], color="tab:blue",
            label=f"PC{j + 1}: freq {component_frequency[j]:.2f}, coef {cookie_fina
    ax.axhline(0, color="gray", lw=0.8)
    ax.set_ylabel("Loading")
    ax.legend(fontsize=8, loc="upper left")
    for (lo, hi), m in zip([(wavelengths[b[0]], wavelengths[b[-1]]) for b in bands]
        if m >= band_importance["mean"].nlargest(3).min():
            ax.axvspan(lo, hi, color="orange", alpha=0.15)
axes[-1].set_xlabel("Wavelength (nm); shaded = three most important bands from Q4.8
plt.tight_layout()
plt.show()
```

component	selection frequency	variance share	full-fit coefficient (per score SD)
1	0.99	0.9352	-1.6055
2	0.83	0.0481	-0.7373
3	1.00	0.0073	2.4615
4	0.80	0.0044	0.8561
5	1.00	0.0028	1.6868
6	0.84	0.0010	0.6448



Q4.10. Perform final PCR evaluation

2 points

All choices about preprocessing, component selection, and fitting have now been made using the calibration data. The separate prediction batch lets us assess how the frozen procedure performs on additional samples. Comparing the final models here completes the predictive evaluation without using this batch to revise their design.

Task: Freeze your chosen procedure, refit it on all calibration data, and evaluate it on the separate prediction batch. Compare it with the 95%-variance PCR baseline, refitted on all calibration data. Quantify uncertainty in the performance comparison and state what that uncertainty measures.

Artifacts: Final comparison and uncertainty summary.

Constraints: Do not revise model choices after examining the prediction batch. Distinguish uncertainty due to the finite evaluation sample from uncertainty due to developing and fitting the model. Because the staff previously explored this batch when designing the exercise, treat this evaluation as an instructional comparison rather than new confirmatory evidence.

Run this question only after all calibration analyses are finished. Call `load_cookie_test()` here, not before this question.

In [58]:
```
# Q4.10: final evaluation on the separate prediction batch (first and only use).
cookie_X_test, cookie_y_test = load_cookie_test()
final_lasso_test = predict_pcr_lasso(cookie_final, cookie_X_test)  # frozen in Q4.5
final_baseline_test, k_final = pcr_k95(cookie_X, cookie_y, cookie_X_test)
cookie_test_rmse = pd.Series({f"95%-variance PCR (k = {k_final})": rmse(cookie_y_te
                              "lasso on PC scores": rmse(cookie_y_test, final_lasso
display(cookie_test_rmse.round(3).to_frame())
print(f"Predicting the calibration mean gives RMSE {rmse(cookie_y_test, cookie_y.me

rng_test = np.random.default_rng(10109)
boot_rows = []
for _ in range(5000):
    i = rng_test.integers(0, len(cookie_y_test), size=len(cookie_y_test))
    r_base = rmse(cookie_y_test[i], final_baseline_test[i])
    r_lasso = rmse(cookie_y_test[i], final_lasso_test[i])
    boot_rows.append((r_base, r_lasso, r_base - r_lasso))
boot = np.array(boot_rows)
cookie_test_uncertainty = pd.DataFrame(
    np.percentile(boot, [2.5, 97.5], axis=0).T, columns=["2.5%", "97.5%"],
    index=["95%-variance PCR RMSE", "lasso RMSE", "difference (baseline - lasso)"])
display(cookie_test_uncertainty.round(3))
print(f"Lasso has smaller squared error on {int(np.sum((cookie_y_test - final_lasso
      f"of {len(cookie_y_test)} prediction-batch samples; share of resamples where
```

	test RMSE (sucrose %)
95%-variance PCR (k = 2)	4.556
lasso on PC scores	0.992

```
Predicting the calibration mean gives RMSE 3.90
```

	2.5%	97.5%
95%-variance PCR RMSE	3.616	5.467
lasso RMSE	0.670	1.346
difference (baseline - lasso)	2.575	4.601

```
Lasso has smaller squared error on 31 of 32 prediction-batch samples; share of resam
ples where lasso wins: 1.000
```

Written response — Q4.10

We froze the procedure before first calling `load_cookie_test()`: centering, PCA, score standardization and the lasso (5-fold-tuned α = 0.079, 20 components), plus the 95%-variance baseline (k = 2), both refitted on all 39 calibration samples. On the 32 prediction-batch samples:

model	test RMSE (sucrose %)	95% bootstrap interval
95%-variance PCR (k = 2)	4.56	[3.62, 5.47]
lasso on standardized PC scores	**0.99**	[0.67, 1.35]
difference (baseline − lasso)	3.56	[2.58, 4.60]

The response-informed procedure is far more accurate: smaller squared error on 31 of 32 samples and ahead in every bootstrap resample. Its test RMSE (0.99) is close to its outer-CV estimate (1.10); the baseline's (4.56) is worse than its CV estimate (3.58) and than predicting the calibration mean (3.90).

What the uncertainty measures. The intervals resample the 32 prediction samples with both fitted models fixed, so they capture only the finite evaluation sample, not variability from fitting on a different calibration sample (about ±1 point, Q4.7; unstable higher-order components, Q4.6) or from future batches. Staff explored this batch while designing the exercise, so this is an instructional comparison, not new confirmatory evidence.

Wrap-up

Both prediction problems, Ames and cookie dough, have now been evaluated after their modeling choices were fixed. Briefly describe one conclusion that depended on distinguishing stability, uncertainty, importance, or significance, and one limitation that remains. You do not need to repeat the earlier syntheses.

- How many hours did you spend on this assignment? Store an int or float in `hours_spent_on_hw`. If you worked in a group, report the average per person.
- Briefly describe how you used AI assistance and how you checked its contributions.
- (Optional) Which aspect of the assignment did you find most challenging?

In [59]:
```
# Report your hours here. Replace None with an int or float.
hours_spent_on_hw = 7
```

In [60]:
```
print("Notebook runtime, seconds:", round(time.time() - notebook_start, 1))
```

```
Notebook runtime, seconds: 669.4
```

Written response — Wrap-up

One conclusion that depended on the distinctions. In Ames, `Lot_Frontage` was selected in 91% of retuned bootstrap resamples, so it looked stable. In the independent inference sample its coefficient is −0.003 with an interval spanning zero (classical p = 0.74), and shuffling it leaves held-out error unchanged. Selection stability, coefficient evidence and predictive importance answer different questions, and only the separate analyses showed that "selected often" is not "important" or "associated with price". Similarly, the global permutation p-value (0.005) shows that the feature set matters as a whole, not which features.

One remaining limitation. All Ames conclusions rest on one fixed split, on neighborhood-median frontage imputation that assumes missingness is close to MAR (Q1.2 showed it clusters by neighborhood and lot type, and MNAR cannot be ruled out), and on independent houses. For the cookie data only 39 calibration spectra were available and the stability analysis held the PCA basis fixed, so uncertainty in the component directions is only partly reflected and everything beyond PCs 1–6 remains uninterpretable.

AI use. We used AI (Codex and Claude) only for brainstorming and code-reviewing. All code pieces and written responses were drafted by us. We checked AI contributions by running the test cases and reviewing every line of code.